PORTFOLIO

SLMAN AL-ODANI

NHL STENDEN EMMEN
Computer Science
Student Number : 5638364
Number of EC's : 240
Slman.al-odani@student.nhlstenden.com
2025-2029

Introduction:

As a Computer Science student, this portfolio reflects the journey I've taken to explore, build, and understand the digital world. From foundational programming concepts to hands-on projects, each section showcases not just what I've learned, but how I've applied it. Whether it's solving problems through code, analyzing systems, or experimenting with new technologies, I've aimed to approach each challenge with curiosity and persistence. This collection is a snapshot of my growth and a starting point for where I hope to go next.

My GitHub name is slmanalodani

Table of Contents

Introduction: 1
Computer science assignments: 3
Week 6: Python: 3
Assignment 1: Hello, Mario-less and cash 3
Assignment 2: Readability 6
Assignment 3: DNA 7
Week 7: SQL: 9
Assignment 1: songs 10
Assignment 2: Movies 12
Assignment 3: Fiftyville 17
Assignment: Blackboard 20
Week 8: HTML, CSS, JavaScript: 24
Assignment 1: Trivia 24
Assignment 2: Homepage 31
Assignment 3: Blackboard 46
Week 9: Flask: 73
Assignment 1: Birthdays 74
Assignment 2: Finance 77
Embedded Systems assignments: 92
Basic movements (functions): 92
Object avoidance: 95
Stay on Track (Arrays): 100
Midterm Race: 108
Test Day: 108
Professional skills: 125
Reflection on the project: 125

Computer science assignments:

Week 6: Python:

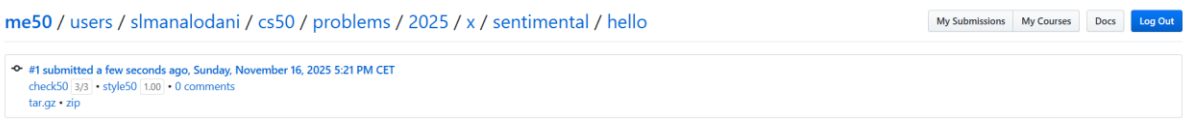
Assignment 1: Hello, Mario-less and cash.

Convert the codes from C to python for the above programs

Codes:

1) Hello.

```
name = input("What is your name? ")
print(f"hello, {name}")
```



https://submit.cs50.io/check50/1f17b52c860dca49d88c604a26972ca33b71ee62

2) Mario-less.

```
from cs50 import get_int
```

```python
def main():

    # ask the user for height and keep asking for a number between 1 and 8
    while True:
        n = get_int("Height: ")
        if 1 <= n <= 8:
            break

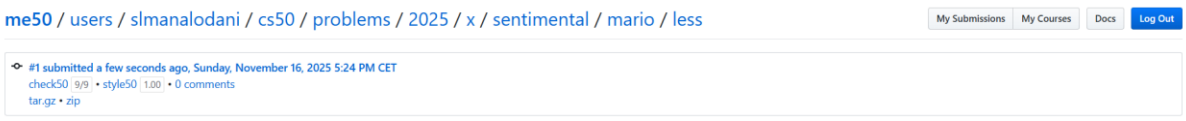
    for i in range(n):
        numberspaces = n - 1 - i
        numberbricks = i + 1
        print_row(numberspaces, numberbricks)


def print_row(spaces, bricks):
    # print spaces
    for i in range(spaces):
        print(" ", end="")  # to avoid new lines with end=""

    # print bricks
    for i in range(bricks):
        print("#", end="")

    print()  # after row printed move to next line


main()
```



https://submit.cs50.io/check50/7f657f122b8c0cb6b978f4680d5959e58b324697

3) Cash.

```python
from cs50 import get_float


def main():
    # Ask the user for change owed
    while True:
```

```
        dollars = get_float("Change: ")
        if dollars > 0:
            break

    # Convert dollars to cents
    cents = round(dollars * 100)

    quarters = cents // 25  # integer division with // no decimal
    cents %= 25

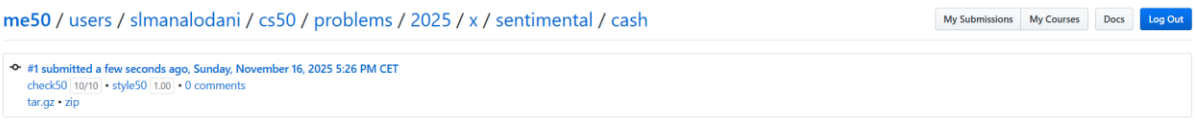
    dimes = cents // 10
    cents %= 10

    nickels = cents // 5
    cents %= 5

    pennies = cents

    totalNumberCoins = quarters + dimes + nickels + pennies
    print(f"{totalNumberCoins}")


main()
```



https://submit.cs50.io/check50/6b7e703c22caeb89b7863c7d5e516706a4e72174

Assignment 2: Readability

Convert the C program Readability from period 1 to python.

Code:

```
def main():
    text = input("Text: ")
    letters = count_letters(text)
    words = count_words(text)
    sentences = count_sentences(text)
```

```
    L = (letters / words) * 100
    S = (sentences / words) * 100
    index = 0.0588 * L - 0.296 * S - 15.8

    grade = round(index)
    if grade < 1:
        print("Before Grade 1")

    elif grade >= 16:
        print("Grade 16+")

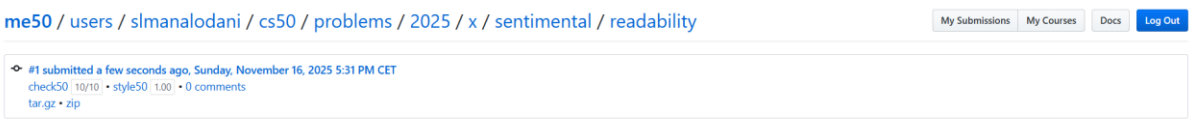
    else:
        print(f"Grade {grade}")


def count_letters(text):
    # adds 1 for every character that passes the isalpha() test.
    return sum(1 for char in text if char.isalpha())


def count_words(text):
    return len(text.split())  # splits the string into a list of words using
spaces as separators.
    # len() counts how many items are in that list


def count_sentences(text):
    return sum(1 for char in text if char in ['.', '!', '?'])


main()
```



https://submit.cs50.io/check50/1139cbc9654f9f27894e5345d0027477cd92150e

Assignment 3: DNA

In a file called `dna.py` in a folder called `dna`, implement a program that identifies to whom a sequence of DNA belongs.

Code:

```
import csv
import sys


def main():

    # Check for command-line usage
    if len(sys.argv) != 3:
        print("Incorrect number of command-line arguments")
        sys.exit(1)

    # Initialize an empty list to store DNA profiles from the database
    database = []

    # Open the CSV file containing the DNA database
    with open(sys.argv[1]) as file:
        # Create a DictReader to analyze each row as a dictionary using the
header row as keys
        reader = csv.DictReader(file)

        # Iterate over each row in the CSV file (each person's DNA profile)
        for person in reader:
            # Add the person's profile dictionary to the database list
            database.append(person)

    # Read DNA sequence file into a variable
    with open(sys.argv[2]) as file:
        # reads the entire contents of the file as one big string.
        dna_sequence = file.read()

    str_counts = {}  # dictionary store STR counts

    for str_name in database[0].keys():  # loop throug each all the column
headers in the database
        if str_name != "name":  # skip the name coloumn since it is not STR
            # use longest_match() to find the longest STR
            str_counts[str_name] = longest_match(dna_sequence, str_name)

    # Check database for matching profiles
```

```
    for person in database:  # loop through each person
        match = True  # Assume a match until proven otherwise
        for str_name in str_counts:  # compare each STR count
            # convert database values to int before comparing bc longest_match
return int so we cant compare string with int
            if int(person[str_name]) != str_counts[str_name]:
                match = False
                break
        if match:
            print(person["name"])
            return

    # if no profiles matched, print no match
    print("No match")


def longest_match(sequence, subsequence):
    """Returns length of longest run of subsequence in sequence."""

    # Initialize variables
    longest_run = 0
    subsequence_length = len(subsequence)
    sequence_length = len(sequence)

    # Check each character in sequence for most consecutive runs of
subsequence
    for i in range(sequence_length):

        # Initialize count of consecutive runs
        count = 0

        # Check for a subsequence match in a "substring" (a subset of
characters) within sequence
        # If a match, move substring to next potential match in sequence
        # Continue moving substring and checking for matches until out of
consecutive matches
        while True:

            # Adjust substring start and end
            start = i + count * subsequence_length
            end = start + subsequence_length

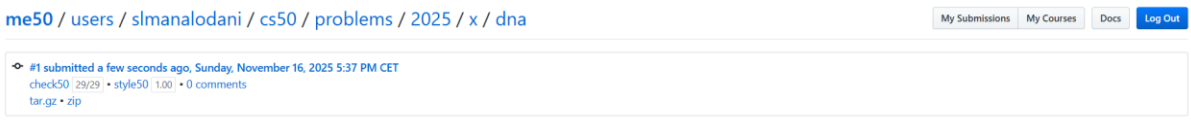
            # If there is a match in the substring
            if sequence[start:end] == subsequence:
                count += 1
```

```
            # If there is no match in the substring
            else:
                break

        # Update most consecutive matches found
        longest_run = max(longest_run, count)

    # After checking for runs at each character in seqeuence, return longest
run found
    return longest_run


main()
```



https://submit.cs50.io/check50/664c0478a337c55dc1d66ee9392354086ee5b05c

Week 7: SQL:

Assignment 1: songs

Write SQL queries to answer questions about a database of the 100 most-streamed songs on Spotify in 2018.

In 1.sql, write a SQL query to list the names of all songs in the database.

Your query should output a table with a single column for the name of each song.

```
SELECT name FROM songs;
```

In 2.sql, write a SQL query to list the names of all songs in increasing order of tempo.

Your query should output a table with a single column for the name of each song.

```
SELECT name
FROM songs
ORDER BY tempo;
```

In 3.sql, write a SQL query to list the names of the top 5 longest songs, in descending order of length.

Your query should output a table with a single column for the name of each song.

```
SELECT name
FROM songs
ORDER BY duration_ms DESC
LIMIT 5;
```

In 4.sql, write a SQL query that lists the names of any songs that have danceability, energy, and valence greater than 0.75.

Your query should output a table with a single column for the name of each song.

```
SELECT name
FROM songs
WHERE danceability > 0.75 AND energy > 0.75 AND valence > 0.75;
```

In 5.sql, write a SQL query that returns the average energy of all the songs.

Your query should output a table with a single column and a single row containing the average energy.

```
SELECT AVG(energy)
FROM songs;
```

In 6.sql, write a SQL query that lists the names of songs that are by Post Malone.

Your query should output a table with a single column for the name of each song.

You should not make any assumptions about what Post Malone's artist_id is.

```
SELECT name
FROM songs
WHERE artist_id = (SELECT id
FROM artists
WHERE name = 'Post Malone');
```

In 7.sql, write a SQL query that returns the average energy of songs that are by Drake.

Your query should output a table with a single column and a single row containing the average energy.

You should not make any assumptions about what Drake's artist_id is.

```
SELECT AVG(energy)
FROM songs
JOIN artists ON songs.artist_id = artists.id
WHERE artists.name = 'Drake';
```

In 8.sql, write a SQL query that lists the names of the songs that feature other artists.

Songs that feature other artists will include "feat." in the name of the song.

Your query should output a table with a single column for the name of each song.

```
SELECT name
FROM songs
WHERE name LIKE '%feat.%';
```

1) If songs.db contains the top 100 songs of one listener from 2018, how would you characterize their audio aura?
2) Hypothesize about why the way you've calculated this aura might not be very representative of the listener. What better ways of calculating this aura would you propose?

The answers:

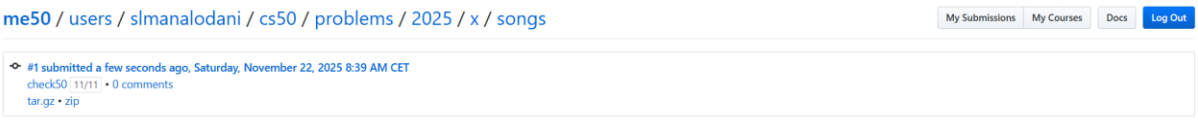
1. I measured the avg energy, danceability and valence.

```
SELECT AVG(energy), AVG(danceability), AVG(valence)
   ...> FROM songs;
```

| 0.65906 | 0.71646 | 0.484443 |

Based on that, I would say his audio aura is energetic, danceable, and emotionally balanced.

2. It only reflects their top 100 songs in 2018, not their full taste or later years. A better way to measure the aura, I think multi-year data: Include listening history across several years to capture evolving taste.



https://submit.cs50.io/check50/25d57868d5ac391c0fad02e5920dcc95509cf629

Assignment 2: Movies

Provided to you is a file called movies.db, a SQLite database that stores data from IMDb about movies, the people who directed and starred in them, and their ratings. Write SQL queries to answer questions about this database of movies.

In 1.sql, write a SQL query to list the titles of all movies released in 2008.

Your query should output a table with a single column for the title of each movie:

```
SELECT title
FROM movies
WHERE year = 2008;
```

In 2.sql, write a SQL query to determine the birth year of Emma Stone.

Your query should output a table with a single column and a single row (not counting the header) containing Emma Stone's birth year.

You may assume that there is only one person in the database with the name Emma Stone.

```
SELECT birth
FROM people
where name = 'Emma Stone';
```

In 3.sql, write a SQL query to list the titles of all movies with a release date on or after 2018, in alphabetical order.

Your query should output a table with a single column for the title of each movie.

Movies released in 2018 should be included, as should movies with release dates in the future.

```
SELECT title
FROM movies
WHERE year >= 2018
```

```
ORDER BY title;
```

In 4.sql, write a SQL query to determine the number of movies with an IMDb rating of 10.0.

Your query should output a table with a single column and a single row (not counting the header) containing the number of movies with a 10.0 rating.

```
SELECT COUNT(title)
FROM movies
WHERE id IN (SELECT movie_id
FROM ratings
WHERE rating = 10);
```

In 5.sql, write a SQL query to list the titles and release years of all Harry Potter movies, in chronological order.

Your query should output a table with two columns, one for the title of each movie and one for the release year of each movie.

You may assume that the title of all Harry Potter movies will begin with the words “Harry Potter”, and that if a movie title begins with the words “Harry Potter”, it is a Harry Potter movie.

```
SELECT title, year
FROM movies
WHERE title LIKE 'Harry Potter%'
ORDER BY year;
```

In 6.sql, write a SQL query to determine the average rating of all movies released in 2012.

Your query should output a table with a single column and a single row (not counting the header) containing the average rating.

```sql
SELECT AVG(rating)
FROM ratings
JOIN movies ON ratings.movie_id = movies.id
WHERE movies.year = 2012;
```

In 7.sql, write a SQL query to list all movies released in 2010 and their ratings, in descending order by rating. For movies with the same rating, order them alphabetically by title.

Your query should output a table with two columns, one for the title of each movie and one for the rating of each movie.

Movies that do not have ratings should not be included in the result.

```sql
SELECT title, rating
FROM movies
JOIN ratings ON movies.id = ratings.movie_id
WHERE year = 2010
ORDER BY ratings.rating DESC, movies.title;
```

In 8.sql, write a SQL query to list the names of all people who starred in Toy Story.

Your query should output a table with a single column for the name of each person.

You may assume that there is only one movie in the database with the title Toy Story.

```sql
SELECT name
FROM people
JOIN stars ON people.id = stars.person_id
JOIN movies ON stars.movie_id = movies.id
WHERE movies.title = 'Toy Story';
```

In 9.sql, write a SQL query to list the names of all people who starred in a movie released in 2004, ordered by birth year.

Your query should output a table with a single column for the name of each person.

People with the same birth year may be listed in any order.

No need to worry about people who have no birth year listed, so long as those who do have a birth year are listed in order.

If a person appeared in more than one movie in 2004, they should only appear in your results once.

```
SELECT DISTINCT name
FROM people
JOIN stars ON people.id = stars.person_id
JOIN movies ON stars.movie_id = movies.id
WHERE movies.year = 2004
ORDER BY people.birth;
```

In 10.sql, write a SQL query to list the names of all people who have directed a movie that received a rating of at least 9.0.

Your query should output a table with a single column for the name of each person.

If a person directed more than one movie that received a rating of at least 9.0, they should only appear in your results once.

```
SELECT DISTINCT name
FROM people
JOIN directors ON people.id = directors.person_id
JOIN movies ON directors.movie_id = movies.id
JOIN ratings ON movies.id = ratings.movie_id
WHERE ratings.rating >= 9;
```

In 11.sql, write a SQL query to list the titles of the five highest rated movies (in order) that Chadwick Boseman starred in, starting with the highest rated.

Your query should output a table with a single column for the title of each movie.

You may assume that there is only one person in the database with the name Chadwick Boseman.

```
SELECT title
FROM movies
JOIN ratings ON movies.id = ratings.movie_id
JOIN stars ON movies.id = stars.movie_id
JOIN people ON stars.person_id = people.id
WHERE people.name = 'Chadwick Boseman'
ORDER BY ratings.rating DESC
```

```
LIMIT 5;
```

In 12.sql, write a SQL query to list the titles of all movies in which both Bradley Cooper and Jennifer Lawrence starred.

Your query should output a table with a single column for the title of each movie.

You may assume that there is only one person in the database with the name Bradley Cooper.

You may assume that there is only one person in the database with the name Jennifer Lawrence.

```
SELECT title
FROM movies
JOIN stars ON movies.id = stars.movie_id
JOIN people ON stars.person_id = people.id
JOIN stars AS stars1 ON movies.id = stars1.movie_id
JOIN people AS people1 ON stars1.person_id = people1.id
WHERE people.name = 'Bradley Cooper' AND people1.name = 'Jennifer Lawrence';
```

In 13.sql, write a SQL query to list the names of all people who starred in a movie in which Kevin Bacon also starred.

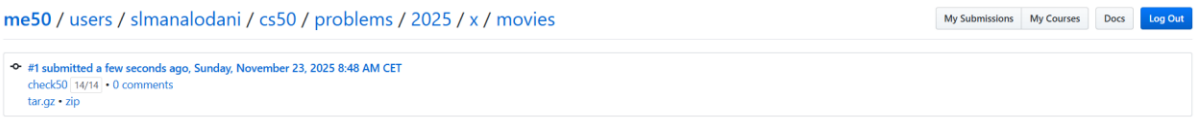
Your query should output a table with a single column for the name of each person.

There may be multiple people named Kevin Bacon in the database. Be sure to only select the Kevin Bacon born in 1958.

Kevin Bacon himself should not be included in the resulting list.

```
SELECT people.name
FROM people
JOIN stars ON people.id = stars.person_id
```

```
JOIN movies ON stars.movie_id = movies.id
JOIN stars AS stars1 ON movies.id = stars1.movie_id
JOIN people AS people1 ON stars1.person_id = people1.id
WHERE people1.name = 'Kevin Bacon' AND people1.birth = '1958'
AND NOT(people.name = 'Kevin Bacon' AND people.birth = 1958);
```



https://submit.cs50.io/check50/59d70d531d60a418e3cfcde324f9192458c4d6ac

Assignment 3: Fiftyville

The CS50 Duck has been stolen! The town of Fiftyville has called upon you to solve the mystery of the stolen duck. Authorities believe that the thief stole the duck and then, shortly afterwards, took a flight out of town with the help of an accomplice. Your goal is to identify:

Who the thief is,

What city the thief escaped to, and

Who the thief's accomplice is who helped them escape

All you know is that the theft took place on July 28, 2024 and that it took place on Humphrey Street.

How will you go about solving this mystery? The Fiftyville authorities have taken some of the town's records from around the time of the theft and prepared a SQLite database for you, fiftyville.db, which contains tables of data from around the town. You can query that table using SQL SELECT queries to access the data of interest to you. Using just the information in the database, your task is to solve the mystery.

The log.sql:

```
-- Keep a log of any SQL queries you execute as you solve the mystery.
-- All you know is that the theft took place on July 28, 2024 and that it took
place on Humphrey Street.
-- i am going to look into the crime scenes report description in july 28,2024
SELECT description
FROM crime_scene_reports
WHERE year = 2024 AND day = 28 AND month = 7
AND street = 'Humphrey Street';
```

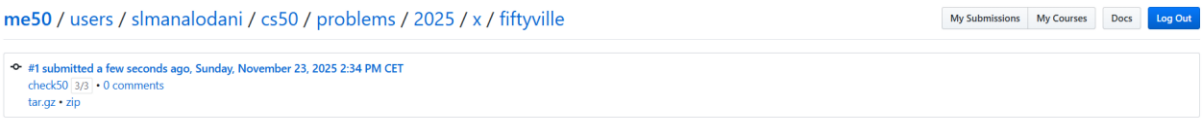
```sql
-- three main infofrom the report it took place at 10:15am
-- we have three witnesses
-- all witnessess mentions bakery
-- i am going to go through the intreviews at that day
SELECT transcript,
FROM interviews
WHERE year = 2024 AND day = 28 AND month = 7;
-- i found that the theft get into a car in the bakery parking lot withen ten
minutes of the theft
-- the thief earliar that morning on Leggett Street was withdrawing money
-- the one who helped him is going to buy the flight ticket
--he talked to him by phone when he left the bakery.for less than a minute
-- he was planning to get out of the fiftywille next day
-- i am going to check bakery logs to see what is the licence plate for the
thief car
SELECT activity, license_plate, minute
FROM bakery_security_logs
WHERE year = 2024 AND month = 7 AND day = 28
AND hour = 10 AND minute >= 10;
-- i got a bunch of licence plate withen 10 minute of the theft
-- exited that parking lot of the bakery
-- i am going to check the atm transactions
SELECT *
FROM atm_transactions
WHERE year = 2024 AND day = 28 AND month = 7
AND atm_location = 'Leggett Street'
AND transaction_type = 'withdraw';
-- i got a couple of accounts numbers that did withdraws at that day
-- i am going to check the phone call that day and the one that lasted
-- less than a minute
SELECT caller, receiver
FROM phone_calls
WHERE year = 2024 and day = 28 AND month = 7
AND duration < 60;
-- i am going to get the name and phone number of people exited the
-- parking lot withen 10 minute of the theft
SELECT name, phone_number, license_plate
FROM people
WHERE license_plate IN (
    '5P2BI95',
    '94KL13X',
    '6P58WS2',
    '4328GD8',
    'G412CB7',
    'L93JTIZ',
    '322W7JE'
);
```

```sql
-- i got the names and phone numbers of these people
-- which are the ones who exited the parking of the bakery
-- withen 10 minute of theft
-- i compared there phonenumbers with the phone numbers of poeple
-- who talked for less than a minute in that day
-- so now i have 3 poeple which are sofia , Diana, Bruce
-- i am going to see if one of them did make a withdraw in that day
-- i am going to get there account numbers
SELECT account_number, people.name, people.passport_number
FROM bank_accounts
JOIN people ON bank_accounts.person_id = people.id
WHERE people.license_plate IN (
 '94KL13X',
 'G412CB7',
 '322W7JE');
-- now I am left with Bruce, Diana
-- i checked the accounts which made a withdraw at that day
-- from Leggett Street and the two of them did that
-- since now i have there passport numbers. I am
-- going to check if they are taking on 29 july 2024
-- a flight out of Fiftyville
-- first lets see what is the id for the Fiftyville airport and what
SELECT id
FROM airports
WHERE city = 'Fiftyville';
-- now lets see if Bruce, Diana if there are taking a flight on 29 in the
morning
-- by checking if their passport numbers are in the flights out of Fiftyville
SELECT airports.city, passengers.passport_number,
flights.destination_airport_id, flights.origin_airport_id, flights.hour
FROM airports
JOIN flights ON airports.id = flights.destination_airport_id
JOIN passengers ON flights.id = passengers.flight_id
WHERE flights.day = 29 AND flights.year = 2024
AND flights.month = 7
AND flights.origin_airport_id = 8
AND (passengers.passport_number = 5773159633 OR passengers.passport_number =
3592750733);
-- i think the theif is Bruce since his flight is to NEW YORK City at 8 in the
morning
-- now lets find the name of the person who he talked too for less than a
minute
-- since i have the number of that person i can find his name
SELECT name
FROM people
WHERE phone_number = '(375) 555-8161';
-- so the helper name is Robin
```

The THIEF is: Bruce

The city the thief ESCAPED TO: New York City

The ACCOMPLICE is: Robin



https://submit.cs50.io/check50/50eb6b06c76a84bae4c7af320c22744cfc4c2458

Assignment: Blackboard

1) Write an SQL query to add your own fictional movies to the movies database.

```
INSERT INTO movies (id, title, year)
VALUES (90000000, 'Journey To The East', 2000),
(90000001, 'Journey To The West', 2001),
(90000002, 'Journey To The South', 2002),
(90000003, 'Journey To The North', 2003);
```

I needed unique id for the movies, so I checked the max id in the movies database.

```
SELECT MAX(id)
FROM movies;
```

2) Write an SQL query to add fictional people to the people's table.

```
INSERT INTO people (id, name, birth)
VALUES (80000000, 'John Star', 1990),
(80000001, 'Sam Star', 1991),
(80000002, 'Viki Star', 1992),
(80000003, 'Sara Star', 1993);
```

3) Write an SQL query linking the first three people of question 2 as stars of the film journey to the north.

```
INSERT INTO stars (movie_id, person_id)
VALUES (90000003, 80000000),
(90000003, 80000001),
(90000003, 80000002);
```

4) Write an SQL query linking Sara Star as director for all the made-up films.

```
INSERT INTO directors (movie_id, person_id)
VALUES (90000000, 80000003),
(90000001, 80000003),
(90000002, 80000003),
(90000003, 80000003);
```

5) Write an SQL query where all the made-up films are rated 8.0

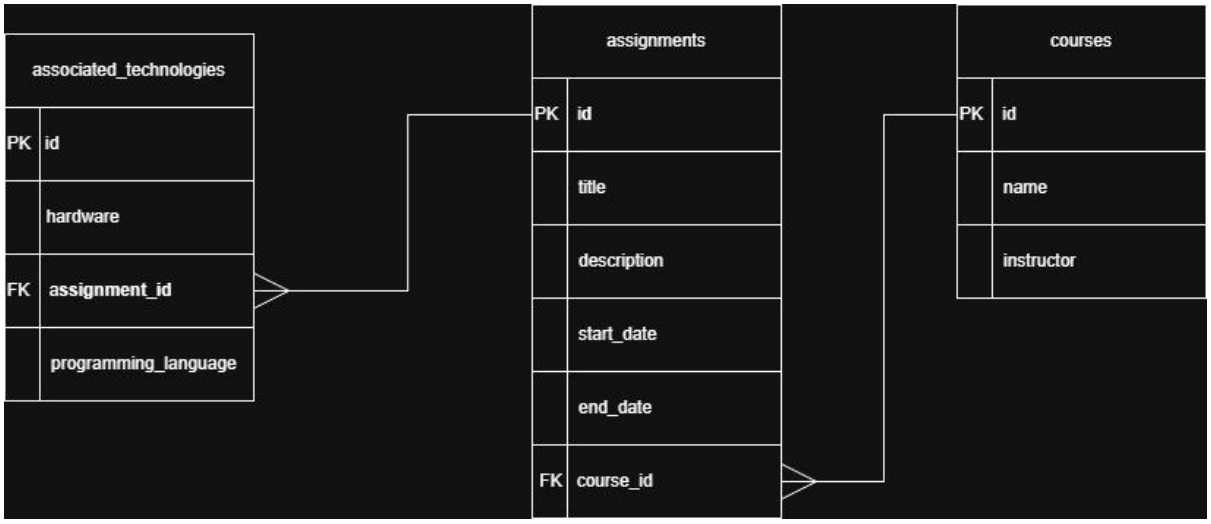
```
INSERT INTO ratings (movie_id, rating, votes)
VALUES (90000000, 8.0, 1000),
(90000001, 8.0, 1000),
(90000002, 8.0, 1000),
(90000003, 8.0, 1000);
```

6) Write an SQL query where the film Journey to the east with a rating of 8.0 becomes a 10.0.

```
UPDATE ratings
SET rating = 10.0
WHERE movie_id = 90000000;
```

7) Create a Portfolio Database with SQLite and ERD Diagram

The ERD Diagram:



I have here the python code to make the database and insert some data into it

```
import sqlite3

#----------Connect to SQlite database(and create file)---------------
conn = sqlite3.connect("portfolio.db")
cursor = conn.cursor() # cursor how u send sql commands to the db.

# ----------Schema Setup--------------------

# create the assignments table
cursor.execute("""
CREATE TABLE IF NOT EXISTS assignments (
    id INTEGER PRIMARY KEY,
    title TEXT NOT NULL,
    description TEXT,
    start_date DATE,
    end_date DATE,
    course_id INTEGER NOT NULL,
    FOREIGN KEY (course_id) REFERENCES courses(id)
)
""")

# create the associated_technologies table
cursor.execute("""
CREATE TABLE IF NOT EXISTS associated_technologies (
    id INTEGER PRIMARY KEY,
    hardware TEXT,
    assignment_id INTEGER NOT NULL,
    programming_language TEXT,
    FOREIGN KEY (assignment_id) REFERENCES assignments(id)
)
""")
```

```
# create the courses table
cursor.execute("""
CREATE TABLE IF NOT EXISTS courses (
    id INTEGER PRIMARY KEY,
    name TEXT,
    instructor TEXT
)
""")

#-------------DATA INSERTS---------------
cursor.execute(
"INSERT INTO courses (id, name, instructor) VALUES (?, ?, ?)",
(1, "Embedded Systems", "Marcel Baron"))

cursor.execute(
"INSERT INTO assignments (id, title, description, start_date, end_date,
course_id) VALUES (?, ?, ?, ?, ?, ?)",
(1, "Basic Movements", "program the relay bot for basic movements", "2025-11-
10", "2025-11-14", 1))

cursor.execute(
"INSERT INTO associated_technologies (id, hardware, assignment_id,
programming_language) VALUES (?, ?, ?, ?)",
(1, "arduino", 1, "C++"))

#---------commit the changes-----------
conn.commit()
```

Reflection:

I designed the portfolio database with three main tables: **courses**, **assignments**, and **associated_technologies**. This structure follows a normalized approach where each table has a clear purpose and avoids duplication of data.

- **Courses table**: Stores high-level information about each course (name, instructor). This prevents repeating course details for every assignment.
- **Assignments table**: Each assignment is linked to a course through a foreign key (`course_id`). This relationship ensures that assignments are always tied to a valid course, maintaining referential integrity.
- **Associated_technologies table**: This table connects specific hardware and programming languages to assignments via the foreign key (`assignment_id`).

This design allows flexibility, since multiple technologies can be linked to a single assignment without cluttering the assignments table.

Why this design is efficient

- **Normalization**: By splitting data into separate tables, redundancy is minimized. For example, instructor names aren't repeated in every assignment row.
- **Scalability**: New courses, assignments, or technologies can be added without changing the schema.
- **Integrity**: Foreign keys enforce valid relationships — no assignment can exist without a course, and no technology can exist without an assignment.
- **Clarity**: The ERD clearly shows a one-to-many relationship from courses → assignments, and another one-to-many from assignments → technologies. This makes the database easy to understand and teach.

Contribution to efficient storage

This design ensures that data is stored in a modular way. Queries can easily join tables to show the full picture (course → assignment → technology), while updates remain simple and localized. For example, changing an instructor's name only requires updating one row in the courses table, not every assignment.

Week 8: HTML, CSS, JavaScript:

Assignment 1: Trivia

Design a webpage using HTML, CSS, and JavaScript to let users answer trivia questions.

In index.html, add beneath "Part 1" a multiple-choice trivia question of your choosing with HTML.

You should use an h3 heading for the text of your question.

You should have one button for each of the possible answer choices. There should be at least three answer choices, of which exactly one should be correct.

Using JavaScript, add logic so that the buttons change colors when a user clicks on them.

If a user clicks on a button with an incorrect answer, the button should turn red and text should appear beneath the question that says "Incorrect".

If a user clicks on a button with the correct answer, the button should turn green and text should appear beneath the question that says "Correct!".

In index.html, add beneath “Part 2” a text-based free response question of your choosing with HTML.

You should use an h3 heading for the text of your question.

You should use an input field to let the user type a response.

You should use a button to let the user confirm their answer.

Using JavaScript, add logic so that the text field changes color when a user confirms their answer.

If the user types an incorrect answer and presses the confirmation button, the text field should turn red and text should appear beneath the question that says “Incorrect”.

If the user types the correct answer and presses the confirmation button, the input field should turn green and text should appear beneath the question that says “Correct!”.

Code: HTML.

```
<!DOCTYPE html>
<html lang="en">
<head>
    <meta charset="UTF-8">
    <title>Trivia!</title>
    <!-- Google font -->
    <link
href="https://fonts.googleapis.com/css2?family=Montserrat:wght@500&disp
lay=swap" rel="stylesheet">
    <!-- Your CSS file -->
    <link href="styles.css" rel="stylesheet">
    <script>
        // Run this code after the page loads
        document.addEventListener("DOMContentLoaded", function() {

            // ===== Part 1: Multiple Choice =====
            // Get all the multiple-choice buttons
            let mcButtons = document.querySelectorAll(".options
button");
            // Get the feedback paragraph under the question
            let mcFeedback =
document.querySelector(".section .feedback");

            // Loop through each button
            mcButtons.forEach(function(button) {
```

```
                button.addEventListener("click", function() {
                    // Reset all buttons (remove old classes)
                    mcButtons.forEach(b => { //  b=> :shorter way to
write function(b)
                        b.classList.remove("correct-answer",
"incorrect-answer");
                    });

                    // Check if this button is correct
                    if (button.dataset.answer === "correct") { //
(=== :strict equality perfect for strings)
                        button.classList.add("correct-answer"); // add
green style
                        mcFeedback.textContent = "Correct!";
                        mcFeedback.style.color = "green";
                    } else {
                        button.classList.add("incorrect-answer"); //
add red style
                        mcFeedback.textContent = "Incorrect";
                        mcFeedback.style.color = "red";
                    }
                });
            });

            // ===== Part 2: Free Response =====
            let submitBtn = document.getElementById("submit-response");
            let freeInput = document.getElementById("free-response");
            let freeFeedback = document.getElementById("free-
feedback");

            submitBtn.addEventListener("click", function() {
                // Reset previous classes
                freeInput.classList.remove("correct-answer",
"incorrect-answer");

                // Get the user's answer
                let answer = freeInput.value.trim().toLowerCase(); //
trim() removes spaces, toLowerCase() for case insensitivity

                // Check if the answer is "paris"
                if (answer === "paris") {
```

```
                    freeInput.classList.add("correct-answer"); // green
style
                    freeFeedback.textContent = "Correct!";
                    freeFeedback.style.color = "green";
                } else {
                    freeInput.classList.add("incorrect-answer"); // red
style
                    freeFeedback.textContent = "Incorrect";
                    freeFeedback.style.color = "red";
                }
            });
        });
    </script>
</head>
<body>
    <div class="header">
        <h1>Trivia!</h1>
    </div>

    <div class="container">
        <!-- Part 1: Multiple Choice -->
        <div class="section">
            <h2>Part 1: Multiple Choice</h2>
            <hr>
            <h3>Which element has the chemical symbol "O"?</h3>
            <div class="options">
                <!-- Each button has data-answer to mark correct/wrong
-->
                <button data-answer="wrong">Gold</button>
                <button data-answer="correct">Oxygen</button>
                <button data-answer="wrong">Silver</button>
            </div>
            <p class="feedback"></p>
        </div>

        <!-- Part 2: Free Response -->
        <div class="section">
            <h2>Part 2: Free Response</h2>
            <hr>
            <h3>What is the capital of France?</h3>
            <!-- Input field for user to type answer -->
```

```
            <input type="text" id="free-response" placeholder="Type
your answer here">
            <!-- Button to submit the answer -->
            <button id="submit-response">Submit</button>
            <!-- Feedback text will appear here -->
            <p id="free-feedback"></p>
        </div>
    </div>
</body>
</html>
```

Style.CSS:

```
body {
    background-color: #fff;
    color: #000;
    font-size: 1rem; /* 1rem = 16px by default in browsers */
    font-weight: 400; /* normal weight */
    line-height: 1.5; /* 1.5 times the font size */
    margin: 0; /* remove default margin */
    text-align: left;
}

.container {
    margin-left: auto; /* centers the container */
    margin-right: auto;
    padding-left: 15px; /* add space inside the container so text dont touch
the edges*/
    padding-right: 15px;
}

.header {
    background-color: #477bff;
    color: #fff;
    margin-bottom: 2rem;
    padding: 2rem 1rem;
    text-align: center;
}

.section {
    padding: 0.5rem 2rem 1rem 2rem; /* top right bottom left */
}
```

```css
.section:hover {
    background-color: #f5f5f5;
    transition: color 2s ease-in-out, background-color 0.15s ease-in-out;
/*quick smooth fade when hovering */
}

h1 {
    font-family: 'Montserrat', sans-serif; /* font stack: first choice then
fallback */
    font-size: 48px;
}

button {
    background-color: #d9edff;
    border: 1px solid transparent;
    border-radius: 0.25rem;
    font-size: 0.95rem;
    font-weight: 400;
    line-height: 1.5;
    padding: 0.375rem 0.75rem;
    text-align: center;
    transition: color 0.15s ease-in-out, background-color 0.15s ease-in-out,
border-color 0.15s ease-in-out, box-shadow 0.15s ease-in-out;
    vertical-align: middle;
}


input[type="text"] {
    line-height: 1.8;
    width: 25%;
}

input[type="text"]:hover {
    background-color: #f5f5f5;
    transition: color 2s ease-in-out, background-color 0.15s ease-in-out;
}

/* correct answer styling */
.correct-answer {
    background-color: lightgreen !important; /* !important flag forces this
style to override other background-color*/
    color: #000; /* set the text color to black in hex*/
}
/* Incorrect answer styling */
.incorrect-answer {
    background-color: lightcoral !important;
```

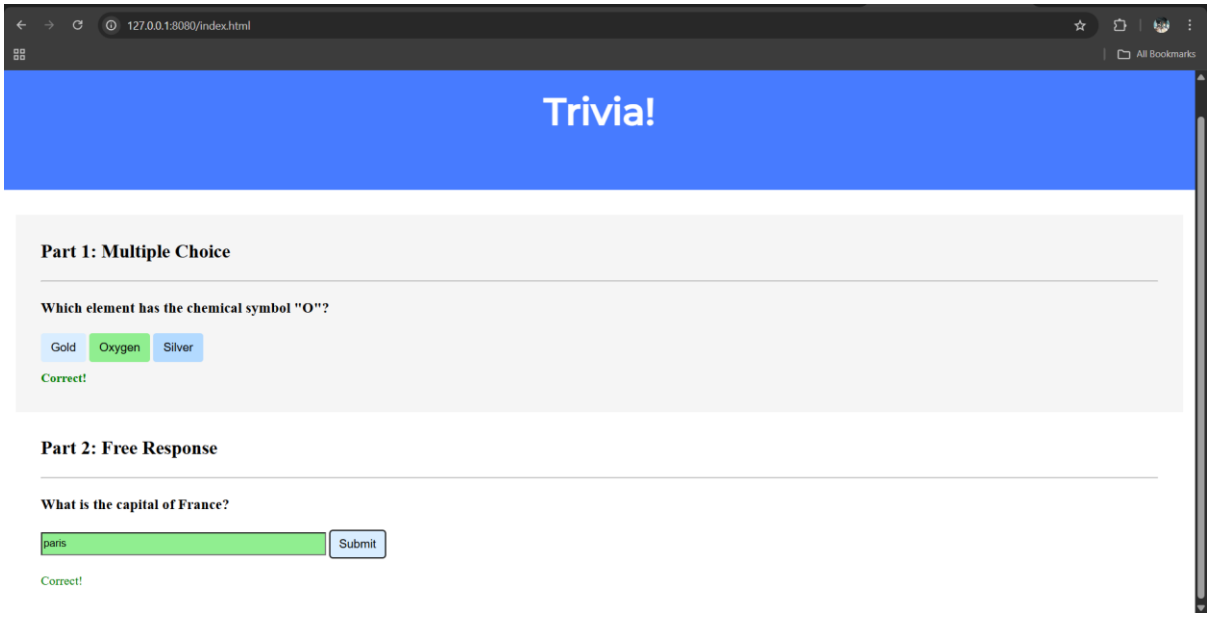
```
    color: #000;
}
/* Feedback text styling*/
.feedback {
    font-weight: 600; /* makes the text semi bold */
    margin-top: 0.5rem; /* adds a little vertical space above the feedback.
0.5rem : half the size of the root font*/
}

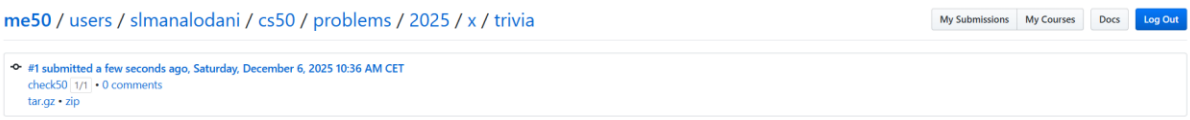
.options button:hover {
    background-color: #b3daff;
    cursor: pointer;
}

input[type="text"]:focus {
    outline: none; /* remove default outline */
    border: 1px solid #477bff; /* blue border on focus */
    box-shadow: 0 0 5px rgba(71, 123, 255, 0.5);  /*
    0 0 5px → means no horizontal or vertical offset, just a blur of 5px.
    rgba(71, 123, 255, 0.5) → semi-transparent blue color (the last number 0.5
= 50% opacity).
    subtle blue glow */
}

#submit-response:hover { /* id selector*/
    background-color: #b3daff;
    cursor: pointer;
}
```

Image:





https://submit.cs50.io/check50/02f4c79c7bd076c8e1aa69a51a0adcf3c55fd401

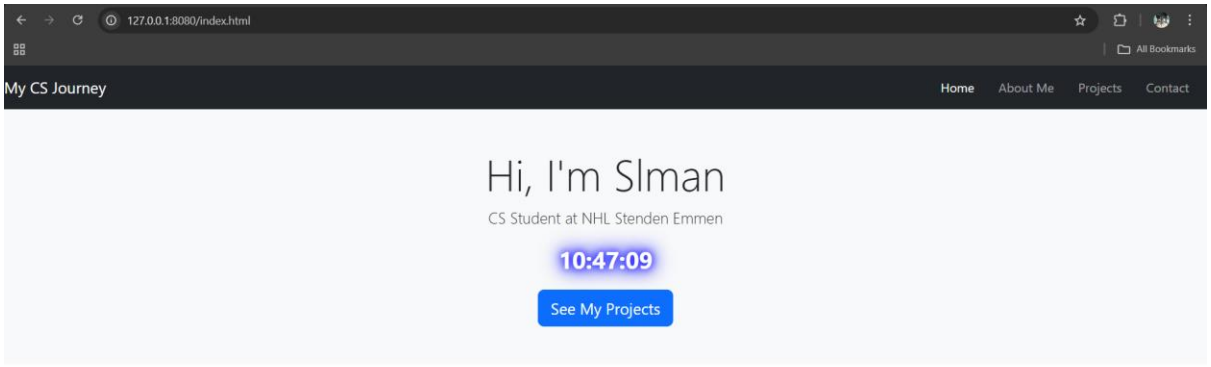
Assignment 2: Homepage

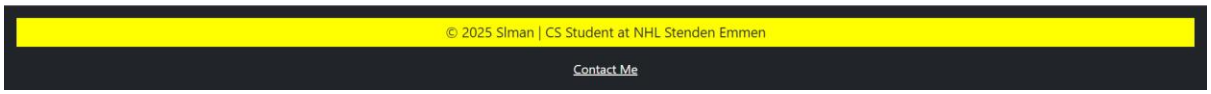
Implement a website in your homepage directory that meets the following requirements:

1. **Pages and Navigation**
    a. Your site must contain at least **four different `.html` pages**, one of which must be named `index.html` (the main page).
    b. It should be possible to navigate from **any page to any other page** using hyperlinks.
2. **HTML Requirements**
    a. Use at least **ten (10) distinct HTML tags** besides `<html>`, `<head>`, `<body>`, and `<title>`.
    b. Repeated use of the same tag (e.g., multiple `<p>` elements) counts only once toward the total.
3. **Bootstrap Integration**
    a. Integrate one or more features from **Bootstrap** into your site.
    b. Include the Bootstrap CSS and JS links in the `<head>` of your pages.
    c. Use Bootstrap components such as a **navigation bar, grid system, cards, or buttons** to enhance your design.

4. **CSS Requirements**
    a. Create a custom stylesheet file named `styles.css`.
    b. Use at least **five (5) different CSS selectors** (e.g., tag, class, ID).
    c. Apply at least **five (5) different CSS properties** (e.g., `font-size`, `margin`, `padding`, `color`, `background-color`, `border`, `box-shadow`).
5. **JavaScript Integration**
    a. Add at least one **interactive feature** using JavaScript.
    b. Examples include:
        i. Displaying an alert when a button is clicked.
        ii. Adding an effect at a recurring interval.
        iii. Making forms, dropdowns, or navigation interactive.
6. **Responsiveness**
    a. Ensure your site looks good on both **mobile devices** and **laptops/desktops**.
    b. Use Bootstrap's responsive grid system and classes to achieve this.
7. **Specification File**
    a. Create a text file named `specification.txt`.
    b. In this file, list:
        i. The **10 HTML tags** you used.
        ii. The **5 CSS properties** you applied.
        iii. A brief one-sentence description of how you integrated **JavaScript** and **Bootstrap**.

Image and code for main homepage:






```
<!DOCTYPE html>

<html lang="en">

    <head>
        <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
rel="stylesheet" integrity="sha384-
QWTKZyjpPEjISv5WaRU9OFeRpok6YctnYmDr5pNlyT2bRjXh0JMhjY6hW+ALEwIH"
crossorigin="anonymous">
        <script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min
.js" integrity="sha384-
YvpcrYf0tY3lHB60NNkmXc5s9fDVZLESaAA55NDzOxhy9GkcIdslK1eN7N6jIeHz"
crossorigin="anonymous"></script>
        <link href="styles.css" rel="stylesheet">
        <title>My Webpage</title>
    </head>

    <body>
        <!-- Navigation bar-->
        <nav class="navbar navbar-expand-lg navbar-dark bg-dark"> <!-- the nav
bar stay collabse until screen is large-->
            <!--my title/brand logo-->
            <a class="navbar-brand" href="index.html">My CS Journey</a>
            <!-- Hamburger button (appers on mobile)-->
            <button class="navbar-toggler" type="button" data-bs-
toggle="collapse" data-bs-target="#navbarNav" aria-controls="navbarNav" aria-
expanded="false" aria-label="Toggle navigation">
                <!-- Icon for the hamburger menu-->
```

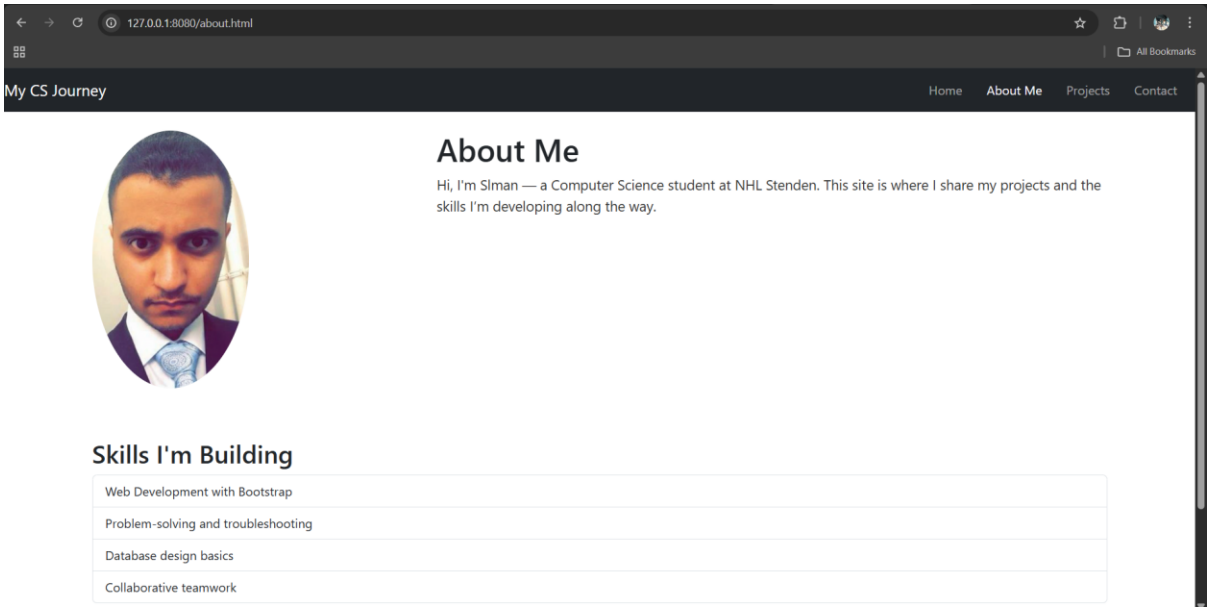
```
                <span class="navbar-toggler-icon"></span> <!--icon inside the
button / inline container-->
            </button>
            <!-- collasible menu items-->
            <div class="collapse navbar-collapse" id="navbarNav">
                <ul class="navbar-nav ms-auto"> <!--ms-auto pushes links to
the right-->
                    <!--home link (active on this page)-->
                    <li class="nav-item">
                        <a class="nav-link active" href="index.html">Home</a>
                    </li>
                    <!--About me link-->
                    <li class="nav-item">
                        <a class="nav-link" href="about.html">About Me</a>
                    </li>
                    <!--Projects link-->
                    <li class="nav-item">
                        <a class="nav-link" href="projects.html">Projects</a>
                    </li>
                    <!--Contacts link-->
                    <li class="nav-item">
                        <a class="nav-link" href="contact.html">Contact</a>
                    </li>
                </ul>
            </div>
        </nav>
        <main class="flex-grow-1"> <!--flex-grow-1 means:
                                    If the parent container uses display: flex,
this element will expand to take up available space.
                                    It grows proportionally compared to other
flex items.
                                    Useful when you want the main content to
stretch and fill the page between header and footer.-->
            <div class="jumbotron text-center bg-light p-5"> <!-- jumbotron is
a Bootstrap component for a prominent header section-->
                <h1 class="display-4">Hi, I'm Slman</h1>
                <p class="lead">CS Student at NHL Stenden Emmen</p> <!-- lead
is typography utility class make the text stand out-->
                <!--Live Clock-->
                <p id="clock" class="lead"></p>
                <a class="btn btn-primary btn-lg" href="projects.html"
role="button">See My Projects</a> <!-- btn-lg makes the button larger
                btn_primary for the color blue - the role makes the reader
treats this link as button-->
            </div>
```

```
            <!-- Page content -->
            <div class="container mt-4"> <!-- mt: margin top scale 4-->
                <p>Welcome to my homepage! Explore my projects and learn more
about my journey in Computer Science.</p>
            </div>
        </main>
        <!-- Footer -->
        <footer class="bg-dark text-white text-center p-3 mt-5"> <!--bg-dark:
dark background / text-white: white text / text-center: center aligned text /
p-3: padding scale 3 / mt-5: margin top scale 5-->
            <p class="highlight">&copy; 2025 Slman | CS Student at NHL Stenden
Emmen</p>
            <a href="contact.html" class="text-white">Contact Me</a>
        </footer>
        <script>
            // Define a function that updates the clock display
            function updateClock() {
                // Create a new Date object with the current date and time
                const now = new Date();
                // Find the element with id="clock" and set its text to the
current time
                // toLocaleTimeString() formats the time according to the
user's locale (e.g. 12h or 24h)
                document.getElementById("clock").textContent =
now.toLocaleTimeString();
            }
            setInterval(updateClock, 1000); // update every second
            // Call updateClock immediately once when the page loads
            // Without this, the clock would stay empty until the first
interval tick
            updateClock();
        </script>

    </body>

</html>
```

Image and code for About Me Page:




```
<!DOCTYPE html>
<html lang="en">

    <head>
        <!-- Bootstrap CSS & JS -->
        <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
rel="stylesheet" integrity="sha384-
QWTKZyjpPEjISv5WaRU9OFeRpok6YctnYmDr5pNlyT2bRjXh0JMhjY6hW+ALEwIH"
crossorigin="anonymous">
        <script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min
.js" integrity="sha384-
YvpcrYf0tY3lHB60NNkmXc5s9fDVZLESaAA55NDzOxhy9GkcIdslK1eN7N6jIeHz"
crossorigin="anonymous"></script>

        <!-- Custom CSS -->
        <link href="styles.css" rel="stylesheet">
        <title>About Me</title>
    </head>

    <body>
        <!--  Navigation bar -->
        <nav class="navbar navbar-expand-lg navbar-dark bg-dark">
            <a class="navbar-brand" href="index.html">My CS Journey</a>
            <button class="navbar-toggler" type="button" data-bs-
toggle="collapse" data-bs-target="#navbarNav" aria-controls="navbarNav" aria-
expanded="false" aria-label="Toggle navigation">
                <span class="navbar-toggler-icon"></span>
            </button>
```

```
            <div class="collapse navbar-collapse" id="navbarNav">
                <ul class="navbar-nav ms-auto">
                    <li class="nav-item"><a class="nav-link"
href="index.html">Home</a></li>
                    <li class="nav-item"><a class="nav-link active"
href="about.html">About Me</a></li>
                    <li class="nav-item"><a class="nav-link"
href="projects.html">Projects</a></li>
                    <li class="nav-item"><a class="nav-link"
href="contact.html">Contact</a></li>
                </ul>
            </div>
        </nav>

        <!--  Page content -->
        <main class="flex-grow-1">
            <div class="container mt-4">
                <div class="row">
                    <!-- Photo -->
                    <div class="col-md-4"> <!-- col-md-4 means on medium+
screens this div takes 4/12 columns-->
                        <img src="images/slman.png" alt="My photo" class="img-
fluid rounded-circle mb-3" width="200">
                    </div>

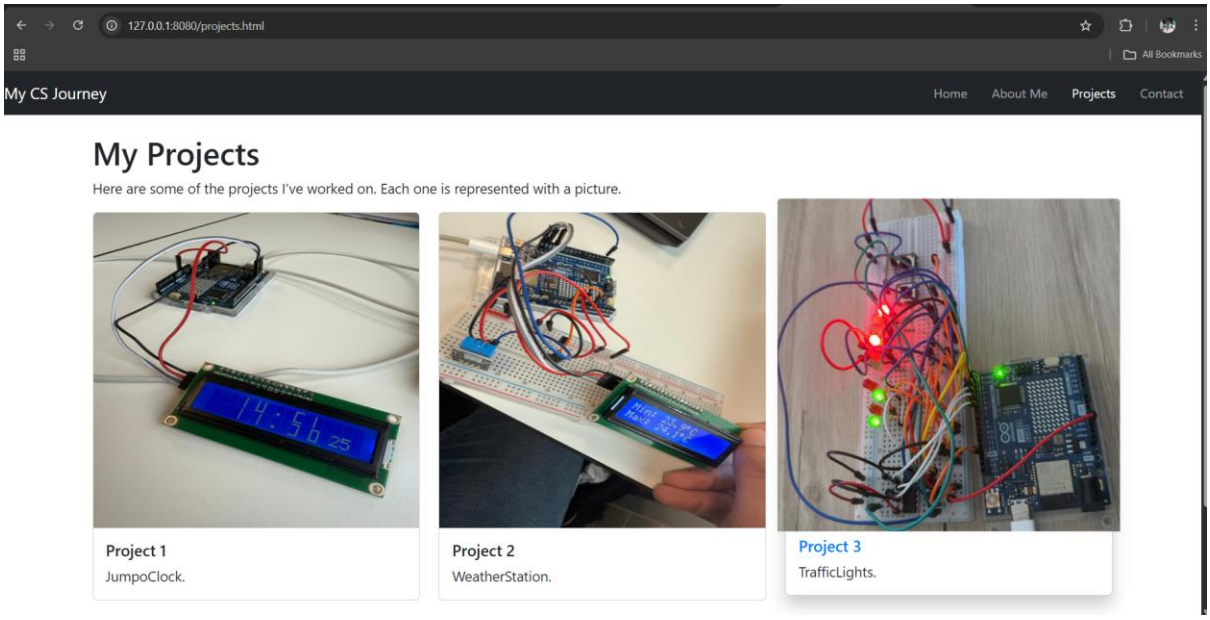
                    <!-- Short bio -->
                    <div class="col-md-8">
                        <h1>About Me</h1>
                        <p>
                            Hi, I'm Slman – a Computer Science student at NHL
Stenden.
                            This site is where I share my projects and the
skills I’m developing along the way.
                        </p>
                    </div>
                </div>

                <!-- Skills list -->
                <div class="mt-5">
                    <h2>Skills I'm Building</h2>
                    <ul class="list-group">
                        <li class="list-group-item">Web Development with
Bootstrap</li>
                        <li class="list-group-item">Problem-solving and
troubleshooting</li>
                        <li class="list-group-item">Database design
basics</li>
```

```
                        <li class="list-group-item">Collaborative
teamwork</li>
                    </ul>
                </div>
            </div>
        </main>
        <!--  Footer -->
        <footer class="bg-dark text-white text-center p-3 mt-5">
            <p class="highlight">&copy; 2025 Slman | CS Student at NHL
Stenden</p>
            <a href="contact.html" class="text-white">Contact Me</a>
        </footer>
    </body>

</html>
```

Image and Code for Projects Page:




```
<!DOCTYPE html>
<html lang="en">

    <head>
        <!-- Bootstrap CSS & JS -->
        <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
rel="stylesheet" integrity="sha384-
QWTKZyjpPEjISv5WaRU9OFeRpok6YctnYmDr5pNlyT2bRjXh0JMhjY6hW+ALEwIH"
crossorigin="anonymous">
```

```
        <script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min
.js" integrity="sha384-
YvpcrYf0tY3lHB60NNkmXc5s9fDVZLESaAA55NDzOxhy9GkcIdslK1eN7N6jIeHz"
crossorigin="anonymous"></script>

        <link href="styles.css" rel="stylesheet">
        <title>Projects</title>
    </head>

    <body>
        <!-- Navigation bar -->
        <nav class="navbar navbar-expand-lg navbar-dark bg-dark">
            <a class="navbar-brand" href="index.html">My CS Journey</a>
            <button class="navbar-toggler" type="button" data-bs-
toggle="collapse" data-bs-target="#navbarNav" aria-controls="navbarNav" aria-
expanded="false" aria-label="Toggle navigation">
                <span class="navbar-toggler-icon"></span>
            </button>
            <div class="collapse navbar-collapse" id="navbarNav">
                <ul class="navbar-nav ms-auto">
                    <li class="nav-item"><a class="nav-link"
href="index.html">Home</a></li>
                    <li class="nav-item"><a class="nav-link"
href="about.html">About Me</a></li>
                    <li class="nav-item"><a class="nav-link active"
href="projects.html">Projects</a></li>
                    <li class="nav-item"><a class="nav-link"
href="contact.html">Contact</a></li>
                </ul>
            </div>
        </nav>

        <!--  Page content -->
        <main class="flex-grow-1">
            <div class="container mt-4">
                <h1>My Projects</h1>
                <p>Here are some of the projects I've worked on. Each one is
represented with a picture.</p>

                <!-- Gallery grid -->
                <div class="row">
                    <!-- Project 1 -->
                    <div class="col-md-4 mb-4"> <!--col-md-4: margin bottom.
in medium+ screens, three of these side by side will fill the row
                                                mb : medium breakpoint. Adds
extra space below the column, so multiple rows of cards don't look cramped.-->
```

```
                    <div class="card">
                        <img src="images/jumpoclock.jpg" class="card-img-
top" alt="Project 1"width="400" height="400"> <!-- card-img-top: Makes the
image span the full width of the card.

Rounds the top corners to match the card’s border radius.

Ensures consistent sizing and alignment with other card components.-->
                        <div class="card-body">
                            <h5 class="card-title">Project 1</h5>
                            <p class="card-text">JumpoClock.</p>
                        </div>
                    </div>
                </div>

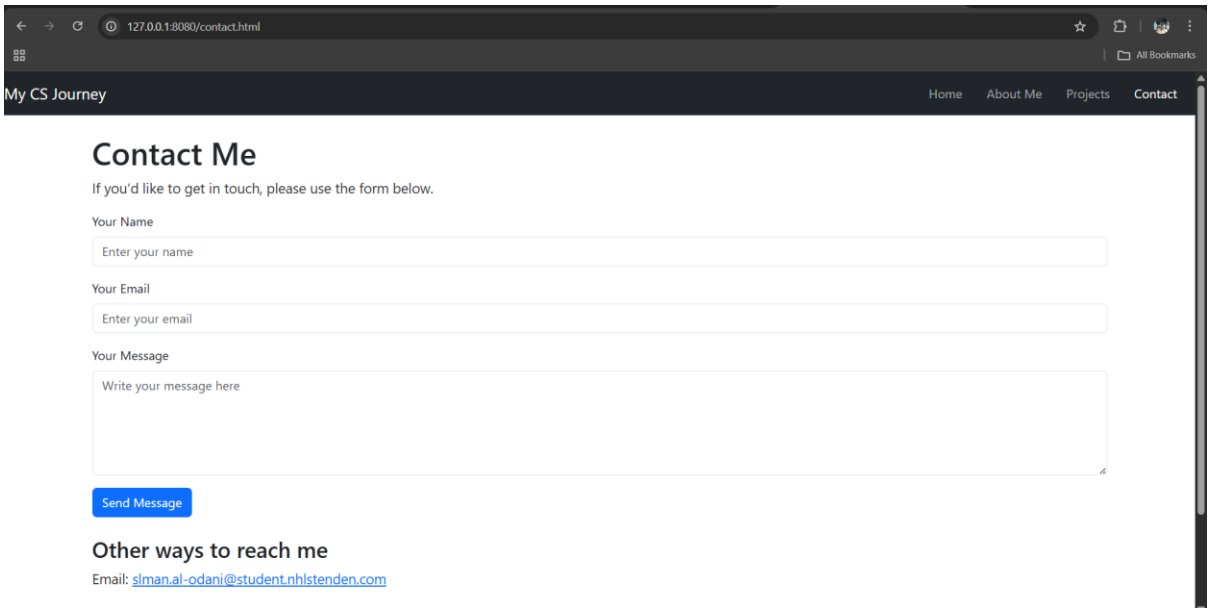
                <!-- Project 2 -->
                <div class="col-md-4 mb-4">
                    <div class="card">
                        <img src="images/weather_station.jpg" class="card-
img-top" alt="Project 2" width="400" height="400">
                        <div class="card-body">
                            <h5 class="card-title">Project 2</h5>
                            <p class="card-text">WeatherStation.</p>
                        </div>
                    </div>
                </div>

                <!-- Project 3 -->
                <div class="col-md-4 mb-4">
                    <div class="card">
                        <img src="images/traffic lights.jpg" class="card-
img-top" alt="Project 3">
                        <div class="card-body">
                            <h5 class="card-title">Project 3</h5>
                            <p class="card-text">TrafficLights.</p>
                        </div>
                    </div>
                </div>
            </div>
        </div>
    </main>
    <!--  Footer -->
    <footer class="bg-dark text-white text-center p-3 mt-5">
        <p class="highlight">&copy; 2025 Slman | CS Student at NHL
Stenden</p>
        <a href="contact.html" class="text-white">Contact Me</a>
    </footer>
```

```
    </body>

</html>
```

Image and Code for the Contact Me page:




```
<!DOCTYPE html>
<html lang="en">

    <head>
        <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
rel="stylesheet" integrity="sha384-
QWTKZyjpPEjISv5WaRU9OFeRpok6YctnYmDr5pNlyT2bRjXh0JMhjY6hW+ALEwIH"
crossorigin="anonymous">
        <script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min
.js" integrity="sha384-
YvpcrYf0tY3lHB60NNkmXc5s9fDVZLESaAA55NDzOxhy9GkcIdslK1eN7N6jIeHz"
crossorigin="anonymous"></script>
        <link href="styles.css" rel="stylesheet">
        <title>Contact</title>
    </head>

    <body>
        <!-- Navigation bar -->
        <nav class="navbar navbar-expand-lg navbar-dark bg-dark">
            <a class="navbar-brand" href="index.html">My CS Journey</a>
```

```
            <button class="navbar-toggler" type="button" data-bs-
toggle="collapse" data-bs-target="#navbarNav" aria-controls="navbarNav" aria-
expanded="false" aria-label="Toggle navigation">
                <span class="navbar-toggler-icon"></span>
            </button>
            <div class="collapse navbar-collapse" id="navbarNav">
                <ul class="navbar-nav ms-auto">
                    <li class="nav-item"><a class="nav-link"
href="index.html">Home</a></li>
                    <li class="nav-item"><a class="nav-link"
href="about.html">About Me</a></li>
                    <li class="nav-item"><a class="nav-link"
href="projects.html">Projects</a></li>
                    <li class="nav-item"><a class="nav-link active"
href="contact.html">Contact</a></li>
                </ul>
            </div>
        </nav>

        <!-- Page content -->
        <main class="flex-grow-1">
            <div class="container mt-4">
                <h1>Contact Me</h1>
                <p>If you'd like to get in touch, please use the form
below.</p>

                <!-- Contact form -->
                <form> <!--Starts the form element. Groups all inputs together
so they can be submitted as one set of data.-->
                    <div class="mb-3">
                        <label for="name" class="form-label">Your Name</label>
<!-- label descripe the input and link to it via for="name" and id="name"-->
                        <input type="text" class="form-control" id="name"
placeholder="Enter your name">
                    </div>

                    <div class="mb-3">
                        <label for="email" class="form-label">Your
Email</label>
                        <input type="email" class="form-control" id="email"
placeholder="Enter your email">
                    </div>

                    <div class="mb-3">
                        <label for="message" class="form-label">Your
Message</label>
```

```
                        <textarea class="form-control" id="message" rows="5"
placeholder="Write your message here"></textarea> <!-- textarea: muilti line
inbutbox-->
                    </div>

                    <button type="submit" class="btn btn-primary">Send
Message</button>
                </form>

                <!-- Extra info -->
                <div class="mt-4">
                    <h3>Other ways to reach me</h3>
                    <p>Email: <a href="mailto:slman.al-
odani@student.nhlstenden.com">slman.al-odani@student.nhlstenden.com</a></p>
                </div>
            </div>
        </main>
        <!-- Footer -->
        <footer class="bg-dark text-white text-center p-3 mt-5">
            <p class="highlight">&copy; 2025 Slman | CS Student at NHL
Stenden</p>
            <a href="contact.html" class="text-white">Contact Me</a>
        </footer>
    </body>

</html>
```

The Style.css file:

```
/* Make sure the body takes full height */
html, body {
  height: 100%;
  margin: 0;
}

/* Use flexbox to push footer down */
body {
  display: flex;
  flex-direction: column;
}

/* Main content grows to fill space */
main {
  flex: 1;
}
```

```css
/* Footer stays at bottom */
footer {
  margin-top: auto;
}

/* Tag selector */
p {
  color: #333;              /* text color */
  font-size: 18px;          /* font size */
}

/* Class selector */
.highlight {
  background-color: yellow; /* highlight background */
  padding: 5px;             /* spacing inside */
}



/* Descendant selector It means:
“Find every <li> element that is inside a <ul> which is inside a <nav>.”
So it only applies to list items (<li>) inside navigation menus.*/
nav ul li {
  list-style-type: none;    /* remove bullet points */
  margin-right: 15px;       /* spacing between links */
}

/* Pseudo-class selector */
a:hover {
  color: red;               /* change link color on hover */
  text-decoration: underline; /* underline on hover */
}

/* ID selector */
#clock {
  font-size: 30px;
  font-weight: bold;
  color: #fff;
  text-shadow: 0 0 10px #00f, 0 0 20px #00f, 0 0 30px #00f;
  margin-top: 15px;
}
/* ============================
   Card Hover Effects
   ============================ */
.card {
    transition: transform 0.3s ease, box-shadow 0.3s ease;
```

```
}

.card:hover {
    transform: translateY(-8px); /* lift the card */
    box-shadow: 0 12px 20px rgba(0, 0, 0, 0.2); /* deeper shadow */
}

.card-img-top {
    transition: transform 0.3s ease;
}

.card:hover .card-img-top {
    transform: scale(1.05); /* zoom image slightly */
}

.card-title {
    transition: color 0.3s ease;
}

.card:hover .card-title {
    color: #007bff; /* change title color on hover */
}
```

The specification file:

HTML Tags Used:

1. <h1>
2. <h2>
3. <p>
4. <a>
5. <img>
6. /
7. <form>
8. <input>
9. <button>
10. <nav>

11. <footer>

12.

13. <textarea>

14. <main>

CSS Properties Used:

1. color

2. font-size

3. background-color

4. padding

5. margin

6. text-align

7. font-weight

8. height

9. display

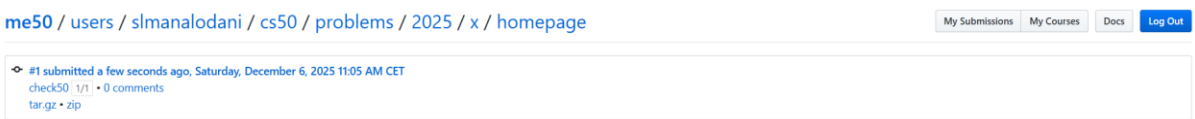
10. flex-direction

JavaScript Usage:

A live digital clock was implemented on the homepage that updates every second.

Bootstrap Usage:

Bootstrap was used for responsive design and components such as the navbar, grid system, buttons, cards, and list groups.



https://submit.cs50.io/check50/3e96815d085ea161dc1b7a9b3b31b41a72dc3801

Assignment 3: Blackboard

Hello, World

Program your first Hello, World webpage. Call your file index.html and open it.

Image:



Code:

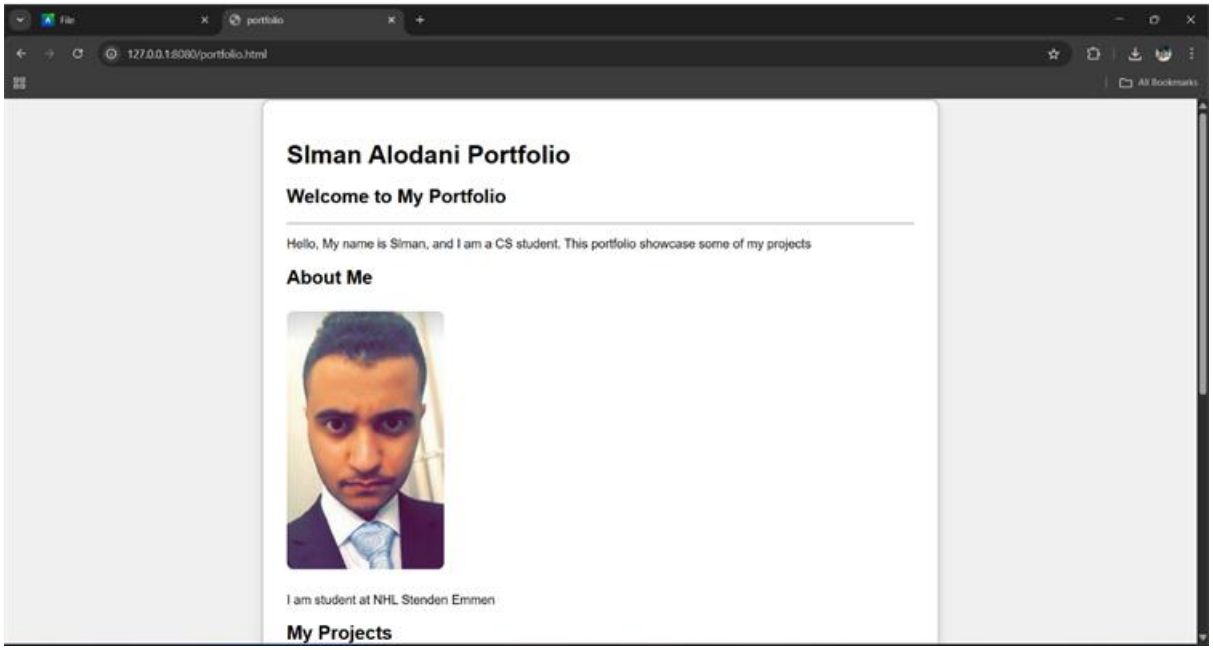
```
 <!DOCTYPE html>

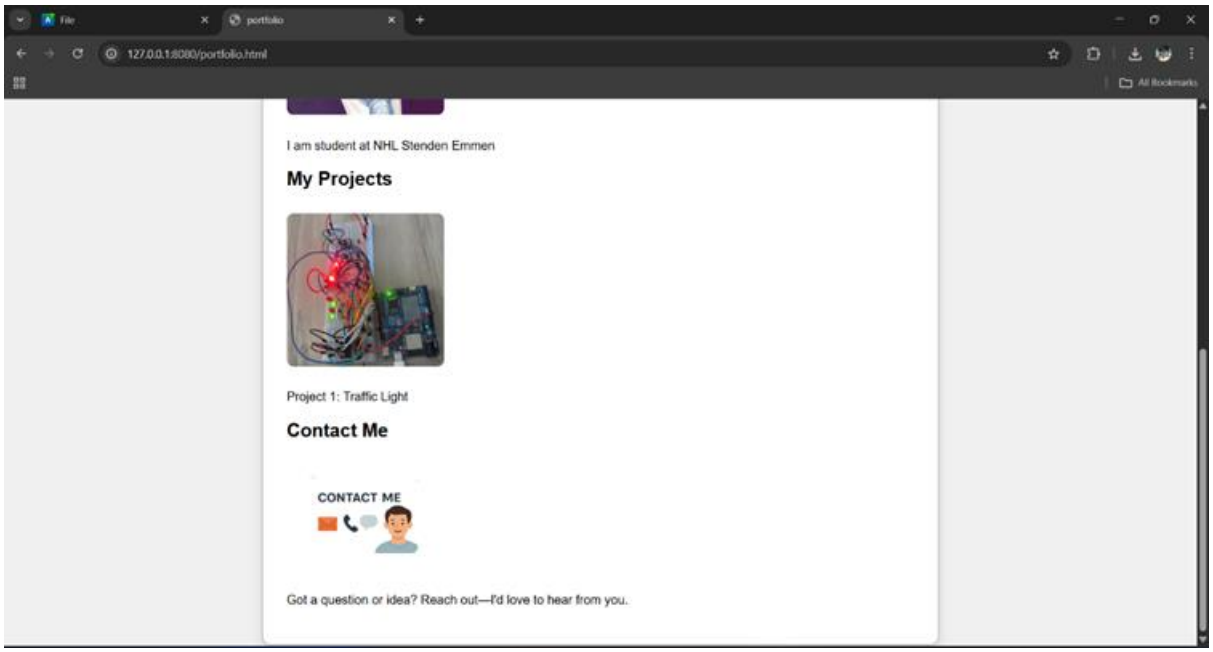
<html lang="en">
    <head>
    <meta charset="UTF-8">
    <title>Hello, World</title>
    </head>
    <body>
        <h1>Hello, World!</h1>
</html>
```

Portfolio

Recreate a portfolio webpage with your own images and text.

Images:





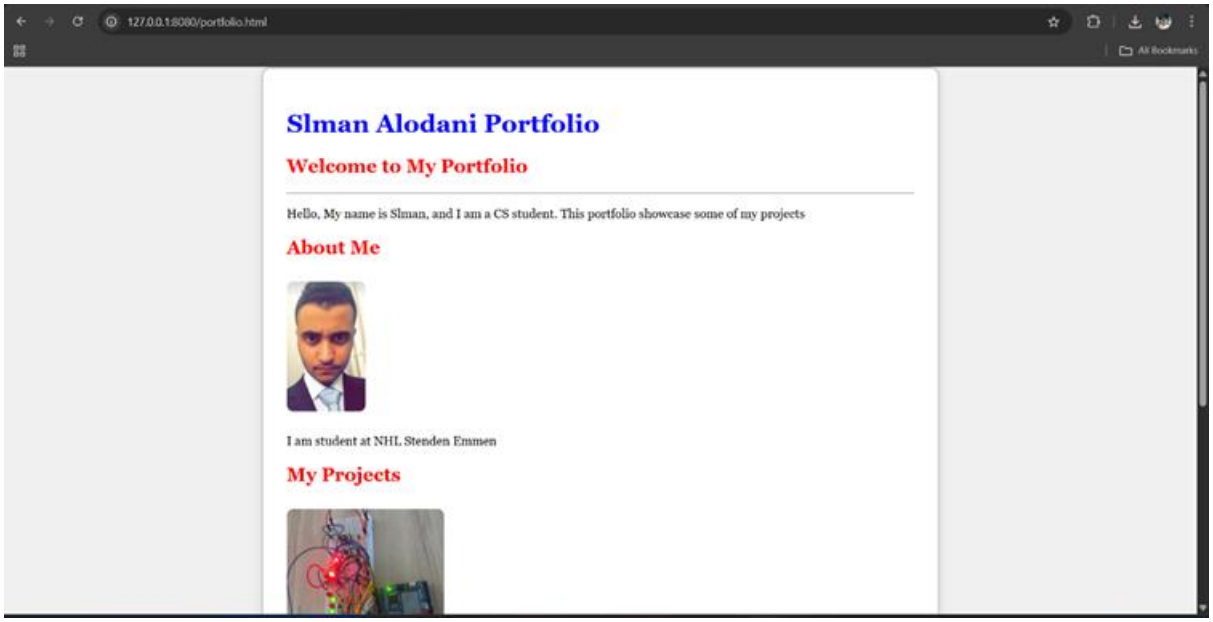
Code:

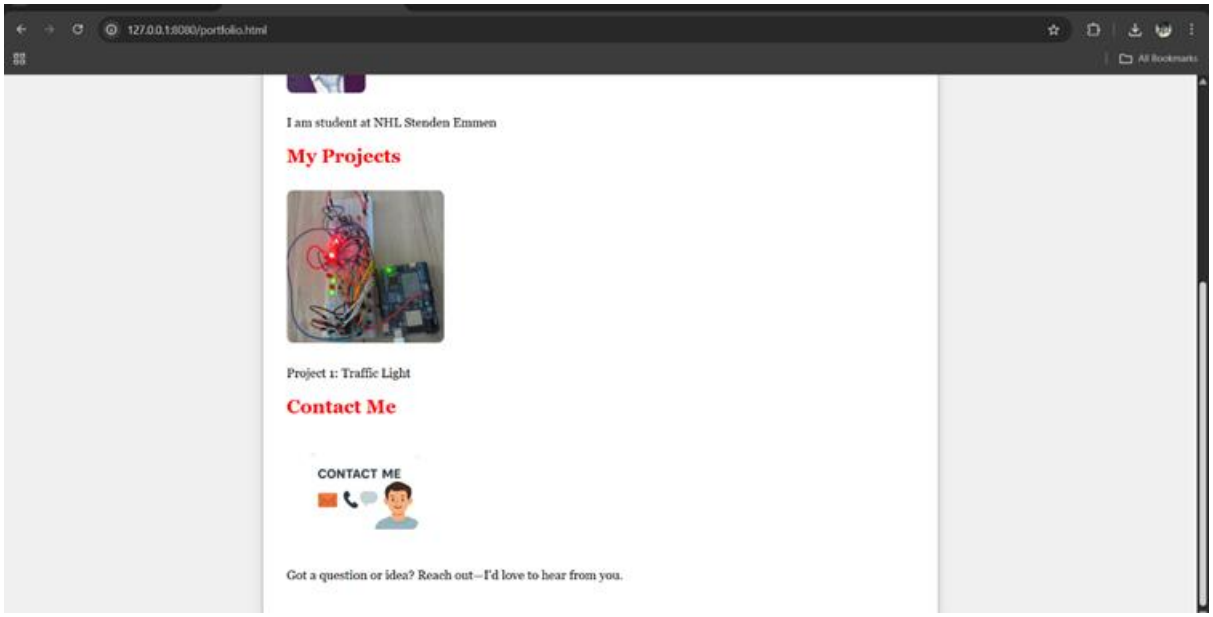
```
<!DOCTYPE html>
```

```html
<html lang="en">
    <head>
    <meta charset="UTF-8">
    <title>portfolio</title>
    <link href="style.css" rel="stylesheet">
    </head>
    <body>
        <div class="page-container">
            <h1>Slman Alodani Portfolio</h1>
            <h2>Welcome to My Portfolio</h2>
            <hr>
            <P>Hello, My name is Slman, and I am a CS student. This portfolio
showcase some of my projects </P>
            <h2>About Me</h2>
            <img src="images/slman.png" alt="profile picture" width="200">
            <p>I am student at NHL Stenden Emmen</p>
            <h2>My Projects</h2>
            <img src="images/traffic lights.jpg" alt="project 1" width="200">
            <p>Project 1: Traffic Light </p>
            <h2>Contact Me</h2>
            <img src="images/contact me.png" alt="email icon" width="200">
            <p>Got a question or idea? Reach out—I'd love to hear from
you.</p>
        </div>
    </body>
</html>
```

Colours, fonts and visual effects

Images:





Code from my Style.css file:

```
/* style.css */

body {
  font-family: Georgia, "Time New Roman", sans-serif;
  background-color: #f0f0f0;
  margin: 0;
  display: flex;
  justify-content: center;
  align-items: center;
  min-height: 100vh; /* full screen height */
}

.page-container {
  background: #fff;
  border: 3px solid #ccc;
  border-radius: 12px;
  padding: 30px;
  max-width: 800px;
  width: 90%;
  box-shadow: 0 6px 12px rgba(0,0,0,0.1);
  text-align: left;
}

img {
  margin: 10px 0;
  border-radius: 8px;
```

```
}

h1{
    color: blue;
}

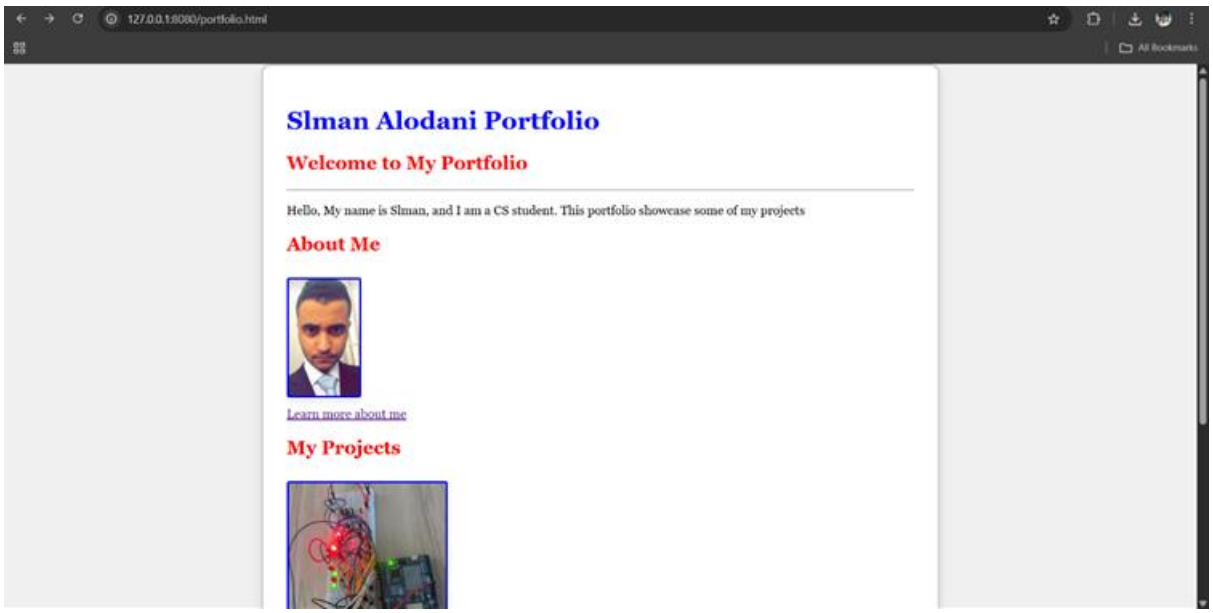
h2{
    color: red;
}
```

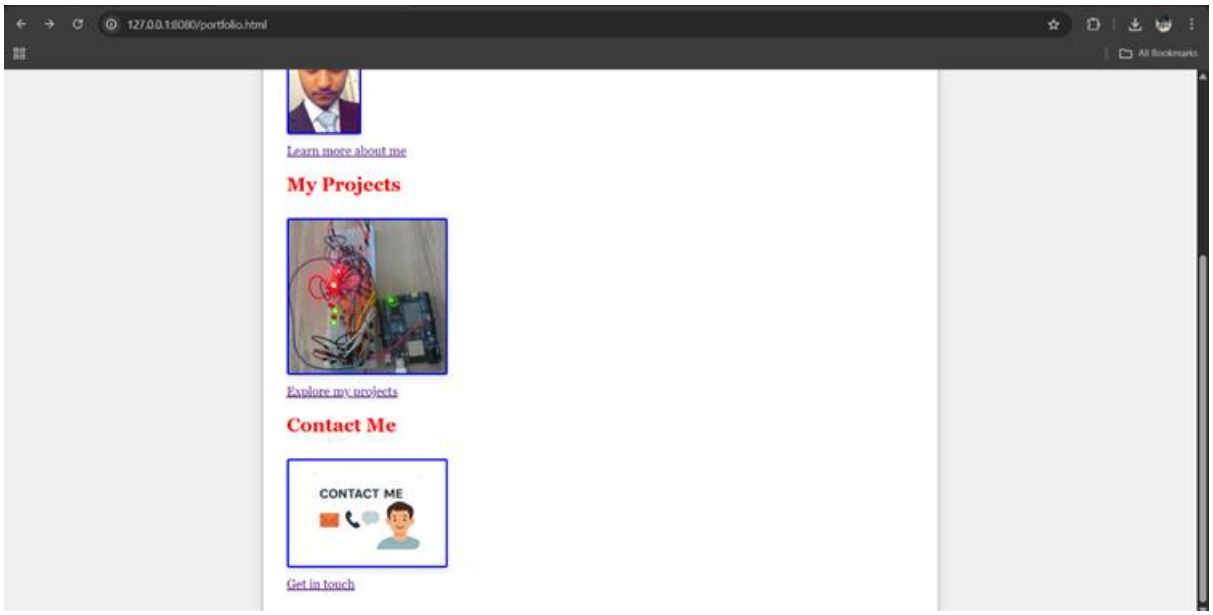
The code for the page:

```
<!DOCTYPE html>
<html lang="en">
    <head>
    <meta charset="UTF-8">
    <title>portfolio</title>
    <link href="style.css" rel="stylesheet">
    </head>
    <body>
        <div class="page-container">
            <h1>Slman Alodani Portfolio</h1>
            <h2>Welcome to My Portfolio</h2>
            <hr>
            <P>Hello, My name is Slman, and I am a CS student. This portfolio
showcase some of my projects </P>
            <h2>About Me</h2>
            <img src="images/slman.png" alt="profile picture" width="100">
            <p>I am student at NHL Stenden Emmen</p>
            <h2>My Projects</h2>
            <img src="images/traffic lights.jpg" alt="project 1" width="200">
            <p>Project 1: Traffic Light </p>
            <h2>Contact Me</h2>
            <img src="images/contact me.png" alt="email icon" width="200">
            <p>Got a question or idea? Reach out—I'd love to hear from
you.</p>
        </div>
    </body>
</html>
```

Add links to the portfolio

Images:





The code after these changes:

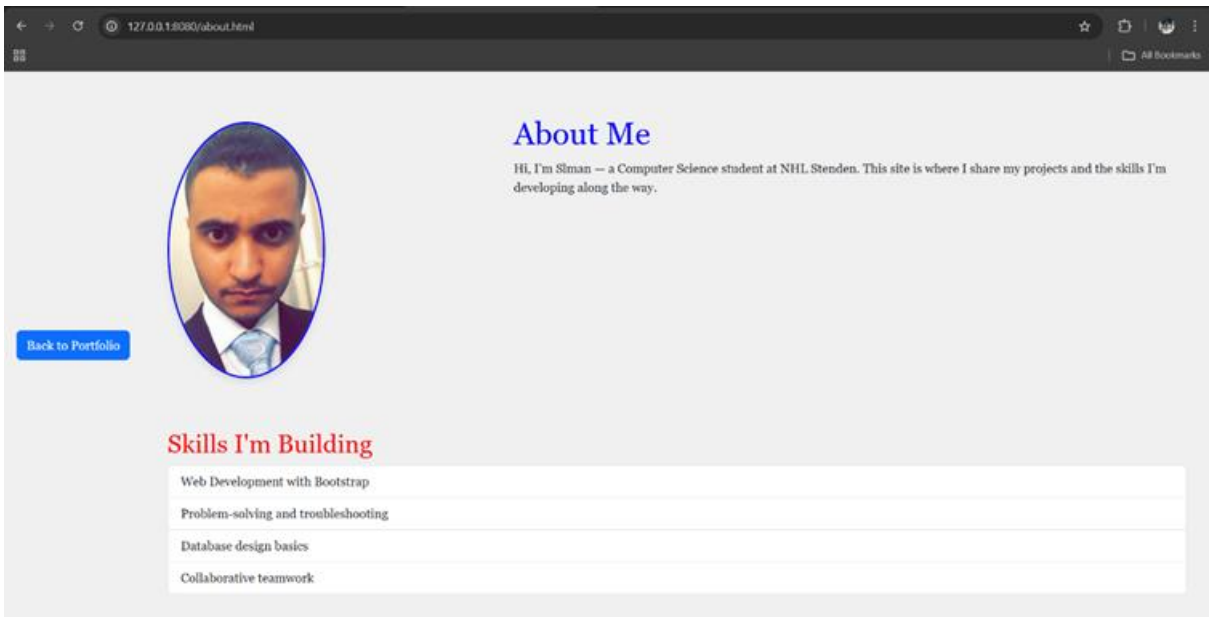
```
<!DOCTYPE html>
<html lang="en">
    <head>
    <meta charset="UTF-8">
    <title>portfolio</title>
```

```
    <link href="style.css" rel="stylesheet">
    </head>
    <body>
        <div class="page-container">
            <h1>Slman Alodani Portfolio</h1>
            <h2>Welcome to My Portfolio</h2>
            <hr>
            <P>Hello, My name is Slman, and I am a CS student. This portfolio
showcase some of my projects </P>
            <h2>About Me</h2>
            <img src="images/slman.png" alt="profile picture" width="90"><br>
            <a href="about.html">Learn more about me</a>
            <h2>My Projects</h2>
            <img src="images/traffic lights.jpg" alt="project 1"
width="200"><br>
            <a href="projects.html">Explore my projects</a>
            <h2>Contact Me</h2>
            <img src="images/contact me.png" alt="email icon" width="200"><br>
            <a href="contact.html">Get in touch</a>
        </div>
    </body>
</html>
```

Create two pages and link them to the main page

About Me page

Image:



Code:

```
<!DOCTYPE html>
```

```
<html lang="en">
    <head>
        <meta charset="UTF-8">
        <!-- Bootstrap CSS & JS -->
        <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
rel="stylesheet" integrity="sha384-
QWTKZyjpPEjISv5WaRU9OFeRpok6YctnYmDr5pNlyT2bRjXh0JMhjY6hW+ALEwIH"
crossorigin="anonymous">
        <script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min
.js" integrity="sha384-
YvpcrYf0tY3lHB60NNkmXc5s9fDVZLESaAA55NDzOxhy9GkcIdslK1eN7N6jIeHz"
crossorigin="anonymous"></script>

        <!-- Custom CSS -->
        <link href="style.css" rel="stylesheet">

        <title>About Me</title>
        </head>

        <body>
            <!-- button to go back to main page-->
            <button onclick="window.location.href='portfolio.html'" class="btn
btn-primary m-3">Back to Portfolio</button>
            <!--  Page content -->
            <main class="flex-grow-1">
                <div class="container mt-4">
                    <div class="row">
                        <!-- Photo -->
                        <div class="col-md-4"> <!-- col-md-4 means on medium+
screens this div takes 4/12 columns-->
                            <img src="images/slman.png" alt="My photo"
class="img-fluid rounded-circle mb-3" width="200">
                        </div>

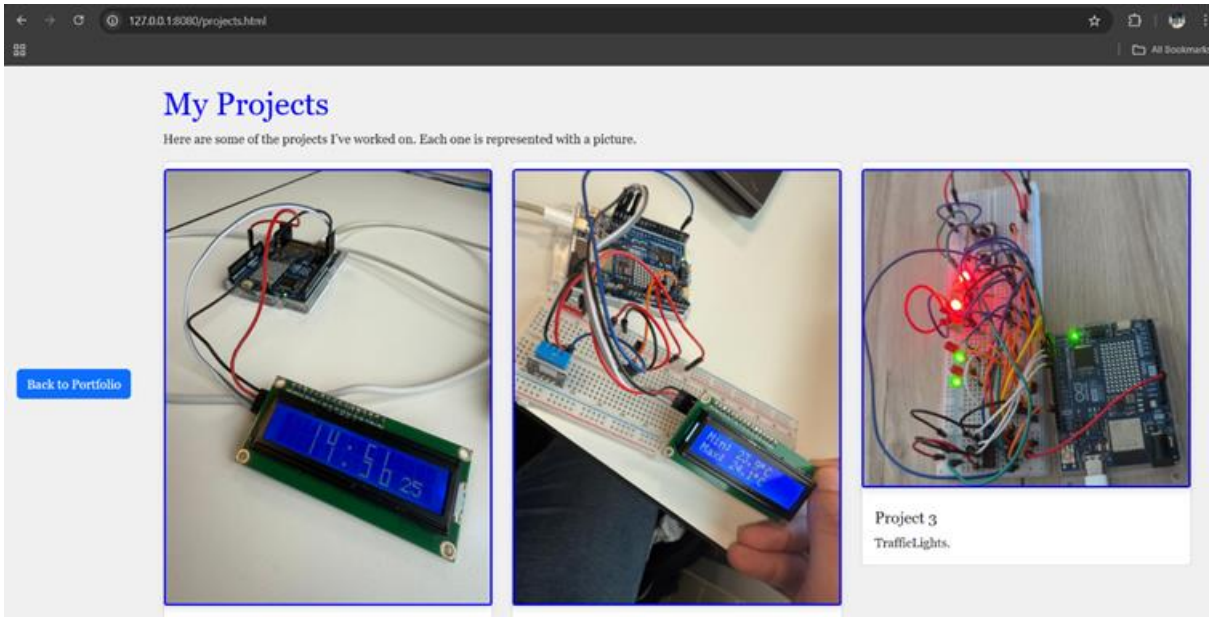
                        <!-- Short bio -->
                        <div class="col-md-8">
                            <h1>About Me</h1>
                            <p>
                                Hi, I'm Slman — a Computer Science student at
NHL Stenden.
                                This site is where I share my projects and the
skills I’m developing along the way.
                            </p>
                        </div>
                    </div>
```

```
                    <!-- Skills list -->
                    <div class="mt-5">
                        <h2>Skills I'm Building</h2>
                        <ul class="list-group">
                            <li class="list-group-item">Web Development with
Bootstrap</li>
                            <li class="list-group-item">Problem-solving and
troubleshooting</li>
                            <li class="list-group-item">Database design
basics</li>
                            <li class="list-group-item">Collaborative
teamwork</li>
                        </ul>
                    </div>
                </div>
            </main>

            </body>
    </html>
```

My Project Page.

Image:



Code:

```
<!DOCTYPE html>
<html lang="en">
    <head>
```

```
        <meta charset="UTF-8">
        <!-- Bootstrap CSS & JS -->
        <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
rel="stylesheet" integrity="sha384-
QWTKZyjpPEjISv5WaRU9OFeRpok6YctnYmDr5pNlyT2bRjXh0JMhjY6hW+ALEwIH"
crossorigin="anonymous">
        <script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min
.js" integrity="sha384-
YvpcrYf0tY3lHB60NNkmXc5s9fDVZLESaAA55NDzOxhy9GkcIdslK1eN7N6jIeHz"
crossorigin="anonymous"></script>

        <!-- Custom CSS -->
        <link href="style.css" rel="stylesheet">

        <title>My Projects</title>
        </head>

        <body>
            <!--  Page content -->

            <!-- button to go back to main page-->
            <button onclick="window.location.href='portfolio.html'" class="btn
btn-primary m-3">Back to Portfolio</button>

            <main class="flex-grow-1">
                <div class="container mt-4">
                    <h1>My Projects</h1>
                    <p>Here are some of the projects I've worked on. Each one
is represented with a picture.</p>

                    <!-- Gallery grid -->
                    <div class="row">
                        <!-- Project 1 -->
                        <div class="col-md-4 mb-4"> <!--col-md-4: margin
bottom. in medium+ screens, three of these side by side will fill the row
                                                    mb : medium breakpoint.
Adds extra space below the column, so multiple rows of cards don't look
cramped.-->
                            <div class="card">
                                <img src="images/jumpoclock.jpg" class="card-
img-top" alt="Project 1"> <!-- card-img-top: Makes the image span the full
width of the card.

Rounds the top corners to match the card's border radius.
```

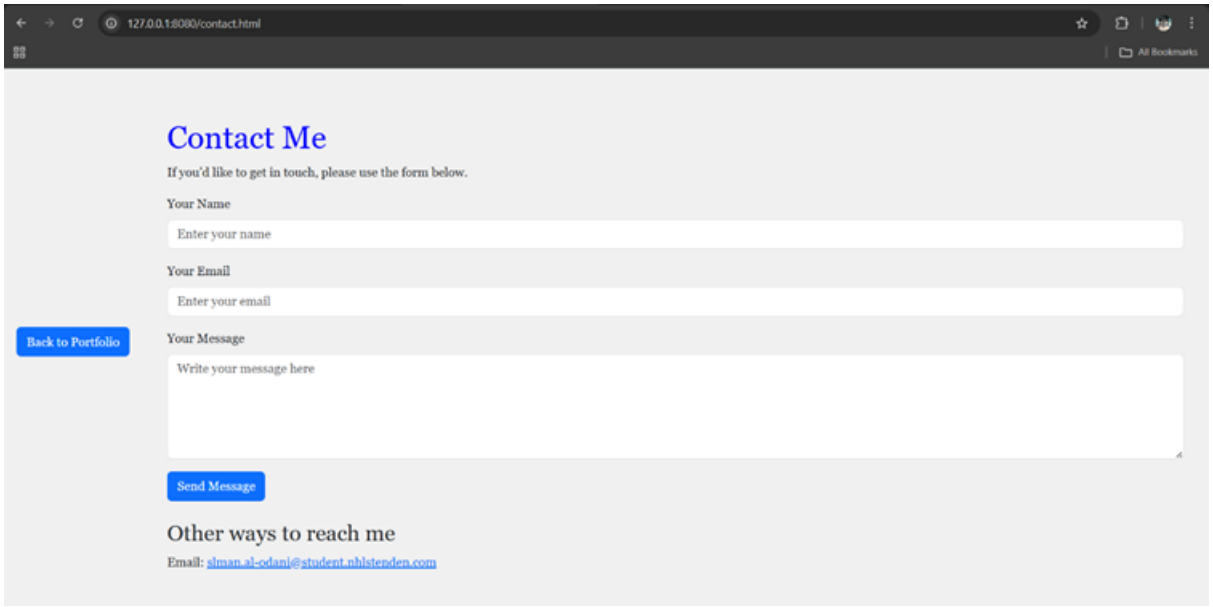
```
Ensures consistent sizing and alignment with other card components.-->
                            <div class="card-body">
                                <h5 class="card-title">Project 1</h5>
                                <p class="card-text">JumpoClock.</p>
                            </div>
                        </div>
                    </div>

                    <!-- Project 2 -->
                    <div class="col-md-4 mb-4">
                        <div class="card">
                            <img src="images/weather_station.jpg"
class="card-img-top" alt="Project 2">
                            <div class="card-body">
                                <h5 class="card-title">Project 2</h5>
                                <p class="card-text">WeatherStation.</p>
                            </div>
                        </div>
                    </div>

                    <!-- Project 3 -->
                    <div class="col-md-4 mb-4">
                        <div class="card">
                            <img src="images/traffic lights.jpg"
class="card-img-top" alt="Project 3">
                            <div class="card-body">
                                <h5 class="card-title">Project 3</h5>
                                <p class="card-text">TrafficLights.</p>
                            </div>
                        </div>
                    </div>
                </div>
            </div>
        </main>
    </body>
</html>
```

Make Contact Form:

Image:



Code:

```
<!DOCTYPE html>
<html lang="en">
    <head>
        <meta charset="UTF-8">
        <!-- Bootstrap CSS & JS -->
        <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
rel="stylesheet" integrity="sha384-
QWTKZyjpPEjISv5WaRU9OFeRpok6YctnYmDr5pNlyT2bRjXh0JMhjY6hW+ALEwIH"
crossorigin="anonymous">
        <script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min
.js" integrity="sha384-
YvpcrYf0tY3lHB60NNkmXc5s9fDVZLESaAA55NDzOxhy9GkcIdslK1eN7N6jIeHz"
crossorigin="anonymous"></script>

        <!-- Custom CSS -->
        <link href="style.css" rel="stylesheet">

        <title>contact</title>
        </head>
```

```
        <body>
            <!-- Page content -->
            <!-- button to go back to main page-->
            <button onclick="window.location.href='portfolio.html'" class="btn
btn-primary m-3">Back to Portfolio</button>

            <main class="flex-grow-1">
                <div class="container mt-4">
                    <h1>Contact Me</h1>
                    <p>If you'd like to get in touch, please use the form
below.</p>

                    <!-- Contact form -->
                    <form> <!--Starts the form element. Groups all inputs
together so they can be submitted as one set of data.-->
                        <div class="mb-3">
                            <label for="name" class="form-label">Your
Name</label> <!-- label descripe the input and link to it via for="name" and
id="name"-->
                            <input type="text" class="form-control" id="name"
placeholder="Enter your name">
                        </div>

                        <div class="mb-3">
                            <label for="email" class="form-label">Your
Email</label>
                            <input type="email" class="form-control"
id="email" placeholder="Enter your email">
                        </div>

                        <div class="mb-3">
                            <label for="message" class="form-label">Your
Message</label>
                            <textarea class="form-control" id="message"
rows="5" placeholder="Write your message here"></textarea> <!-- textarea:
muilti line inbutbox-->
                        </div>

                        <button type="submit" class="btn btn-primary">Send
Message</button>
                    </form>

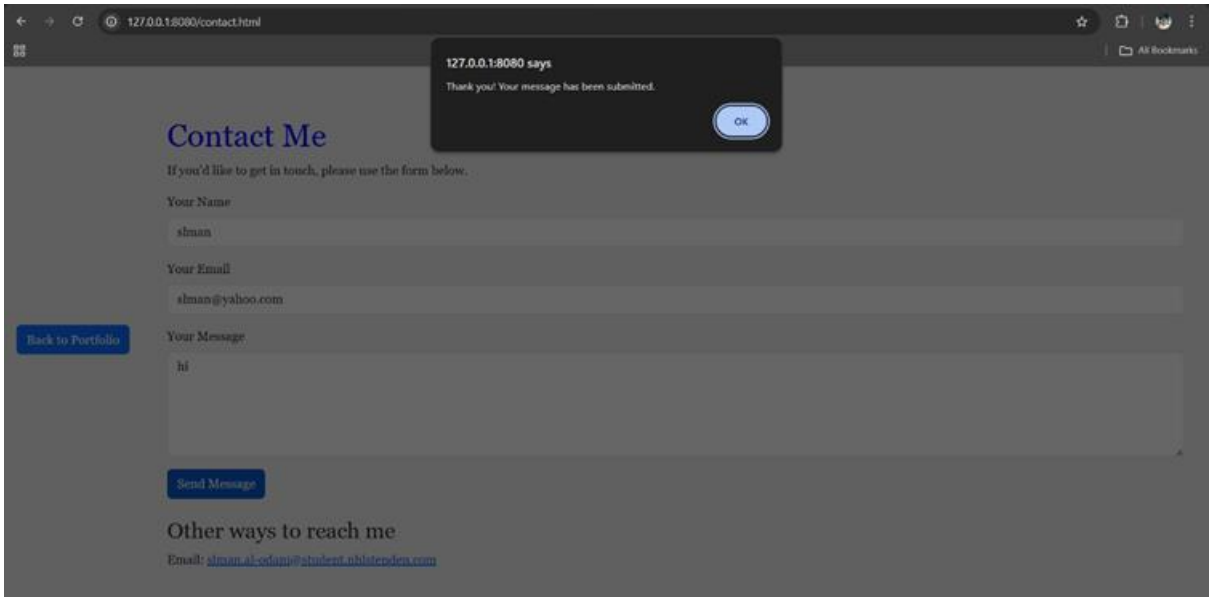
                    <!-- Extra info -->
                    <div class="mt-4">
                        <h3>Other ways to reach me</h3>
```

```
                    <p>Email: <a href="mailto:slman.al-
odani@student.nhlstenden.com">slman.al-odani@student.nhlstenden.com</a></p>
                </div>
            </div>
        </main>

    </body>
</html>
```

JavaScript: make alert when someone finish the contact me form:

Image:



Code:

```
<!DOCTYPE html>
<html lang="en">
    <head>
        <meta charset="UTF-8">
        <!-- Bootstrap CSS & JS -->
        <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
rel="stylesheet" integrity="sha384-
QWTKZyjpPEjISv5WaRU9OFeRpok6YctnYmDr5pNlyT2bRjXh0JMhjY6hW+ALEwIH"
crossorigin="anonymous">
        <script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min
.js" integrity="sha384-
YvpcrYf0tY3lHB60NNkmXc5s9fDVZLESaAA55NDzOxhy9GkcIdslK1eN7N6jIeHz"
crossorigin="anonymous"></script>

        <!-- Custom CSS -->
```

```
        <link href="style.css" rel="stylesheet">

        <title>contact</title>
        </head>

        <body>
            <!-- Page content -->
            <!-- button to go back to main page-->
            <button onclick="window.location.href='portfolio.html'" class="btn
btn-primary m-3">Back to Portfolio</button>

            <main class="flex-grow-1">
                <div class="container mt-4">
                    <h1>Contact Me</h1>
                    <p>If you'd like to get in touch, please use the form
below.</p>



                <!-- Contact form -->
                <form id="contactForm">
                    <div class="mb-3">
                        <label for="name" class="form-label">Your Name</label>
                        <input type="text" class="form-control" id="name"
placeholder="Enter your name">
                    </div>

                    <div class="mb-3">
                        <label for="email" class="form-label">Your
Email</label>
                        <input type="email" class="form-control" id="email"
placeholder="Enter your email">
                    </div>

                    <div class="mb-3">
                        <label for="message" class="form-label">Your
Message</label>
                        <textarea class="form-control" id="message" rows="5"
placeholder="Write your message here"></textarea>
                    </div>

                    <button type="submit" class="btn btn-primary">Send
Message</button>
                </form>

                <!-- JavaScript -->
                <script>
                    // Grab the form by its ID
```

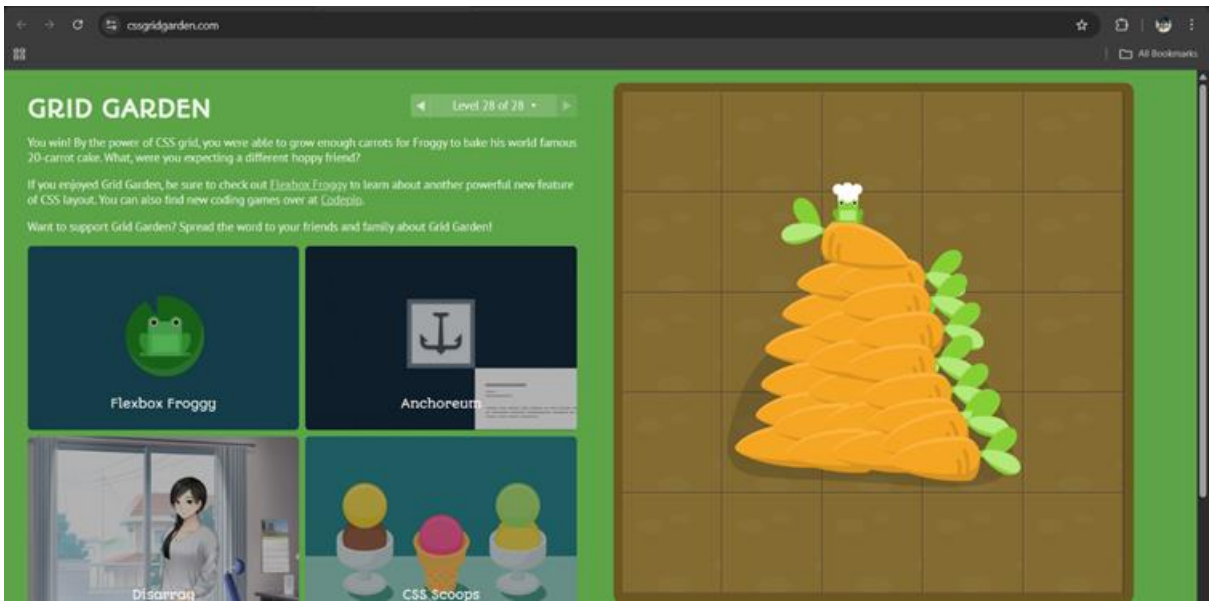
```
                const form = document.getElementById("contactForm");

                // Listen for submit event
                form.addEventListener("submit", function(event) {
                    event.preventDefault(); // Prevents actual page reload
                    alert("Thank you! Your message has been submitted.");
                });
            </script>

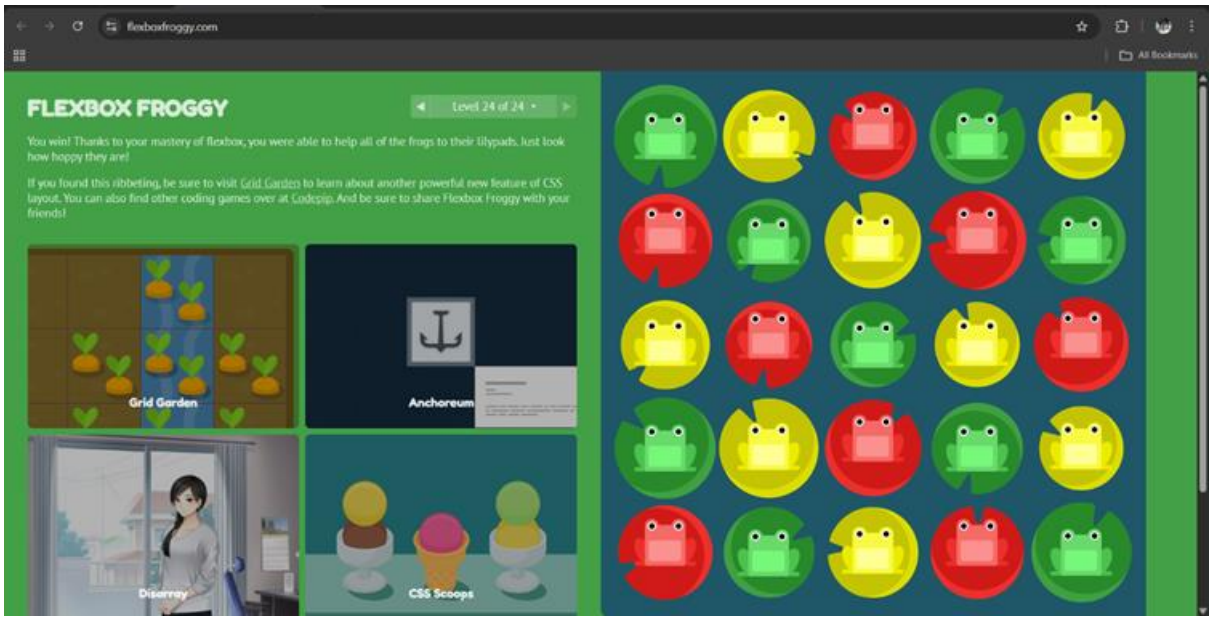

                <!-- Extra info -->
                <div class="mt-4">
                    <h3>Other ways to reach me</h3>
                    <p>Email: <a href="mailto:slman.al-
odani@student.nhlstenden.com">slman.al-odani@student.nhlstenden.com</a></p>
                </div>
            </div>
        </main>

    </body>
</html>
```

Grid Garden: this is an image after completing all the levels

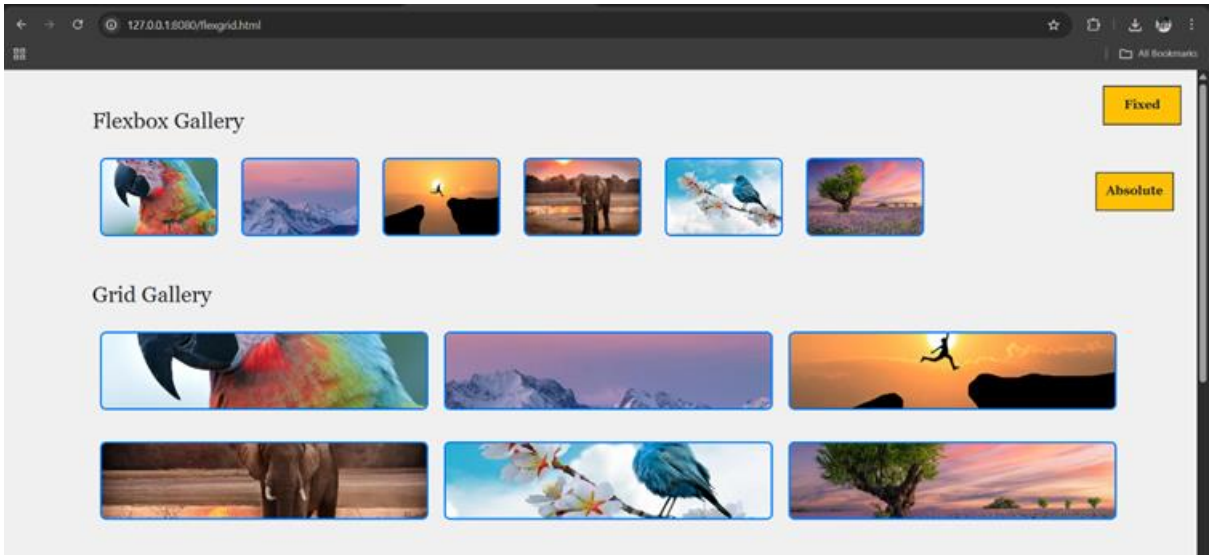


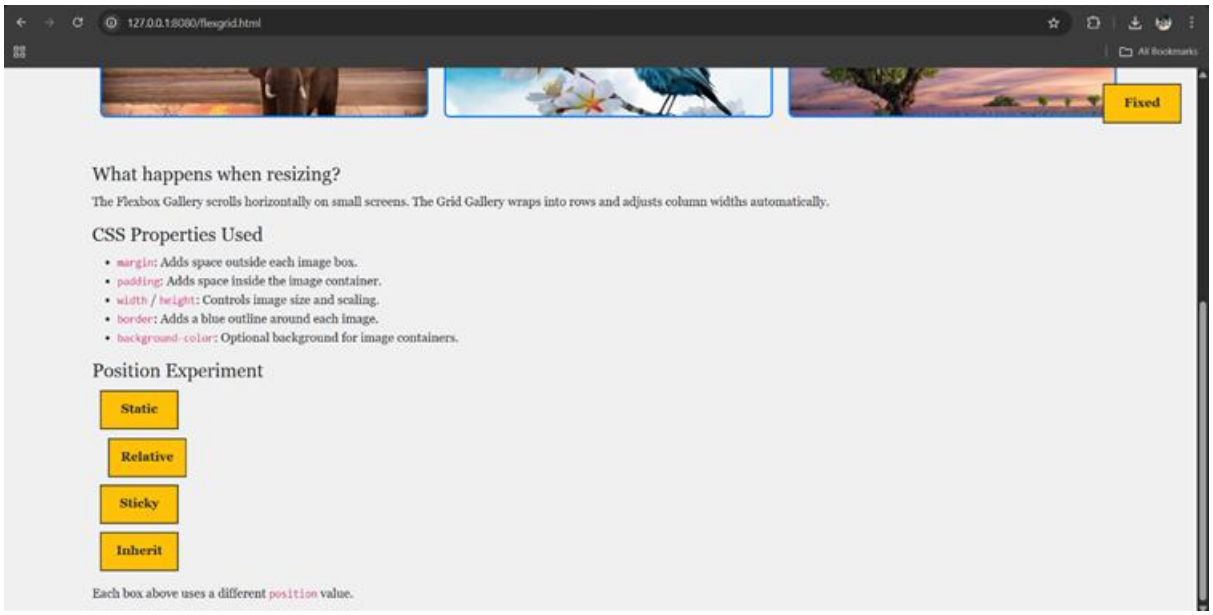
Flexbox froggy: this is an image after completing all the levels



Play with flex and grid Page

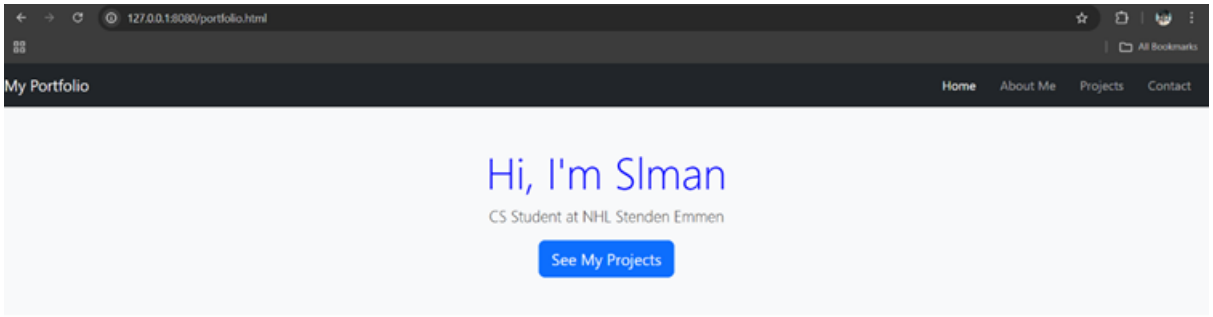
Images:

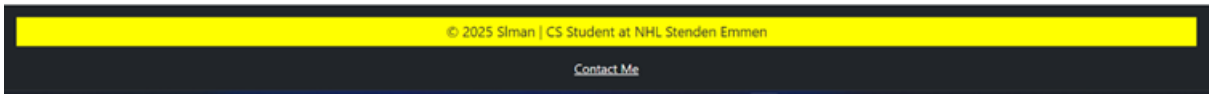




11) Main page with header and footer.

Image:





Code:

```
<!DOCTYPE html>
<html lang="en">
    <head>
        <meta charset="UTF-8">
        <!-- Bootstrap CSS & JS -->
        <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.m
in.css" rel="stylesheet" integrity="sha384-
QWTKZyjpPEjISv5WaRU9OFeRpok6YctnYmDr5pNlyT2bRjXh0JMhjY6hW+ALEwIH"
crossorigin="anonymous">
        <script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bun
dle.min.js" integrity="sha384-
YvpcrYf0tY3lHB60NNkmXc5s9fDVZLESaAA55NDzOxhy9GkcIdslK1eN7N6jIeHz"
crossorigin="anonymous"></script>

        <!-- Custom CSS -->
        <link href="style.css" rel="stylesheet">

        <title>portfolio</title>
        </head>

    <body>
        <!-- Navigation bar-->
        <nav class="navbar navbar-expand-lg navbar-dark bg-dark"> <!--
the nav bar stay collabse until screen is large-->
            <!--my title/brand logo-->
```

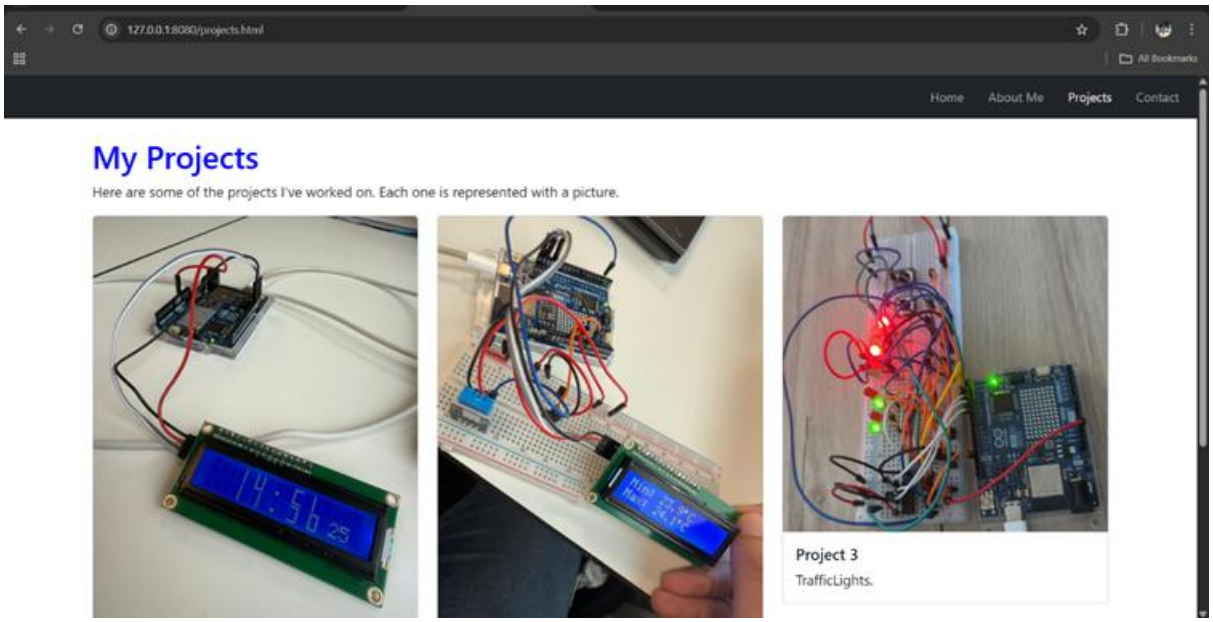
```
            <a class="navbar-brand" href="portfolio.html">My
Portfolio</a>
            <!-- Hamburger button (appers on mobile)-->
            <button class="navbar-toggler" type="button" data-bs-
toggle="collapse" data-bs-target="#navbarNav" aria-controls="navbarNav"
aria-expanded="false" aria-label="Toggle navigation">
                <!-- Icon for the hamburger menu-->
                <span class="navbar-toggler-icon"></span> <!--icon
inside the button / inline container-->
            </button>
            <!-- collasible menu items-->
            <div class="collapse navbar-collapse" id="navbarNav">
                <ul class="navbar-nav ms-auto"> <!--ms-auto pushes
links to the right-->
                    <!--home link (active on this page)-->
                    <li class="nav-item">
                        <a class="nav-link active"
href="portfolio.html">Home</a>
                    </li>
                    <!--About me link-->
                    <li class="nav-item">
                        <a class="nav-link" href="about.html">About
Me</a>
                    </li>
                    <!--Projects link-->
                    <li class="nav-item">
                        <a class="nav-link"
href="projects.html">Projects</a>
                    </li>
                    <!--Contacts link-->
                    <li class="nav-item">
                        <a class="nav-link"
href="contact.html">Contact</a>
                    </li>
                </ul>
            </div>
        </nav>
        <main class="flex-grow-1"> <!--flex-grow-1 means:
                                    If the parent container uses
display: flex, this element will expand to take up available space.
                                    It grows proportionally compared to
other flex items.
                                    Useful when you want the main
content to stretch and fill the page between header and footer.-->
```

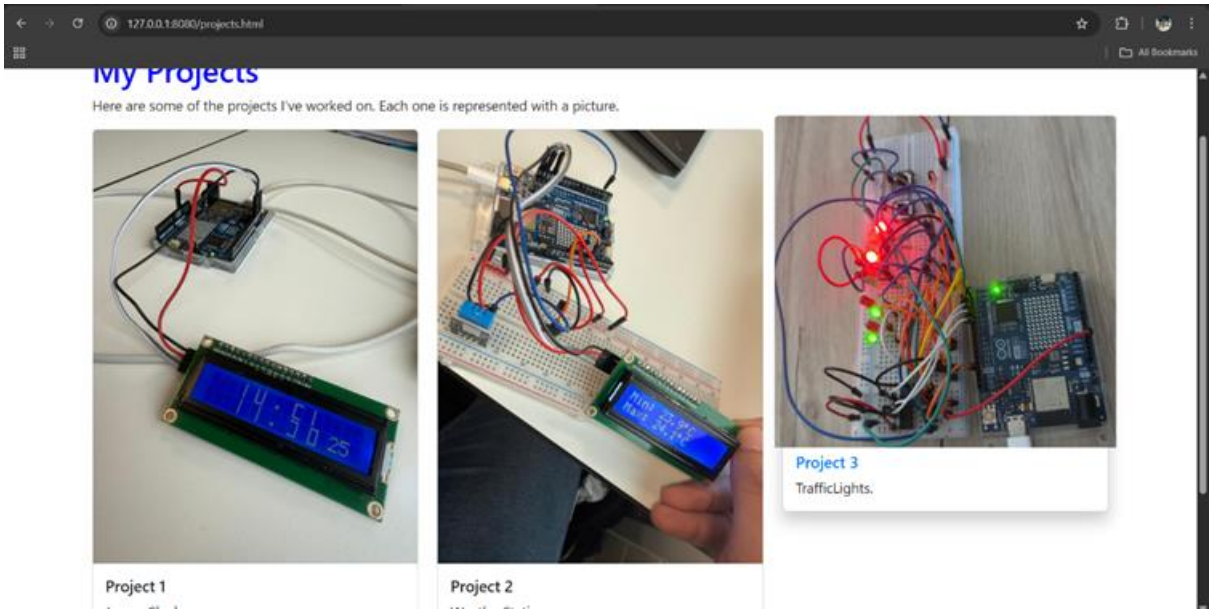
```
                <div class="jumbotron text-center bg-light p-5"> <!--
jumbotron is a Bootstrap component for a prominent header section-->
                    <h1 class="display-4">Hi, I'm Slman</h1>
                    <p class="lead">CS Student at NHL Stenden Emmen</p>
<!-- lead is typography utility class make the text stand out-->
                    <a class="btn btn-primary btn-lg"
href="projects.html" role="button">See My Projects</a> <!-- btn-lg
makes the button larger
                    btn_primary for the color blue - the role makes the
reader treats this link as button-->
                </div>

        </main>
        <!-- Footer -->
        <footer class="bg-dark text-white text-center p-3 mt-5"> <!--
bg-dark: dark background / text-white: white text / text-center: center
aligned text / p-3: padding scale 3 / mt-5: margin top scale 5-->
            <p class="highlight">&copy; 2025 Slman | CS Student at NHL
Stenden Emmen</p>
            <a href="contact.html" class="text-white">Contact Me</a>
        </footer>
    </body>
</html>
```

12) Add hover element to the projects page:

Image:






```
<!DOCTYPE html>
<html lang="en">

    <head>
        <!-- Bootstrap CSS & JS -->
        <link
href="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/css/bootstrap.min.css"
rel="stylesheet" integrity="sha384-
QWTKZyjpPEjISv5WaRU9OFeRpok6YctnYmDr5pNlyT2bRjXh0JMhjY6hW+ALEwIH"
crossorigin="anonymous">
        <script
src="https://cdn.jsdelivr.net/npm/bootstrap@5.3.3/dist/js/bootstrap.bundle.min
.js" integrity="sha384-
YvpcrYf0tY3lHB60NNkmXc5s9fDVZLESaAA55NDzOxhy9GkcIdslK1eN7N6jIeHz"
crossorigin="anonymous"></script>

        <link href="styles.css" rel="stylesheet">
        <title>Projects</title>
    </head>

    <body>
        <!-- Navigation bar -->
        <nav class="navbar navbar-expand-lg navbar-dark bg-dark">
            <a class="navbar-brand" href="index.html">My CS Journey</a>
            <button class="navbar-toggler" type="button" data-bs-
toggle="collapse" data-bs-target="#navbarNav" aria-controls="navbarNav" aria-
expanded="false" aria-label="Toggle navigation">
                <span class="navbar-toggler-icon"></span>
            </button>
            <div class="collapse navbar-collapse" id="navbarNav">
                <ul class="navbar-nav ms-auto">
```

```
                    <li class="nav-item"><a class="nav-link"
href="index.html">Home</a></li>
                    <li class="nav-item"><a class="nav-link"
href="about.html">About Me</a></li>
                    <li class="nav-item"><a class="nav-link active"
href="projects.html">Projects</a></li>
                    <li class="nav-item"><a class="nav-link"
href="contact.html">Contact</a></li>
                </ul>
            </div>
        </nav>

        <!--  Page content -->
        <main class="flex-grow-1">
            <div class="container mt-4">
                <h1>My Projects</h1>
                <p>Here are some of the projects I've worked on. Each one is
represented with a picture.</p>

                <!-- Gallery grid -->
                <div class="row">
                    <!-- Project 1 -->
                    <div class="col-md-4 mb-4"> <!--col-md-4: margin bottom.
in medium+ screens, three of these side by side will fill the row
                                                mb : medium breakpoint. Adds
extra space below the column, so multiple rows of cards don't look cramped.-->
                        <div class="card">
                            <img src="images/jumpoclock.jpg" class="card-img-
top" alt="Project 1"width="400" height="400"> <!-- card-img-top: Makes the
image span the full width of the card.

Rounds the top corners to match the card's border radius.

Ensures consistent sizing and alignment with other card components.-->
                            <div class="card-body">
                                <h5 class="card-title">Project 1</h5>
                                <p class="card-text">JumpoClock.</p>
                            </div>
                        </div>
                    </div>

                    <!-- Project 2 -->
                    <div class="col-md-4 mb-4">
                        <div class="card">
                            <img src="images/weather_station.jpg" class="card-
img-top" alt="Project 2" width="400" height="400">
                            <div class="card-body">
                                <h5 class="card-title">Project 2</h5>
```

```
                            <p class="card-text">WeatherStation.</p>
                        </div>
                    </div>
                </div>

                <!-- Project 3 -->
                <div class="col-md-4 mb-4">
                    <div class="card">
                        <img src="images/traffic lights.jpg" class="card-
img-top" alt="Project 3">
                        <div class="card-body">
                            <h5 class="card-title">Project 3</h5>
                            <p class="card-text">TrafficLights.</p>
                        </div>
                    </div>
                </div>
            </div>
        </div>
    </main>
    <!--  Footer -->
    <footer class="bg-dark text-white text-center p-3 mt-5">
        <p class="highlight">&copy; 2025 Slman | CS Student at NHL
Stenden</p>
        <a href="contact.html" class="text-white">Contact Me</a>
    </footer>
</body>

</html>
```

Style.css file:

```
/* ============================
   Global Styles
   ============================ */
/* Make sure the body takes full height */
html, body {
  height: 100%;
  margin: 0;
}

/* Page container for centered content */
.page-container {
    background: #fff;
    border: 3px solid #ccc;
```

```
    border-radius: 12px;
    padding: 30px;
    max-width: 800px;
    width: 90%;
    box-shadow: 0 6px 12px rgba(0,0,0,0.1);
    text-align: left;
}

/* ============================
   Typography
   ============================ */
h1 {
    color: blue;
}

h2 {
    color: red;
}

/* ============================
   Images
   ============================ */
.item img {
    width: 150px;
    height: 100px;
    object-fit: cover; /* crops excess while keeping aspect ratio */
    border: 3px solid #007bff;
    border-radius: 8px;
    margin: 10px;
}



/* ============================
   Layout Elements
   ============================ */
main {
    flex: 1; /* Main content grows to fill space */
}

nav ul li {
    list-style-type: none; /* Remove bullet points */
    margin-right: 15px;    /* Spacing between links */
}

/* ============================
   Flexbox Gallery
```

```
   ============================ */
.flex-gallery {
    display: flex;
    flex-wrap: nowrap;      /* Keep items in one row */
    overflow-x: auto;       /* Scroll horizontally if needed */
    gap: 10px;              /* Space between items */
    margin-top: 20px;
}

.flex-gallery .item img {
    max-width: 150px;
    border: 3px solid #007bff;
    border-radius: 8px;
}

/* ============================
   Grid Gallery
   ============================ */
.grid-gallery {
    display: grid;
    grid-template-columns: repeat(3, 1fr); /* 3 equal columns */
    gap: 20px;
    margin-top: 20px;
}

.grid-gallery .item img {
    width: 100%;
    border: 3px solid #007bff;
    border-radius: 8px;
}

/* ============================
   Position Experiment Boxes
   ============================ */
.position-box {
    width: 100px;
    height: 50px;
    margin: 10px;
    padding: 10px;
    background-color: #ffc107;
    border: 2px solid #333;
    text-align: center;
    font-weight: bold;
}

/* Specific position styles */
.static   { position: static; }
```

```
.relative { position: relative; top: 1px; left: 10px; right: 1px; bottom:
1px; }
.fixed    { position: fixed; top: 10px; right: 10px; }
.sticky   { position: sticky; top: 0; }
.inherit  { position: inherit; }

.absolute {
    position: absolute;
    top: 120px;  /* move it lower */
    right: 20px;
    z-index: 10; /* ensure it stays above other elements */
}

/* Main content grows to fill space */
main {
  flex: 1;
}

/* Footer stays at bottom */
footer {
  margin-top: auto;
}

/* Tag selector */
p {
  color: #333;              /* text color */
  font-size: 18px;          /* font size */
}

/* Class selector */
.highlight {
  background-color: yellow; /* highlight background */
  padding: 5px;             /* spacing inside */
}

/* Pseudo-class selector */
a:hover {
  color: red;               /* change link color on hover */
  text-decoration: underline; /* underline on hover */
}

/* Use flexbox to push footer down */
body {
  display: flex;
  flex-direction: column;
}
```

```css
/* ============================
   Card Hover Effects
   ============================ */
.card {
    transition: transform 0.3s ease, box-shadow 0.3s ease;
}

.card:hover {
    transform: translateY(-8px); /* lift the card */
    box-shadow: 0 12px 20px rgba(0, 0, 0, 0.2); /* deeper shadow */
}

.card-img-top {
    transition: transform 0.3s ease;
}

.card:hover .card-img-top {
    transform: scale(1.05); /* zoom image slightly */
}

.card-title {
    transition: color 0.3s ease;
}

.card:hover .card-title {
    color: #007bff; /* change title color on hover */
}
```

Week 9: Flask:

Assignment 1: Birthdays

Create a web application to keep track of friends' birthdays

The app.py code:

```python
import os

from cs50 import SQL
from flask import Flask, redirect, render_template, request

# Configure application
app = Flask(__name__)
```

```
# Ensure templates are auto-reloaded
app.config["TEMPLATES_AUTO_RELOAD"] = True

# Configure CS50 Library to use SQLite database
db = SQL("sqlite:///birthdays.db")


@app.after_request
def after_request(response):
    """Ensure responses aren't cached"""
    response.headers["Cache-Control"] = "no-cache, no-store, must-revalidate"
    response.headers["Expires"] = 0
    response.headers["Pragma"] = "no-cache"
    return response


@app.route("/", methods=["GET", "POST"])
def index():
    if request.method == "POST":

        # Access the form data

        name = request.form.get("name")
        if not name:
            return redirect("/") # Redirect if name is empty

        month = request.form.get("month")
        if not month:
            return redirect("/") # Redirect if month is empty
        try:
            month = int(month)
        except ValueError: # catch if the month is not an integer and redirect
rather than crashing the app
            return redirect("/") # Redirect if month is not an integer
        if month < 1 or month > 12:
            return redirect("/") # Redirect if month is out of range

        day = request.form.get("day")
        if not day:
            return redirect("/") # Redirect if day is empty
        try:
            day = int(day)
        except ValueError:
            return redirect("/") # Redirect if day is not an integer
        if day < 1 or day > 31:
```

```
            return redirect("/") # Redirect if day is out of range

        #insert data into the database
        db.execute("INSERT INTO birthdays (name, month, day) VALUES(?, ?, ?)",
name, month, day)

        return redirect("/")

    else:

        # Display the entries in the database on index.html

        # Query the database for all birthdays
        birthdays = db.execute("SELECT * FROM birthdays")

        # render the birthdays page
        return render_template("index.html", birthdays=birthdays) # Render the
HTML file and give it a variable called birthdays,
                                                                  # and the
value of that variable should be the Python list stored in birthdays
```

The index.html code:

```
<!DOCTYPE html>

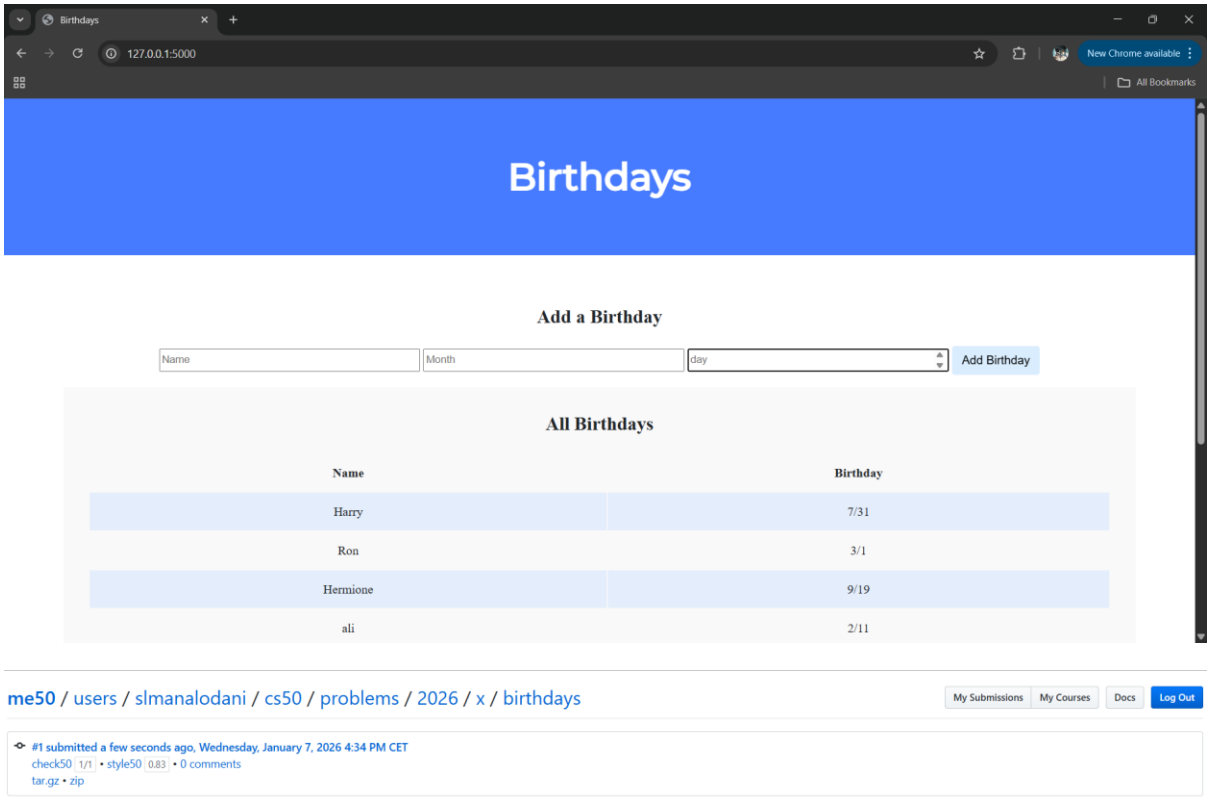
<html lang="en">
    <head>
        <link
href="https://fonts.googleapis.com/css2?family=Montserrat:wght@500&display=swa
p" rel="stylesheet">
        <link href="/static/styles.css" rel="stylesheet">
        <title>Birthdays</title>
    </head>
    <body>
        <div class="header">
            <h1>Birthdays</h1>
        </div>
        <div class="container">
            <div class="section">
```

```
                <h2>Add a Birthday</h2>
                <!--Create a form for users to submit a name, a month, and a
day -->
                <form action="/" method="post"> <!-- here we decide which
route should be requisted after the form is submitted-->
                    <input name="name" placeholder="Name" type="text"
required>
                    <input name="month" placeholder="Month" type="number"
min="1" max="12" required>
                    <input name="day" placeholder="day" type="number" min="1"
max="31" required>
                    <input type="submit" value="Add Birthday">
                </form>
            </div>

            <div class="section">

                <h2>All Birthdays</h2>
                <table>
                    <thead> <!-- the header of the table -->
                        <tr> <!-- table row -->
                            <th>Name</th> <!-- table header cell-->
                            <th>Birthday</th>
                        </tr>
                    </thead>
                    <tbody> <!-- the body of the table-->
                        <!--Loop through the database entries to display them
in this table -->
                        {% for birthday in birthdays %}
                        <tr>
                            <td>{{ birthday.name }}</td> <!-- table data cell-
->
                            <td>{{ birthday.month }}/{{ birthday.day }}</td>
                        </tr>
                        {% endfor %}
                    </tbody>
                </table>
            </div>
        </div>
    </body>
</html>
```

Image of the webpage:



https://submit.cs50.io/check50/86be2c3e2321eb759535d79a8f5926ed7ea5b5ed

Assignment 2: Finance

Implement a website via which users can “buy” and “sell” stocks.

The app.py code:

```
import os

from cs50 import SQL
from flask import Flask, flash, redirect, render_template, request, session
from flask_session import Session
from werkzeug.security import check_password_hash, generate_password_hash

from helpers import apology, login_required, lookup, usd

# Configure application
app = Flask(__name__)
```

```
# Custom filter
app.jinja_env.filters["usd"] = usd

# Configure session to use filesystem (instead of signed cookies)
app.config["SESSION_PERMANENT"] = False
app.config["SESSION_TYPE"] = "filesystem"
Session(app)

# Configure CS50 Library to use SQLite database
db = SQL("sqlite:///finance.db")


@app.after_request
def after_request(response):
    """Ensure responses aren't cached"""
    response.headers["Cache-Control"] = "no-cache, no-store, must-revalidate"
    response.headers["Expires"] = 0
    response.headers["Pragma"] = "no-cache"
    return response


@app.route("/")
@login_required
def index():
    """Show portfolio of stocks"""
    user_id = session["user_id"]

    # get the user's stocks from the database
    rows = db.execute(
        "SELECT symbol, SUM(shares) as total_shares FROM transactions WHERE
user_id = ? GROUP BY symbol",
        user_id
    )

    print(rows)  # for debugging

    portfolio = [] # list to hold stock info

    # look up each stock's current price
    for row in rows:
        stock = lookup(row["symbol"])
        price = stock["price"]
        total_value = price * row["total_shares"]

        portfolio.append({
```

```
            "symbol": row["symbol"],
            "shares": row["total_shares"],
            "price": price,
            "total_value": total_value
        })
    # get user's cash balance
    cash = db.execute("SELECT cash FROM users WHERE id = ?",
user_id)[0]["cash"]

    # calculate total portfolio value
    total_portfolio_value = cash + sum(item["total_value"] for item in
portfolio)

    return render_template("index.html", portfolio=portfolio, cash=cash,
total_portfolio_value=total_portfolio_value)

@app.route("/buy", methods=["GET", "POST"])
@login_required
def buy():
    """Buy shares of stock"""
    if request.method == "GET":
        return render_template("buy.html")
    else:
        # to handle POST request
        symbol = request.form.get("symbol")
        if not symbol:
            return apology("must provide symbol")
        shares = request.form.get("shares")
        if not shares or not shares.isdigit() or int(shares) <= 0:
            return apology("must provide a positive integer for shares")
        shares = int(shares)

        # look up the stock
        stock = lookup(symbol)
        if stock is None:
            return apology("invalid symbol")

        # calculate total cost
        price = stock["price"]
        total_cost = price * shares

        # check if user has enough cash
        user_id = session["user_id"]
        cash = db.execute("SELECT cash FROM users WHERE id = ?",
user_id)[0]["cash"] # we getting a list of dicts, so we access the first
element and then the "cash" key
```

```
        if total_cost > cash:
            return apology("insufficient funds")

        # record the transaction
        db.execute(
            "INSERT INTO transactions (user_id, symbol, shares, price, type)
VALUES(?, ?, ?, ?, ?)",
            user_id, symbol, shares, price, "buy"
            )
        # update user's cash
        db.execute(
            "UPDATE users SET cash = cash - ? WHERE id = ?",
            total_cost, user_id
        )

        # redirect to home page
        return redirect("/")

@app.route("/history")
@login_required
def history():
    """Show history of transactions"""
    user_id = session["user_id"]

    # get user's transactions from the database
    rows = db.execute(
        "SELECT symbol, shares, price, type, timestamp FROM transactions WHERE
user_id = ? ORDER BY timestamp DESC",
        user_id
    )
    return render_template("history.html", transactions=rows)



@app.route("/login", methods=["GET", "POST"])
def login():
    """Log user in"""

    # Forget any user_id
    session.clear()

    # User reached route via POST (as by submitting a form via POST)
    if request.method == "POST":
        # Ensure username was submitted
        if not request.form.get("username"):
            return apology("must provide username", 403)
```

```
        # Ensure password was submitted
        elif not request.form.get("password"):
            return apology("must provide password", 403)

        # Query database for username
        rows = db.execute(
            "SELECT * FROM users WHERE username = ?",
request.form.get("username")
        )

        # Ensure username exists and password is correct
        if len(rows) != 1 or not check_password_hash(
            rows[0]["hash"], request.form.get("password")
        ):
            return apology("invalid username and/or password", 403)

        # Remember which user has logged in
        session["user_id"] = rows[0]["id"]

        # Redirect user to home page
        return redirect("/")

    # User reached route via GET (as by clicking a link or via redirect)
    else:
        return render_template("login.html")


@app.route("/logout")
def logout():
    """Log user out"""

    # Forget any user_id
    session.clear()

    # Redirect user to login form
    return redirect("/")


@app.route("/quote", methods=["GET", "POST"])
@login_required
def quote():
    """Get stock quote."""
    if request.method == "GET":
        return render_template("quote.html")
    else:
```

```
        # to handle POST request
        symbol = request.form.get("symbol")
        if not symbol:
            return apology("must provide symbol")
        # look up the stock
        stock = lookup(symbol)
        if stock is None:
            return apology("invalid symbol")
        # render quoted.html with stock info
        return render_template("quoted.html", stock=stock)

@app.route("/register", methods=["GET", "POST"])
def register():
    """Register user"""
    if request.method == "GET":
        return render_template("register.html")
    else:
        # to handle post request
        username = request.form.get("username")
        if not username:
            return apology("must provide username")
        # validate password and confirmation
        password = request.form.get("password")
        confirmation = request.form.get("confirmation")

        if not password or not confirmation:
            return apology("must provide password and confirmation")

        if password != confirmation:
            return apology("password and confirmation do not match")

        # hash the password
        hash = generate_password_hash(password)

        # insert new user into db
        try:
            new_user_id = db.execute(
                "INSERT INTO users (username, hash) VALUES(?, ?)",
                username, hash
            )
        except ValueError:
            return apology("username already exists")

        # log in the new user
        session["user_id"] = new_user_id
        return redirect("/")
```

```
@app.route("/sell", methods=["GET", "POST"])
@login_required
def sell():
    """Sell shares of stock"""
    user_id = session["user_id"]

    if request.method == "GET":

        # get user's stocks from the database
        rows = db.execute(
            "SELECT symbol, SUM(shares) AS total_shares FROM transactions
WHERE user_id = ? GROUP BY symbol HAVING total_shares > 0",
            user_id
        )
        return render_template("sell.html", portfolio=rows)
    else:
        # to handle POST request
        symbol = request.form.get("symbol")
        shares = request.form.get("shares")

        # validate symbol
        if not symbol:
            return apology("must provide symbol")

        # validate shares
        try:
            shares = int(shares)
            if shares <= 0:
                return apology("must provide a positive integer for shares")
        except:
            return apology("must provide a positive integer for shares")

        # check if user owns enough shares
        rows = db.execute(
            "SELECT SUM(shares) AS total_shares FROM transactions WHERE
user_id = ? AND symbol = ?",
            user_id, symbol
        )
        owned = rows[0]["total_shares"]

        if owned is None or shares > owned:
            return apology("insufficient shares")

        # look up the stock
```

```
        stock = lookup(symbol)
        price = stock["price"]

        # insert the negative transaction
        db.execute(
            "INSERT INTO transactions (user_id, symbol, shares, price , type)
VALUES(?, ?, ?, ?, ?)",
            user_id, symbol, -shares, price, "sell"
        )
        # add cash to user's account
        db.execute(
            "UPDATE users SET cash = cash + ? WHERE id = ?",
            price * shares, user_id
        )
  return redirect("/")

# personal touch (adding cash)

@app.route("/add_cash", methods=["GET", "POST"])
@login_required
def add_cash():
    user_id = session["user_id"]

    if request.method == "GET":
        return render_template("add_cash.html")

    else:
        amount = request.form.get("amount")

        # validate amount
        try:
            amount = float(amount)
            if amount <= 0:
                return apology("amount must be positive")
        except:
            return apology("invalid amount")

        # update user's cash
        db.execute(
            "UPDATE users SET cash = cash + ? WHERE id = ?",
            amount, user_id
        )

        return redirect("/")
```

The index.html code:

```
{% extends "layout.html" %}

{% block title %}
    Index
{% endblock %}

{% block main %}

<table class="table table-striped">
    <thead>
        <tr>
            <th>Symbol</th>
            <th>Shares</th>
            <th>Price</th>
            <th>Total Value</th>
        </tr>
    </thead>
    <tbody>
        {% for stock in portfolio %}
        <tr>
            <td>{{ stock.symbol }}</td>
            <td>{{ stock.shares }}</td>
            <td>{{ stock.price | usd }}</td>
            <td>{{ stock.total_value | usd }}</td>
        </tr>
        {% endfor %}
    </tbody>
</table>

<div class="mt-4">
    <h4>Cash: {{ cash | usd }}</h4>
    <h4>Total Portfolio Value: {{ total_portfolio_value | usd }}</h4>
</div>

{% endblock %}
```

The buy.html code:

```
{% extends "layout.html" %}

{% block title %}
    Buy
{% endblock %}

{% block main %}
    <form action="/buy" method="post">
        <div class="mb-3">
            <input autocomplete="off" autofocus class="form-control mx-
auto w-auto" name="symbol" placeholder="Symbol" type="text">
        </div>
        <div class="mb-3">
            <input class="form-control mx-auto w-auto" name="shares"
placeholder="Shares" type="number" min="1">
        </div>
        <button class="btn btn-primary" type="submit">Buy</button>
    </form>
{% endblock %}
```

The history.html code:

```
{% extends "layout.html" %}

{% block title %}
    Buy
{% endblock %}

{% block main %}
    <form action="/buy" method="post">
        <div class="mb-3">
            <input autocomplete="off" autofocus class="form-control mx-
auto w-auto" name="symbol" placeholder="Symbol" type="text">
        </div>
        <div class="mb-3">
            <input class="form-control mx-auto w-auto" name="shares"
placeholder="Shares" type="number" min="1">
        </div>
```

```
        <button class="btn btn-primary" type="submit">Buy</button>
    </form>
{% endblock %}
```

The Quote.html code:

```
{% extends "layout.html" %}

{% block title %}
    Quote
{% endblock %}

{% block main %}
    <form action="/quote" method="post">
        <input autocomplete="off" autofocus name="symbol"
placeholder="Symbol" type="text">
        <button type="submit">Look up</button>
    </form>
{% endblock %}
```

The Quoted.html Code:

```
{% extends "layout.html" %}

{% block title %}
    Quoted
{% endblock %}

{% block main %}
    <h1>Quote Results</h1>
    <p>Name: {{ stock.name }}</p>
    <p>Symbol: {{ stock.symbol }}</p>
    <p>Price: {{ stock.price | usd }}</p>
{% endblock %}
```

The Register.html Code:

```
{% extends "layout.html" %}
```

```
{% block title %}
    Register
{% endblock %}

{% block main %}
    <form action="/register" method="post">
        <div class="mb-3">
            <input autocomplete="off" autofocus class="form-control mx-
auto w-auto" name="username" placeholder="Username" type="text">
        </div>
        <div class="mb-3">
            <input class="form-control mx-auto w-auto" name="password"
placeholder="Password" type="password">
        </div>
        <div class="mb-3">
            <input class="form-control mx-auto w-auto"
name="confirmation" placeholder="Password (again)" type="password">
        </div>

        <button class="btn btn-primary" type="submit">Register</button>
    </form>
{% endblock %}
```

The sell.html code:

```
{% extends "layout.html" %}

{% block title %}
    Sell
{% endblock %}

{% block main %}
<form action="/sell" method="post">
    <div class="mb-3">
        <select class="form-select" name="symbol">
            <option disabled selected value="">Select stock</option>
            {% for stock in portfolio %}
                <option
value="{{ stock.symbol }}">{{ stock.symbol }}</option>
```

```
            {% endfor %}
        </select>
    </div>
    <div class="mb-3">
        <input class="form-control" name="shares" placeholder="Shares"
type="number" min="1">
    </div>
    <button class="btn btn-primary" type="submit">Sell</button>
</form>
{% endblock %}
```

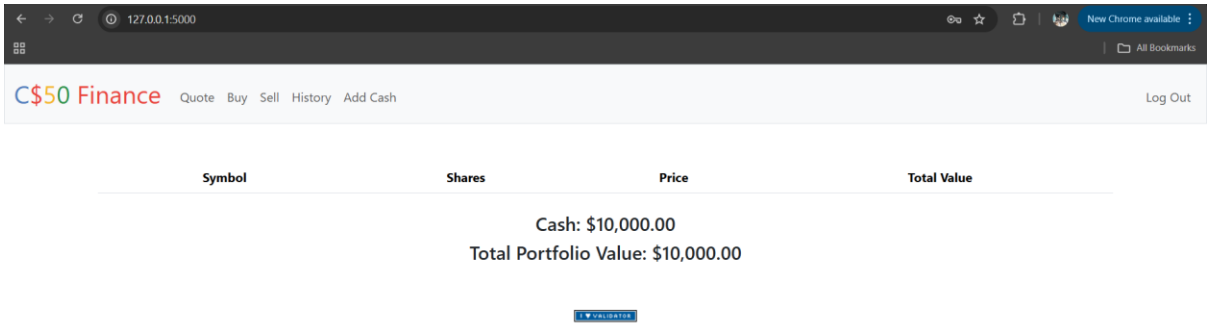
Add cash.html code:

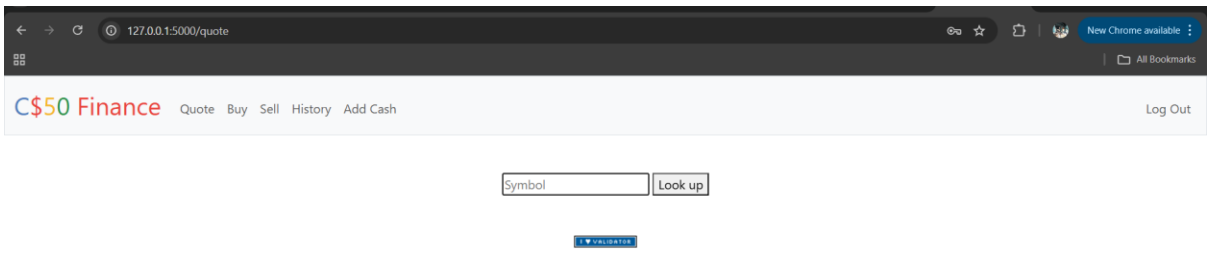
```
{% extends "layout.html" %}

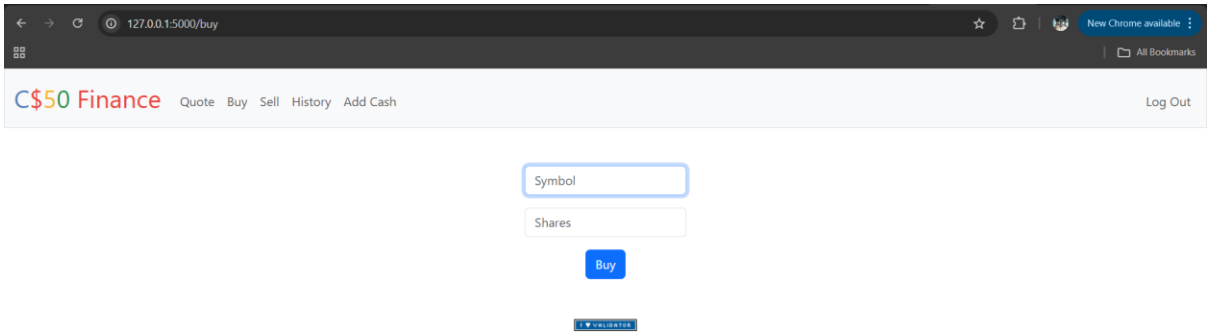
{% block title %}
    Add Cash
{% endblock %}

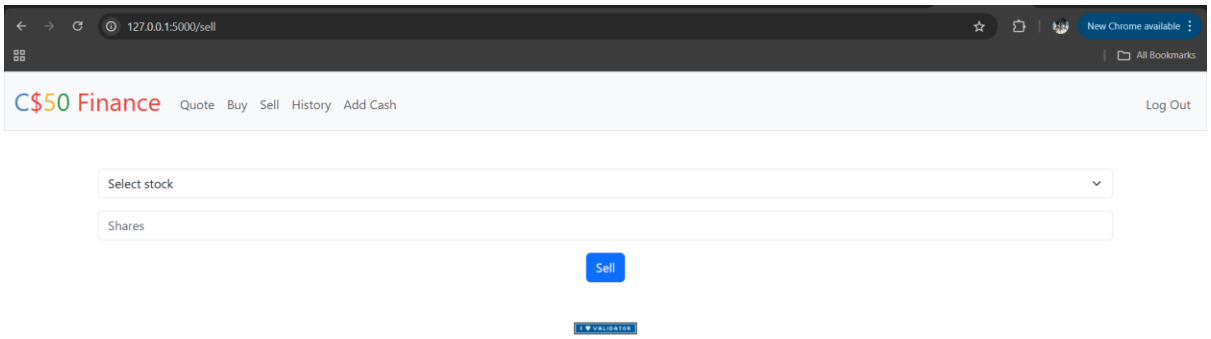
{% block main %}
<form action="/add_cash" method="post">
    <div class="mb-3">
        <input class="form-control" name="amount" placeholder="Amount
to add" type="number" step="0.01" min="0">
    </div>
    <button class="btn btn-primary" type="submit">Add Cash</button>
</form>
{% endblock %}
```

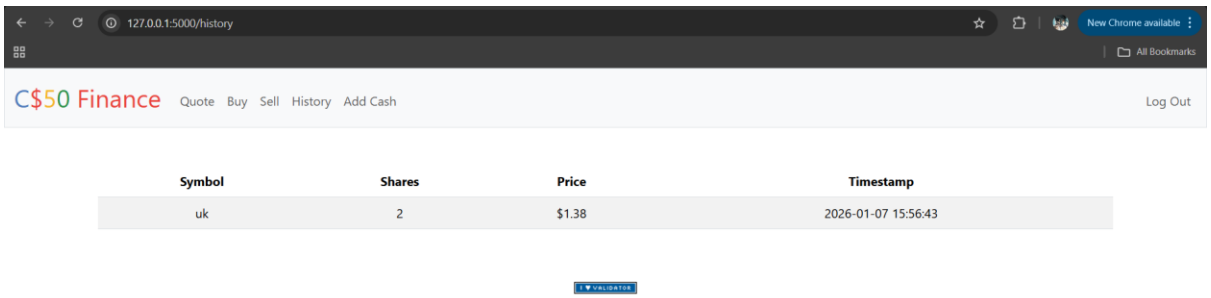
Image of the webpage

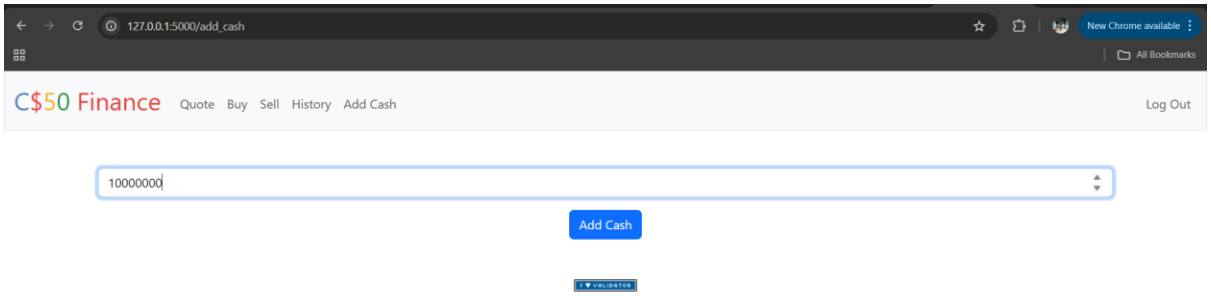

127.0.0.1:5000
New Chrome available
All Bookmarks
C$50 Finance
Quote
Buy
Sell
History
Add Cash
Log Out
Symbol
Shares
Price
Total Value
Cash: $10,000.00
Total Portfolio Value: $10,000.00

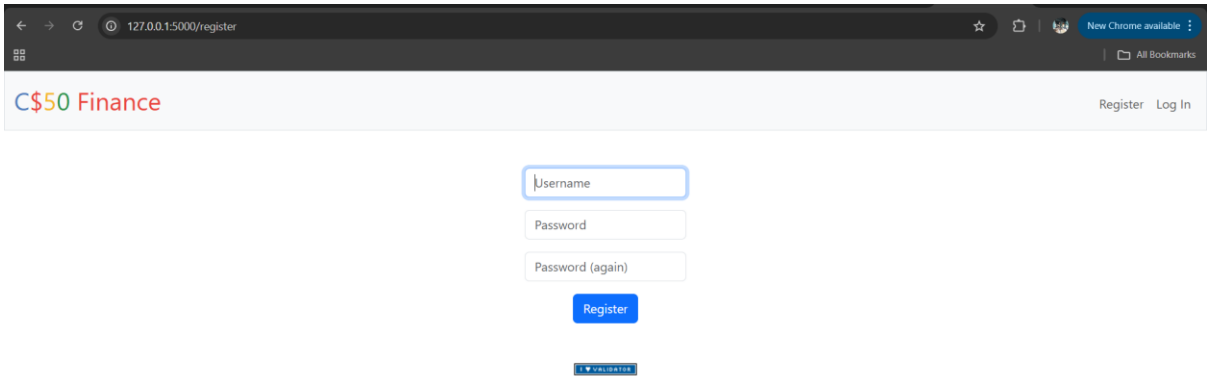

127.0.0.1:5000/quote
New Chrome available
All Bookmarks
C$50 Finance
Quote
Buy
Sell
History
Add Cash
Log Out
Symbol
Look up

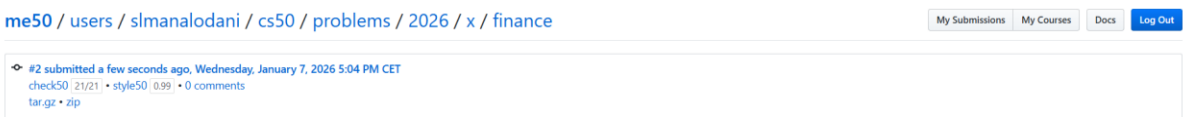

127.0.0.1:5000/buy
New Chrome available
All Bookmarks
C$50 Finance
Quote
Buy
Sell
History
Add Cash
Log Out
Symbol
Shares
Buy


127.0.0.1:5000/sell
New Chrome available
All Bookmarks
C$50 Finance
Quote
Buy
Sell
History
Add Cash
Log Out
Select stock
Shares
Sell


127.0.0.1:5000/history
New Chrome available
All Bookmarks
C$50 Finance
Quote
Buy
Sell
History
Add Cash
Log Out
Symbol
Shares
Price
Timestamp
uk
2
$1.38
2026-01-07 15:56:43







https://submit.cs50.io/check50/92aa70e3310f642f27aec59c017049a7e75cfb35

Embedded Systems assignments:

Basic movements (functions):

I have to program the relay bot to make basic movements which are:

Move forward for 1 meter

Move backward to where we started

Do a 90-degree curve to the right

Do a 90-degree curve to the left

Do a 180 turn around its axes

The Code:

```
#include <Arduino.h>

//------------ Motor Control Pins ------------
const int MOTORA1 = 6;
const int MOTORA2 = 5;
const int MOTORB1 = 10;
const int MOTORB2 = 9;
// motor A control leftwheel - motor B control rightwheel

//------------ Function Prototypes -----------
void motorStop();
void motorForward();
void motorBackward();
void curveRight90();
void curveLeft90();
void turnAround180();

void setup() {

  pinMode(MOTORA1, OUTPUT);
  pinMode(MOTORA2, OUTPUT);
  pinMode(MOTORB1, OUTPUT);
  pinMode(MOTORB2, OUTPUT);

}

void loop() {

  motorForward();
  motorBackward();
  curveRight90();
  curveLeft90();
  turnAround180();

}
//------------ Motor Control -----------------
void motorStop() {
```

```
  // strong break
  digitalWrite(MOTORA1, HIGH);
  digitalWrite(MOTORA2, HIGH);
  digitalWrite(MOTORB1, HIGH);
  digitalWrite(MOTORB2, HIGH);

  unsigned long stopStartTime = millis();
  while (millis() - stopStartTime < 3000) {
  }
}

void motorForward() {

  const int SPEEDMOTORA = 215;
  const int SPEEDMOTORB = 200;

  // left wheel
  analogWrite(MOTORA1, 0);
  analogWrite(MOTORA2, SPEEDMOTORA);

  // Right wheel
  analogWrite(MOTORB1, SPEEDMOTORB);
  analogWrite(MOTORB2, 0);

  unsigned long stopStartTime = millis();
  while (millis() - stopStartTime < 5000) {
  }
  motorStop();

}

void motorBackward() {

  const int SPEEDMOTORA = 215;
  const int SPEEDMOTORB = 215;

  // left wheel
  analogWrite(MOTORA1, SPEEDMOTORA);
  analogWrite(MOTORA2, 0);

  // right wheel
  analogWrite(MOTORB1, 0);
  analogWrite(MOTORB2, SPEEDMOTORB);

  unsigned long stopStartTime = millis();
```

```
  while (millis() - stopStartTime < 5200) {
  }
  motorStop();
}

void curveRight90() {

  const int SPEEDMOTORA = 255;
  const int SPEEDMOTORB = 150;

  // Left wheel
  analogWrite(MOTORA1, 0);
  analogWrite(MOTORA2, SPEEDMOTORA);

  // Right wheel
  analogWrite(MOTORB1, SPEEDMOTORB);
  analogWrite(MOTORB2, 0);

  unsigned long stopStartTime = millis();
  while (millis() - stopStartTime < 2000) {
  }
  motorStop();

}
void curveLeft90() {

  const int SPEEDMOTORA = 180;
  const int SPEEDMOTORB = 255;

  // Left wheel
  analogWrite(MOTORA1, 0);
  analogWrite(MOTORA2, SPEEDMOTORA);

  // Right wheel
  analogWrite(MOTORB1, SPEEDMOTORB);
  analogWrite(MOTORB2, 0);

  unsigned long stopStartTime = millis();
  while (millis() - stopStartTime < 2500) {
  }
  motorStop();

}

void turnAround180() {
```

```
  const int SPEEDMOTORA = 220;
  const int SPEEDMOTORB = 220;

  // Left wheel forward
  analogWrite(MOTORA1, 0);
  analogWrite(MOTORA2, SPEEDMOTORA);

  // Right wheel backward
  analogWrite(MOTORB1, 0);
  analogWrite(MOTORB2, SPEEDMOTORB);

  unsigned long stopStartTime = millis();
  while (millis() - stopStartTime < 1370) {
  }
  motorStop();

}
```

Object avoidance:

The assignment is to make the relay bot avoid an object using the ultrasonic sensor

Code:

```
#include <Arduino.h>

//------------ Motor Control Pins ------------
const int MOTORA1 = 6;
const int MOTORA2 = 5;
const int MOTORB1 = 10;
const int MOTORB2 = 9;
// motor A control leftwheel - motor B control rightwheel

//----------- Ultra Sonic Sensor Pins --------
const int TRIG = 7;
const int ECHO = 8;

//-------------Motor Sensor Pins-----------
const int MOTORLEFTSENSOR = 3;
const int MOTORRIGHTSENSOR = 2;
```

```
volatile long motorLeft_count = 0;
volatile long motorRight_count = 0;

//------------ Function Prototypes -----------
void motorStop();
void motorForward();
void motorBackward();
void curveRight90();
void curveLeft90();
void turnAround180();
float ultraSonicSensor();
void movingForwardAvoidObstacle();
void obstacleRoutine();
void motorCountleft();
void motorCountRight();
void CountReset();


void setup() {

  // Serial.begin(9600);
  pinMode(MOTORA1, OUTPUT);
  pinMode(MOTORA2, OUTPUT);
  pinMode(MOTORB1, OUTPUT);
  pinMode(MOTORB2, OUTPUT);

  pinMode(TRIG, OUTPUT);
  pinMode(ECHO, INPUT);

  pinMode(MOTORLEFTSENSOR, INPUT_PULLUP);
  pinMode(MOTORRIGHTSENSOR, INPUT_PULLUP);

  attachInterrupt(digitalPinToInterrupt(MOTORLEFTSENSOR),  motorCountleft,
RISING);
  attachInterrupt(digitalPinToInterrupt(MOTORRIGHTSENSOR), motorCountRight,
RISING);

}

void loop() {
  movingForwardAvoidObstacle();
  // motorForward();
  // curveRight90();
  // motorBackward();
  // curveLeft90();
  // turnAround180();
}
```

```
//------------ Motor Control -----------------
void motorStop() {
  // break
  const int stopDuration = 3000;
  analogWrite(MOTORA1, 0);
  analogWrite(MOTORA2, 0);
  analogWrite(MOTORB1, 0);
  analogWrite(MOTORB2, 0);

  unsigned long startTime = millis();
  while (millis() - startTime < stopDuration){
  }
}

void motorForward() {

  const int SPEEDMOTORA = 200;
  const int SPEEDMOTORB = 185;

  // left wheel
  analogWrite(MOTORA1, 0);
  analogWrite(MOTORA2, SPEEDMOTORA);

  // Right wheel
  analogWrite(MOTORB1, SPEEDMOTORB);
  analogWrite(MOTORB2, 0);

}

void motorBackward() {
  CountReset();
  const int SPEEDMOTORA = 215;
  const int SPEEDMOTORB = 215;
  const int leftWheelDistance = 100;
  const int rightWheelDistance = 100;

  // left wheel
  analogWrite(MOTORA1, SPEEDMOTORA);
  analogWrite(MOTORA2, 0);
  // right wheel
  analogWrite(MOTORB1, 0);
  analogWrite(MOTORB2, SPEEDMOTORB);

  while (motorRight_count < rightWheelDistance && motorLeft_count <
leftWheelDistance){
  }
}
```

```
void curveRight90() {
  CountReset();
  const int SPEEDMOTORA = 255;
  const int SPEEDMOTORB = 150;
  const int leftWheelDistance = 50;
  const int rightWheelDistance = 50;

  // Left wheel
  analogWrite(MOTORA1, 0);
  analogWrite(MOTORA2, SPEEDMOTORA);
  // Right wheel
  analogWrite(MOTORB1, SPEEDMOTORB);
  analogWrite(MOTORB2, 0);

  while (motorRight_count < rightWheelDistance && motorLeft_count <
leftWheelDistance){
  }
}

void curveLeft90() {
  CountReset();

  const int SPEEDMOTORA = 200;
  const int SPEEDMOTORB = 255;
  const int leftWheelDistance = 220;
  const int rightWheelDistance = 220;

  // Left wheel
  analogWrite(MOTORA1, 0);
  analogWrite(MOTORA2, SPEEDMOTORA);

  // Right wheel
  analogWrite(MOTORB1, SPEEDMOTORB);
  analogWrite(MOTORB2, 0);
  while (motorRight_count < rightWheelDistance && motorLeft_count <
leftWheelDistance){
  }
}

void turnAround180() {

  const int SPEEDMOTORA = 220;
  const int SPEEDMOTORB = 220;

  // Left wheel forward
  analogWrite(MOTORA1, 0);
```

```
  analogWrite(MOTORA2, SPEEDMOTORA);

  // Right wheel backward
  analogWrite(MOTORB1, 0);
  analogWrite(MOTORB2, SPEEDMOTORB);

  delay(1370);
  motorStop();
}

float ultraSonicSensor(){    // this will measure the distance to an object

  float duration, distance;

  digitalWrite(TRIG, LOW);
  delayMicroseconds(2);
  digitalWrite(TRIG, HIGH);
  delayMicroseconds(10);
  digitalWrite(TRIG, LOW);

  duration = pulseIn(ECHO, HIGH, 30000); // 30ms timeout after that it retrun
0
  if(duration == 0){
    return -1;
  }
  distance = (duration*.0343) / 2; // in cm
  return distance;


}
void movingForwardAvoidObstacle() {
  float distance = ultraSonicSensor();

  if (distance > 9 || distance == -1) {
   motorForward();
  } else {
    obstacleRoutine();
  }
}
void obstacleRoutine() {
  bool clear = false;
  while(!clear){
    // Back up safely
    motorBackward();
    // Sequence: Right → Left → Left → Right
    curveRight90();
    curveLeft90();
    curveLeft90();
```

```
    curveRight90();
    clear = true;
  }
}
void motorCountleft() {
  motorLeft_count++;
}

void motorCountRight() {
  motorRight_count++;
}
void CountReset(){
  motorLeft_count = 0;
  motorRight_count = 0;
}
```

Stay on Track (Arrays):

The assignment is to make the relay Bot follow the line and stay on track.

The code:

```
#include <Arduino.h>

//------------ Motor Control Pins ------------
const int MOTORA1 = 6;
const int MOTORA2 = 5;
const int MOTORB1 = 10;
const int MOTORB2 = 9;
// motor A control leftwheel - motor B control rightwheel

//----------- Ultra Sonic Sensor Pins --------
const int TRIG = 7;
const int ECHO = 8;

//-------------Motor Sensor Pins-----------
const int MOTORLEFTSENSOR = 3;
const int MOTORRIGHTSENSOR = 2;

volatile long motorLeft_count = 0;
volatile long motorRight_count = 0;

//-------------Movement Distance-------------
const int backwardDistance = 100;
const int curveRight90Distance = 50;
```

```
const int curveLeft90Distance = 300;

//-------------Line Sesnors--------------
const int lineSensorPins[8] = {A0, A1, A2, A3, A4, A5, A6, A7}; //
lineSensorPins
int thresholds[8]; // Thresholds calculated during auto-callibration
int lastError = 0; // to remember the last turn after losing the line

//------------ Function Prototypes -----------
void motorStop();
void motorForward();
void motorForwardSlowly();
void motorBackward(int distance);
void curveRight90(int distance);
void curveLeft90(int distance);
void turnAround180();
float ultraSonicSensor();
void movingForwardAvoidObstacle();
void obstacleRoutine();
void motorCountleft();
void motorCountRight();
void CountReset();
void autoCalibrateSensors();
void lineFollowing();
bool sensorState(int i);
int computeLineError();

void setup() {

  //Serial.begin(9600);
  // set motor pins as OUTPUT
  pinMode(MOTORA1, OUTPUT);
  pinMode(MOTORA2, OUTPUT);
  pinMode(MOTORB1, OUTPUT);
  pinMode(MOTORB2, OUTPUT);

  pinMode(TRIG, OUTPUT);
  pinMode(ECHO, INPUT);

  pinMode(MOTORLEFTSENSOR, INPUT_PULLUP);
  pinMode(MOTORRIGHTSENSOR, INPUT_PULLUP);

  attachInterrupt(digitalPinToInterrupt(MOTORLEFTSENSOR),  motorCountleft,
RISING);
  attachInterrupt(digitalPinToInterrupt(MOTORRIGHTSENSOR), motorCountRight,
RISING);
```

```
  // set all the sensor pins as INPUT
  for (int i = 0; i < 8; i++) {
    pinMode(lineSensorPins[i], INPUT);
    thresholds[i] = 514; // fixed number for black detection
  }

  // initial state
  motorStop();
  //autoCalibrateSensors(); this is off for now
}

void loop() {
  lineFollowing();
  //movingForwardAvoidObstacle();
  // motorForward();
  // curveRight90();
  // motorBackward();
  // curveLeft90();
  // turnAround180();
}
//------------ Motor Control -----------------
void motorStop() {
  const int stopDuration = 100;
  unsigned long startTime = millis();
  while (millis() - startTime < stopDuration){
    analogWrite(MOTORA1, 0);
    analogWrite(MOTORA2, 0);
    analogWrite(MOTORB1, 0);
    analogWrite(MOTORB2, 0);
  }
}
void motorForwardSlowly(){

  const int SPEEDMOTORA = 190;
  const int SPEEDMOTORB = 175;

  // left wheel
  analogWrite(MOTORA1, 0);
  analogWrite(MOTORA2, SPEEDMOTORA);

  // Right wheel
  analogWrite(MOTORB1, SPEEDMOTORB);
  analogWrite(MOTORB2, 0);

}
```

```
void motorForward() {

  const int SPEEDMOTORA = 200;
  const int SPEEDMOTORB = 185;

  // left wheel
  analogWrite(MOTORA1, 0);
  analogWrite(MOTORA2, SPEEDMOTORA);

  // Right wheel
  analogWrite(MOTORB1, SPEEDMOTORB);
  analogWrite(MOTORB2, 0);

}

void motorBackward(int distance) {
  CountReset();
  const int SPEEDMOTORA = 215;
  const int SPEEDMOTORB = 215;
  while (motorRight_count <= distance && motorLeft_count <= distance){

    // left wheel
    analogWrite(MOTORA1, SPEEDMOTORA);
    analogWrite(MOTORA2, 0);
    // right wheel
    analogWrite(MOTORB1, 0);
    analogWrite(MOTORB2, SPEEDMOTORB);

  }
}

void curveRight90(int distance) {
  CountReset();
  const int SPEEDMOTORA = 255;
  const int SPEEDMOTORB = 150;
  while (motorRight_count <= distance && motorLeft_count <= distance){

    // Left wheel
    analogWrite(MOTORA1, 0);
    analogWrite(MOTORA2, SPEEDMOTORA);
    // Right wheel
    analogWrite(MOTORB1, SPEEDMOTORB);
    analogWrite(MOTORB2, 0);

  }
}
```

```
void curveLeft90(int distance) {
  CountReset();
  const int SPEEDMOTORA = 200;
  const int SPEEDMOTORB = 255;
  while (motorRight_count <= distance && motorLeft_count <= distance){

    // Left wheel
    analogWrite(MOTORA1, 0);
    analogWrite(MOTORA2, SPEEDMOTORA);

    // Right wheel
    analogWrite(MOTORB1, SPEEDMOTORB);
    analogWrite(MOTORB2, 0);
  }
}

void turnAround180() {

  const int SPEEDMOTORA = 220;
  const int SPEEDMOTORB = 220;

  // Left wheel forward
  analogWrite(MOTORA1, 0);
  analogWrite(MOTORA2, SPEEDMOTORA);

  // Right wheel backward
  analogWrite(MOTORB1, 0);
  analogWrite(MOTORB2, SPEEDMOTORB);

  delay(1370);
  motorStop();
}

float ultraSonicSensor(){    // this will measure the distance to an object

  float duration, distance;

  digitalWrite(TRIG, LOW);
  delayMicroseconds(2);
  digitalWrite(TRIG, HIGH);
  delayMicroseconds(10);
  digitalWrite(TRIG, LOW);
```

```
  duration = pulseIn(ECHO, HIGH, 30000); // 30ms timeout after that it retrun
0
  if(duration == 0){
    return -1;
  }
  distance = (duration*.0343) / 2; // in cm
  return distance;

}
void movingForwardAvoidObstacle() {
  float distance = ultraSonicSensor();

  if (distance >= 9 || distance == -1) {
   motorForward();
  } else {
    obstacleRoutine();
  }
}
void obstacleRoutine() {
  bool clear = false;
  while(!clear){
    // Back up safely

    motorBackward(backwardDistance);
    // Sequence: Right → Left → Left → Right
    curveRight90(curveRight90Distance);
    curveLeft90(curveLeft90Distance);
    curveLeft90(curveLeft90Distance);
    curveRight90(curveRight90Distance);
    clear = true;
  }
}
void motorCountleft() {
  motorLeft_count++;
}

void motorCountRight() {
  motorRight_count++;
}
void CountReset(){
  motorLeft_count = 0;
  motorRight_count = 0;
}
void autoCalibrateSensors(){ // this function: move forward slowly, detects 3
black lines and calculates the threshold

  int minValue[8]; // for white backgraoud
```

```
  int maxValue[8]; // for black line

  // set min values for all sensors to 1023
  // set max values for all sensors to 0
  for (int i = 0; i < 8; i++){
    minValue[i] = 1023;
    maxValue[i] = 0;
  }

  int lineCrossing = 0; // how many blacklines crossed so far
  bool onLine = false; // true if the robot is currently on a black line
  motorForwardSlowly();

  // Loop only handles sensor sampling and crossing detection
  while (lineCrossing < 10){
    bool blackDetected = false; // no sensor see black at the start of the
cycle

    for (int i = 0; i < 8; i++){ // loop to read every sensor
      int val = analogRead(lineSensorPins[i]);

      // track min and max values for each sensor
      if (val < minValue[i]){
        minValue[i] = val; // update min value
      }
      if (val > maxValue[i]){
        maxValue[i] = val; // update max value
      }

      // Dynamic black detection:
      // compare current reading against the mid point of min/m ax so far
      int dynamicThreshold = (minValue[i] + maxValue[i]) / 2;
      if (val > dynamicThreshold){
        blackDetected = true;
      }
    }
    // if entered black line (blackDetected = true, onLine = false) count one
crossing
    // if back to white (blackDeteced = false), reset onLine dectect next
black line
    if (blackDetected && !onLine){
      lineCrossing++;
      onLine = true;
    } else if (!blackDetected){
      onLine = false;
    }
```

```
  }
  // after the designated line crossings, compute thresholds for each sensor
  for (int i = 0; i < 8; i++){
    thresholds[i] = (minValue[i] + maxValue[i]) / 2;
  }
}
// check if a sensor see black or white
bool sensorState(int i){
  int val = analogRead(lineSensorPins[i]);
  return val > thresholds[i];
}
// compute line error using "weighted avg method"
int computeLineError(){
  long weightedSum = 0; // sum of index * sensor value
  long totalWeight = 0; // sum of sensor values

  for (int i = 0; i < 8; i++){
    int val = analogRead(lineSensorPins[i]);

    if (sensorState(i)) {
      weightedSum += i * val; // weight index by intensity
      totalWeight += val; // accumulate weight
    }
  }

  if (totalWeight == 0){         // down i am fliping the negative so it turn
the other way
    return (lastError > 0 ? -5 : 5 ); // no line detected from all sensors
keep turining in last known direction *these values can be tuned*
  }

  // compute weighted avarage index
  float avgIndex = (float)weightedSum / (float)totalWeight;

  // shift relative to cennter (between sensors 3 and 4)
  int error = (int)(avgIndex - 3.5);

  lastError = error; // remember last correction
  return error;

}
void lineFollowing(){
  // Continuously adjust motor speeds based on line position
  int error = computeLineError();

  const int BASE_SPEED = 180;   // base motor speed it was 150
```

```
  const int CORRECTION = 60;    // correction factor *values can be tuned*

  // Adjust speeds based on error
  int leftSpeed  = BASE_SPEED - (error * CORRECTION);
  int rightSpeed = BASE_SPEED + (error * CORRECTION);

  // Clamp speeds to valid range
  leftSpeed  = constrain(leftSpeed, 0, 255);
  rightSpeed = constrain(rightSpeed, 0, 255);

  // Drive motors with adjusted speeds
  analogWrite(MOTORA1, 0);
  analogWrite(MOTORA2, leftSpeed);
  analogWrite(MOTORB1, rightSpeed);
  analogWrite(MOTORB2, 0);

}
```

Midterm Race:

I used the same code from stay on track assignment.

The relay bot followed the line without any problems.

Test Day:

Things that were missing are:

- The dashboard
- The neo pixels
- The starting sequence
- After gripping the object, the bot turned to the left more than expected.

Things that went well:

- The line following
- The object avoidance
- Dropping the object at the end point.

The code below includes the starting sequence and the neo pixels that have been added to the relay Bot after the test.

```
#include <Adafruit_NeoPixel.h> // NeoPixel library

//------------ NeoPixel Setup ----------------
#define NEOPIXEL_PIN 4 // connected to D4
#define NUM_PIXELS 4 // 4 NeoPixels
unsigned long lastBlinkTime = 0;
bool blinkState = false;

Adafruit_NeoPixel pixels(NUM_PIXELS, NEOPIXEL_PIN, NEO_GRB + NEO_KHZ800);

//------------Bot Name-----------------------
String botName = "Unknown";

//------------ Motor Control Pins ------------
const int MOTORA1 = 6;
const int MOTORA2 = 5;
const int MOTORB1 = 10;
const int MOTORB2 = 9;
// motor A control leftwheel - motor B control rightwheel

//----------- Ultra Sonic Sensor Pins --------
const int TRIG = 7;
const int ECHO = 8;

//------------ Control the Gripper--------------
const int GRIPPER = 12;
const int GRIPPEROPEN = 2000;
const int GRIPPERCLOSE = 1225;

//------------ MotorSpeed---------------------
int leftSpeed = 0;
int rightSpeed = 0;

//------------Button ---------------
const int BUTTON1 = 11;
int buttonState = 0;
//-------------Motor Sensor Pins-----------
const int MOTORLEFTSENSOR = 3;
const int MOTORRIGHTSENSOR = 2;

volatile long motorLeft_count = 0;
volatile long motorRight_count = 0;

//-------------Movement Distance-------------
const int backwardDistance = 90;
const int curveRight90Distance = 50;
```

```
const int curveLeft90Distance = 180;
const int startingDistance = 1;
const int curveLeftAroundItSelfDistance = 40;

// --- Obstacle persistence counter ---
static int obstacleCounter = 0;   // counts consecutive "too close" readings
const int OBSTACLE_THRESHOLD = 15; // cm
const int OBSTACLE_CONFIRM = 3;    // how many cycles to confirm obstacle
const int STARTING_FLAG = 15;

//-------------Line Sesnors--------------
const int lineSensorPins[8] = {A0, A1, A2, A3, A4, A5, A6, A7}; //
lineSensorPins
int thresholds[8];
int lastError = 0; // to remember the last turn after losing the line

//------------Define the bot behavior state-----------------
enum BotState {  // PascalCase naming for the type
  STARTING_POINT, // UPPER_SNAKE_CASE for the values
  SEARCH_OBJECT,
  TURN_LEFT,
  FOLLOW_LINE,
  DROP_OBJECT,
  DONE
};
BotState currentState = STARTING_POINT; // start with searching for object

// covert enum to text for json
const char* stateToText(BotState state) {
  switch (state) {
    case STARTING_POINT:return "STARTING_POINT";
    case SEARCH_OBJECT: return "SEARCH_OBJECT";
    case TURN_LEFT:     return "TURN_LEFT";
    case FOLLOW_LINE:   return "FOLLOW_LINE";
    case DROP_OBJECT:   return "DROP_OBJECT";
    case DONE:          return "DONE";
    default:            return "UNKNOWN"; // for debugging
  }
}

//------------ Function Prototypes -----------
void motorStop();
void motorStrongStop();
void motorForward(int distance);
void motorForwardSlowly();
void motorBackward(int distance);
void curveRight90(int distance);
```

```
void curveLeft90(int distance);
float ultraSonicSensor();
void obstacleRoutine();
void motorCountleft();
void motorCountRight();
void CountReset();
void lineFollowing();
bool sensorState(int i);
int computeLineError();
void gripper(int pulse);
void runMission();
void curveLeftAroundItSelf(int distance);
void sendTelemetry(String botName, int leftSpeed, int rightSpeed, BotState
state);
void blinkStartingLights();
void movingForwardLights();
void movingLeftLights();
void movingRightLights();
void movingBackwordLights();

void setup() {

  Serial.begin(9600);
  // set motor pins as OUTPUT
  pinMode(MOTORA1, OUTPUT);
  pinMode(MOTORA2, OUTPUT);
  pinMode(MOTORB1, OUTPUT);
  pinMode(MOTORB2, OUTPUT);

  pinMode(GRIPPER, OUTPUT);

  pinMode(TRIG, OUTPUT);
  pinMode(ECHO, INPUT);
  pinMode(BUTTON1, INPUT);

  pinMode(MOTORLEFTSENSOR, INPUT_PULLUP);
  pinMode(MOTORRIGHTSENSOR, INPUT_PULLUP);

  attachInterrupt(digitalPinToInterrupt(MOTORLEFTSENSOR),  motorCountleft,
RISING);
  attachInterrupt(digitalPinToInterrupt(MOTORRIGHTSENSOR), motorCountRight,
RISING);

  // set all the sensor pins as INPUT
  for (int i = 0; i < 8; i++) {
    pinMode(lineSensorPins[i], INPUT);
```

```
    thresholds[i] = 515; // fixed number for black detection
  }

  //------------NeoPixel-----------------
  pixels.begin();
  pixels.show(); // Initialize all pixels to off

  // initial state
  motorStop();
  gripper(GRIPPEROPEN); // open the gripper at the start

}

void loop() {
  //read the state of the button
  buttonState = digitalRead(BUTTON1);
  // check if the button is pressed
  if (buttonState == LOW) {
    motorStop(); // stop the robot
  } else {
    runMission();
  }

}
//------------ Motor Control -----------------
void motorStrongStop(){
  const int stopDuration = 1000;
  unsigned long startTime = millis();
  while (millis() - startTime < stopDuration){
    digitalWrite(MOTORA1, HIGH);
    digitalWrite(MOTORA2, HIGH);
    digitalWrite(MOTORB1, HIGH);
    digitalWrite(MOTORB2, HIGH);
    leftSpeed = 0;
    rightSpeed = 0;

  }
}
void motorStop() {
  const int stopDuration = 50;
  unsigned long startTime = millis();
  while (millis() - startTime < stopDuration){
    analogWrite(MOTORA1, 0);
    analogWrite(MOTORA2, 0);
    analogWrite(MOTORB1, 0);
    analogWrite(MOTORB2, 0);
```

```
    leftSpeed = 0;
    rightSpeed = 0;
  }
}
void motorForwardSlowly(){

  const int SPEEDMOTORA = 200;
  const int SPEEDMOTORB = 175;

  // left wheel
  analogWrite(MOTORA1, 0);
  analogWrite(MOTORA2, SPEEDMOTORA);

  // Right wheel
  analogWrite(MOTORB1, SPEEDMOTORB);
  analogWrite(MOTORB2, 0);

  // update the global speed
  leftSpeed = SPEEDMOTORA;
  rightSpeed = SPEEDMOTORB;
}
void motorForward(int distance) {
  CountReset();  // reset encoders

  const int BASE_LEFT  = 225;   // base PWM for left motor
  const int BASE_RIGHT = 195;   // base PWM for right motor
  const float K = 0.3;          // proportional gain (tuneable)

  while (motorLeft_count <= distance || motorRight_count <= distance) {
    // Difference between wheels
    long diff = motorLeft_count - motorRight_count;

    // Apply proportional correction
    int left  = BASE_LEFT  - (diff * K);
    int right = BASE_RIGHT + (diff * K);

    left = constrain(left, 0, 225);
    right = constrain(right, 0, 195);

    // Drive motors forward
    analogWrite(MOTORA1, 0);
    analogWrite(MOTORA2, left);
    analogWrite(MOTORB1, right);
    analogWrite(MOTORB2, 0);

    // update the global speed
```

```
    leftSpeed = left;
    rightSpeed = right;
  }
}



void motorBackward(int distance) {
  CountReset();
  const int SPEEDMOTORA = 215;
  const int SPEEDMOTORB = 215;
  while (motorRight_count <= distance && motorLeft_count <= distance){

    // left wheel
    analogWrite(MOTORA1, SPEEDMOTORA);
    analogWrite(MOTORA2, 0);
    // right wheel
    analogWrite(MOTORB1, 0);
    analogWrite(MOTORB2, SPEEDMOTORB);

    // update the global speed
    leftSpeed = SPEEDMOTORA;
    rightSpeed = SPEEDMOTORB;

    //LED
    movingBackwordLights();

  }
}

void curveRight90(int distance) {
  CountReset();
  const int SPEEDMOTORA = 255;
  const int SPEEDMOTORB = 100;
  while (motorRight_count <= distance && motorLeft_count <= distance){

    // Left wheel
    analogWrite(MOTORA1, 0);
    analogWrite(MOTORA2, SPEEDMOTORA);
    // Right wheel
    analogWrite(MOTORB1, SPEEDMOTORB);
    analogWrite(MOTORB2, 0);

    // update the global speed
    leftSpeed = SPEEDMOTORA;
    rightSpeed = SPEEDMOTORB;
```

```
    //LED
    movingRightLights();

  }
}
void curveLeftAroundItSelf(int distance) {
  CountReset();
  const int SPEEDMOTORA = 0;
  const int SPEEDMOTORB = 255;
  while (motorRight_count <= distance && motorLeft_count <= distance){

    // Left wheel
    analogWrite(MOTORA1, 0);
    analogWrite(MOTORA2, SPEEDMOTORA);

    // Right wheel
    analogWrite(MOTORB1, SPEEDMOTORB);
    analogWrite(MOTORB2, 0);

    // update the global speed
    leftSpeed = SPEEDMOTORA;
    rightSpeed = SPEEDMOTORB;

    // LED
    movingLeftLights();
  }
}

void curveLeft90(int distance) {
  CountReset();
  const int SPEEDMOTORA = 100;
  const int SPEEDMOTORB = 255;
  while (motorRight_count <= distance && motorLeft_count <= distance){

    // Left wheel
    analogWrite(MOTORA1, 0);
    analogWrite(MOTORA2, SPEEDMOTORA);

    // Right wheel
    analogWrite(MOTORB1, SPEEDMOTORB);
    analogWrite(MOTORB2, 0);

    // update the global speed
    leftSpeed = SPEEDMOTORA;
    rightSpeed = SPEEDMOTORB;
```

```
    //LED
    movingLeftLights();
  }
}

float ultraSonicSensor(){    // this will measure the distance to an object

  static unsigned long lastPingTime = 0;
  static float lastValidDistance = -1;
  unsigned long currentTime = millis();
  // only send new ping if the enough time has passed
  if (currentTime - lastPingTime >= 150) { // 150 ms between real pings

    float duration, distance;

    digitalWrite(TRIG, LOW);
    delayMicroseconds(2);
    digitalWrite(TRIG, HIGH);
    delayMicroseconds(10);
    digitalWrite(TRIG, LOW);

    duration = pulseIn(ECHO, HIGH, 30000); // 30ms timeout after that it
retrun 0

    if(duration == 0){
      distance = -1; // no echo
    } else {
      distance = (duration*.0343) / 2; // in cm
    }

    // update the last valid reading and timer is we get a valid value
    if (distance != -1){
      lastValidDistance = distance;
    }
    lastPingTime = currentTime; // update the timer
  }
  return lastValidDistance;
}

void obstacleRoutine() {

  // Step 1: back up slightly
  motorBackward(10);

  // Step 2: initial left turn
  curveLeft90(35);
```

```
  // step 3 : turn right
  curveRight90(60);
  currentState = FOLLOW_LINE;
}

void motorCountleft() {
  motorLeft_count++;
}

void motorCountRight() {
  motorRight_count++;
}
void CountReset(){
  motorLeft_count = 0;
  motorRight_count = 0;
}

// check if a sensor see black or white
bool sensorState(int i){
  int val = analogRead(lineSensorPins[i]);
  return val > thresholds[i];
}
// compute line error using "weighted avg method"
int computeLineError(){
  long weightedSum = 0; // sum of index * sensor value
  long totalWeight = 0; // sum of sensor values

  for (int i = 0; i < 8; i++){
    int val = analogRead(lineSensorPins[i]);

    if (sensorState(i)) {
      weightedSum += i * val; // weight index by intensity
      totalWeight += val; // accumulate weight
    }
  }

if (totalWeight == 0) {
    if (lastError > 0) {
        return -10;   // turn left
    } else if (lastError < 0) {
        return 10;    // turn right
    } else {
        return 0;
    }
}
```

```
  // compute weighted avarage index
  float avgIndex = (float)weightedSum / (float)totalWeight;

  // shift relative to cennter (between sensors 3 and 4)
  int error = (int)(avgIndex - 3.5);

  lastError = error; // remember last correction
  return error;

}
void lineFollowing(){
  // Continuously adjust motor speeds based on line position
  int error = computeLineError();

  const int BASE_SPEED = 220;   // base motor speed it was 150
  const int CORRECTION = 295;    // correction factor *values can be tuned*

  // Adjust speeds based on error
  leftSpeed  = BASE_SPEED - (error * CORRECTION);
  rightSpeed = BASE_SPEED + (error * CORRECTION);

  // Clamp speeds to valid range
  leftSpeed  = constrain(leftSpeed, 0, 220);
  rightSpeed = constrain(rightSpeed, 0, 220);

  // Drive motors with adjusted speeds
  analogWrite(MOTORA1, 0);
  analogWrite(MOTORA2, leftSpeed);
  analogWrite(MOTORB1, rightSpeed);
  analogWrite(MOTORB2, 0);

  // LED
  movingForwardLights();
}

void gripper(int pulse){
  static unsigned long timer; // using static let you initialize the timer to
zero
  static long lastPulse;
  if (pulse > 0) lastPulse = pulse;
  if (millis() > timer) {
    timer = millis() + 20;   // Sets the next time when a pulse should be
sent.
    digitalWrite(GRIPPER, 1);
    delayMicroseconds(lastPulse);
```

```
    digitalWrite(GRIPPER, 0);
  }
}
void runMission(){
  switch (currentState) {
    case STARTING_POINT: {
      motorStop();
      blinkStartingLights();

      float distance = ultraSonicSensor();

      if (distance > STARTING_FLAG ) {
      // when there is not obstacle
      currentState = SEARCH_OBJECT;
      obstacleCounter = 0;
      } else {
        // Obstacle detected → increment counter
        obstacleCounter++;
        if (obstacleCounter >= OBSTACLE_CONFIRM) {
         motorStop();
         obstacleCounter = 0;
        }
      }

      sendTelemetry(botName, leftSpeed, rightSpeed, currentState);
      break;
    }

    case SEARCH_OBJECT: {
      movingForwardLights();
      gripper(GRIPPEROPEN);
      motorForward(startingDistance);
      static int blackCounter = 0;
      // check if all sensors see black
      bool allBlack = true;
      for (int i = 0; i < 8; i++) {
        if (!sensorState(i)){
          allBlack = false;
          break;
        }
      }
      if (allBlack){
        blackCounter++;
      } else {
        blackCounter = 0;
      }
      // check if we see black for x times or more
```

```
      if (blackCounter >= 4){
        motorStop();
        gripper(GRIPPERCLOSE);
        currentState = TURN_LEFT;
        blackCounter = 0; // reset the counter for future  use
      }
      sendTelemetry(botName, leftSpeed, rightSpeed, currentState);
      break;
    }

    case TURN_LEFT: {
      motorStrongStop();
      curveLeftAroundItSelf(curveLeftAroundItSelfDistance);
      currentState = FOLLOW_LINE;
      sendTelemetry(botName, leftSpeed, rightSpeed, currentState);
      break;
    }

    case FOLLOW_LINE: {
      float distance = ultraSonicSensor();

      if (distance > OBSTACLE_THRESHOLD || distance == -1) {
        // No obstacle → reset counter and follow line
        obstacleCounter = 0;
        lineFollowing();

        // --- Black line detection for drop zone ---
        static int blackCounter = 0;
        bool allBlack = true;
        for (int i = 0; i < 8; i++) {
          if (!sensorState(i)){
            allBlack = false;
            break;
          }
        }
        if (allBlack) {
          blackCounter++;
        } else {
          blackCounter = 0;
        }
        if (blackCounter >= 10) {
          currentState = DROP_OBJECT;
          blackCounter = 0;
        }

      } else {
        // Obstacle detected → increment counter
```

```
        obstacleCounter++;
        if (obstacleCounter >= OBSTACLE_CONFIRM) {
          motorStrongStop();
          obstacleRoutine();
          obstacleCounter = 0; // reset after avoidance
        }
      }
      sendTelemetry(botName, leftSpeed, rightSpeed, currentState);
      break;
    }

    case DROP_OBJECT: {
      motorStrongStop(); // for 1 sec stop
      motorBackward(8);
      motorStrongStop();
      gripper(GRIPPEROPEN);
      motorBackward(backwardDistance);
      currentState = DONE;
      sendTelemetry(botName, leftSpeed, rightSpeed, currentState);
      break;
    }

    case DONE: {
      motorStop();

      // Notify master that mission is finished
      Serial.print(botName);
      Serial.println(",DONE");
      sendTelemetry(botName, leftSpeed, rightSpeed, currentState);
      break;
    }
  }
}
void sendTelemetry(String botName, int leftSpeed, int rightSpeed, BotState
state) {
  Serial.print(botName);
  Serial.print(",");
  Serial.print(leftSpeed);
  Serial.print(",");
  Serial.print(rightSpeed);
  Serial.print(",");
  Serial.println(stateToText(state));
}

void blinkStartingLights() {
  unsigned long now = millis();
```

```
  // Blink every 300 ms
  if (now - lastBlinkTime >= 300) {
    lastBlinkTime = now;
    blinkState = !blinkState;  // toggle ON/OFF
  }

  if (blinkState) {
    // ON state

    // BACK LEDs (0,1) = red
    pixels.setPixelColor(0, pixels.Color(0, 255, 0));
    pixels.setPixelColor(1, pixels.Color(0, 255, 0));

    // FRONT LEDs (2,3) = white (GRB format!)
    pixels.setPixelColor(2, pixels.Color(255, 255, 255));
    pixels.setPixelColor(3, pixels.Color(255, 255, 255));

  } else {
    // OFF state
    pixels.clear();
  }
  pixels.show();
}

void movingForwardLights() {
  pixels.clear();
  // Set back LEDs to yellow
  pixels.setPixelColor(0, pixels.Color(255, 255, 0));
  pixels.setPixelColor(1, pixels.Color(255, 255, 0));

  // Set front LEDs to white
  pixels.setPixelColor(2, pixels.Color(255, 255, 255));
  pixels.setPixelColor(3, pixels.Color(255, 255, 255));

  pixels.show(); // Apply the colors
}

void movingLeftLights() {
  unsigned long now = millis();

  // Blink every 300 ms
  if (now - lastBlinkTime >= 300) {
    lastBlinkTime = now;
    blinkState = !blinkState;  // toggle ON/OFF
  }
```

```
  if (blinkState) {
    // ON state

    // BACK LEDs (0,1) = bluesh
    pixels.setPixelColor(0, pixels.Color(0, 255, 255)); // left

    // FRONT LEDs (2,3) = blush (GRB format!)
    pixels.setPixelColor(3, pixels.Color(255, 255, 255)); // left

  } else {
    // OFF state
    pixels.clear();
  }
  pixels.show();
}

void movingRightLights() {
  unsigned long now = millis();

  // Blink every 300 ms
  if (now - lastBlinkTime >= 300) {
    lastBlinkTime = now;
    blinkState = !blinkState;  // toggle ON/OFF
  }

  if (blinkState) {
    // ON state

    // BACK LEDs (0,1) = bluesh
    //pixels.setPixelColor(0, pixels.Color(0, 255, 255)); // left
    pixels.setPixelColor(1, pixels.Color(0, 255, 255)); // right

    // FRONT LEDs (2,3) = blush (GRB format!)
    pixels.setPixelColor(2, pixels.Color(0, 255, 255)); // right
    //pixels.setPixelColor(3, pixels.Color(255, 255, 255)); // left

  } else {
    // OFF state
    pixels.clear();
  }
  pixels.show();
}

void movingBackwordLights() {
  unsigned long now = millis();
```

```
  // Blink every 300 ms
  if (now - lastBlinkTime >= 300) {
    lastBlinkTime = now;
    blinkState = !blinkState;  // toggle ON/OFF
  }

  if (blinkState) {
    // ON state

    // BACK LEDs (0,1) = red
    pixels.setPixelColor(0, pixels.Color(0, 255, 0)); // left
    pixels.setPixelColor(1, pixels.Color(0, 255, 0)); // right

  } else {
    // OFF state
    pixels.clear();
  }
  pixels.show();
}
```

Professional skills:

Reflection on the project:

I worked together with Aisha on our relay bot, which we named **“Unknown.”** The main goal of our bot was to follow the line while carrying an object, avoid obstacles along the way, and finally drop the object at the designated endpoint. Throughout the project, we collaborated closely on every stage — coding, wiring, testing, and debugging — to make sure the bot could perform all of its required tasks.

During the development process, I faced several technical challenges. One of the biggest difficulties was getting the bot to move in a straight line and reliably detect obstacles. Tuning the motor sensors to achieve precise turns was also more complicated than I expected. I solved these issues through repeated testing,

debugging, and adjusting the logic step by step. These challenges taught me that achieving consistent results is difficult when working with hardware, because even small variations can affect performance.

I also gained valuable knowledge while working on the ultrasonic sensor. We used it to detect and avoid obstacles, but at first it was constantly sending signals during line following, which created unnecessary noise and could interfere with other bots. To fix this, I modified the code so the sensor only sent signals every few milliseconds instead of continuously. This reduced the noise and made the bot's behavior more stable.

In terms of team organization, we started the project by working together as one large group on all three relay bots. After about two weeks, we reorganized into three smaller teams of two, with each pair focusing on one specific bot and its unique challenges. This structure helped us concentrate better on the tasks that belonged to our assigned bot.

At the beginning of the project, we stayed on track with all the assignments and communicated well as a group. However, toward the final assignments, we began to experience communication issues, which made it harder to stay aligned and manage the remaining tasks efficiently.

When it came to task distribution within my own team, Aisha and I chose to work on everything together rather than splitting responsibilities. We collaborated on coding, wiring, testing, and debugging as a pair, which helped us stay consistent and understand every part of the bot's development.

Throughout the project, collaboration played an important role in how our team worked. Within my pair, Aisha and I communicated well and supported each other during every stage of the process. We shared ideas, tested solutions together, and helped each other understand the problems we were facing. Working as a pair made it easier to stay focused and solve issues faster because we were always on the same page.

In the larger group, collaboration was strong at the beginning. Everyone contributed, shared updates, and helped each other when needed. However, as the project progressed, communication within the full group became more difficult. Toward the final assignments, we struggled to stay aligned, which made teamwork less efficient and sometimes slowed down our progress.

Despite these challenges, I learned a lot about working with others. I realized how important clear communication is, especially when multiple teams depend on each other. I also learned that supporting teammates and staying open to feedback makes the work smoother and more enjoyable. Overall, the collaboration within my pair was

very effective, and the experience helped me improve my teamwork skills for future projects.

Assignments Card:

NHL STENDEN
university of applied sciences

P2. Embedded Systems

Name	Student nr.	Group	Year
Slman Hamood			

The student is responsible for safe keeping of this card

Assignments

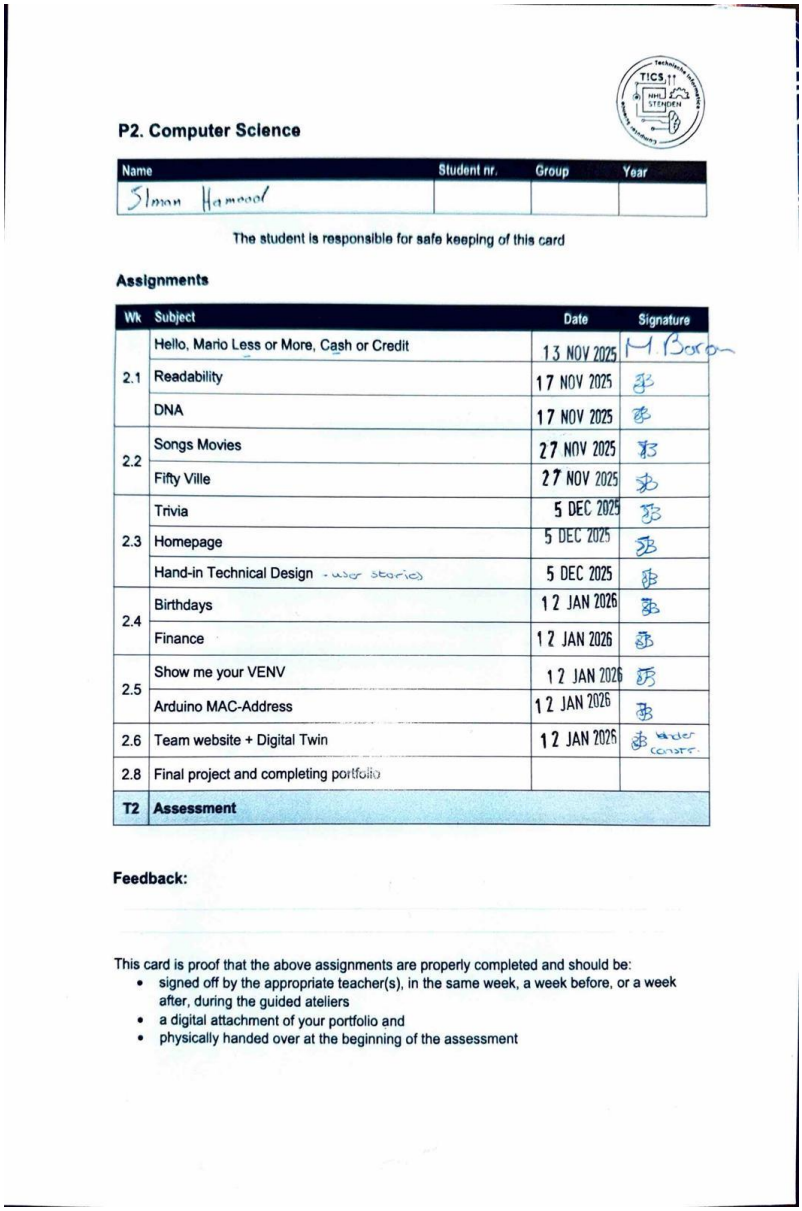
Wk	Subject	Date	Signature
2.1	Basic movements *(Functions)*	13 NOV 2025	Board
2.2	Object avoidance	21 NOV 2025	Board
2.3	Stay on track *(Arrays)*	27 NOV 2025	Board
2.4	**Midterm race** *(subgroup)*	- 3 DEC 2025	Board
2.5	Connectivity *(HC12)*		
2.6	Data storage *(SQL)*		
2.7	**Test Day** + Demo Dashboard *(team)*	5 JAN 2026	M. Boar
2.8	**Race Day** *(Circuit Challenge)* **+ Project Dossier**		
T2	**Assessment**		

Feedback:

** Marcel signed off on wrong page. (CS)

This card is proof that the above assignments are properly completed and should be:

- signed off by the appropriate teacher(s), in the same week during the guided ateliers
- a digital attachment of your personal portfolio and
- physically handed over at the beginning of the assessment



P2. Computer Science

Name	Student nr.	Group	Year
Slman Hamood			

The student is responsible for safe keeping of this card

Assignments

Wk	Subject	Date	Signature
2.1	Hello, Mario Less or More, Cash or Credit	13 NOV 2025	M. Boron
	Readability	17 NOV 2025	
	DNA	17 NOV 2025	
2.2	Songs Movies	27 NOV 2025	
	Fifty Ville	27 NOV 2025	
2.3	Trivia	5 DEC 2025	
	Homepage	5 DEC 2025	
	Hand-in Technical Design - user stories	5 DEC 2025	
2.4	Birthdays	12 JAN 2026	
	Finance	12 JAN 2026	
2.5	Show me your VENV	12 JAN 2026	
	Arduino MAC-Address	12 JAN 2026	
2.6	Team website + Digital Twin	12 JAN 2026	Under construction
2.8	Final project and completing portfolio		
T2	**Assessment**		

Feedback:

This card is proof that the above assignments are properly completed and should be:

- signed off by the appropriate teacher(s), in the same week, a week before, or a week after, during the guided ateliers
- a digital attachment of your portfolio and
- physically handed over at the beginning of the assessment